SCHOOLWAY: PARENT MONITORING SYSTEM FOR STUDENT BUS BOARDING USING NFC AND FLUTTER APP

BY

CHAI BOON HONG

A REPORT

SUBMITTED TO

Universiti Tunku Abdul Rahman

in partial fulfillment of the requirements

for the degree of

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)

(COMMUNICATIONS AND NETWORKING)

Faculty of Information and Communication Technology

(Kampar Campus)

2026

COPYRIGHT STATEMENT

© 2026 CHAI BOON HONG. All rights reserved.

This Final Year Project report is submitted in partial fulfillment of the requirements for the degree of Bachelor of Information Technology (Honours) (Communications and Networking) at Universiti Tunku Abdul Rahman (UTAR). This Final Year Project report represents the work of the author, except where due acknowledgment has been made in the text. No part of this Final Year Project report may be reproduced, stored, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the author or UTAR, in accordance with UTAR's Intellectual Property Policy.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Miss Tan Lyk Yin**, for her guidance, patience, and continuous support throughout the completion of this project. Her advice and encouragement have helped me a lot in developing this project and deepening my understanding of system development, particularly in school transportation management systems. I am truly thankful for the opportunity and support that she has given me throughout this journey.

I would also like to give my heartfelt thanks to Liew Yi Mei for her patience, understanding, and unconditional support during both the good and difficult times. Her encouragement has meant a lot to me throughout the completion of this project.

Finally, I would like to thank my parents and family for their endless love, support, and encouragement. Their belief in me has given me the strength and motivation to complete this project successfully.

ABSTRACT

School transportation management in Malaysia remains heavily reliant on manual processes, leaving parents without real-time visibility into their children's whereabouts and exposing students to safety risks during transit. Existing school bus monitoring solutions mainly focus on GPS-based location tracking, but often lack integrated attendance verification, role-based communication, and school-specific operational workflows. As a result, coordination among parents, drivers, and school administrators remains fragmented, leading to issues such as missed pickups, delayed updates, and uncertainty over whether students have boarded or alighted safely. The objective of this project is to develop SchoolWay, a mobile-based school bus tracking and management system that provides real-time GPS tracking, automated attendance recording via NFC scanning, and instant push notifications to bridge the communication gap between parents, drivers, and administrators. Unlike existing systems that typically provide only location tracking, SchoolWay combines real-time GPS tracking, NFC-based attendance recording, and instant push notifications in a unified platform with role-based access for parents, drivers, and administrators. SchoolWay is built using Flutter for cross-platform mobile development, Firebase as the backend for real-time data synchronisation and authentication, and Google Maps API for live bus location rendering. The system's success will be evaluated through functional testing, user acceptance testing (UAT) with target users, and performance benchmarking of real-time update latency. This project contributes a practical, scalable solution to a gap in school safety infrastructure, empowering parents with transparency, reducing administrative overhead, and ultimately enhancing the safety and accountability of school bus operations in Malaysia.

Area of study: Mobile Application Development, NFC Technology Systems, Real-Time System

Keywords: School Bus Tracking System, Real-Time GPS Tracking, Flutter, Firebase, NFC Technology, Push Notifications, Mobile Application, Role-Based Access Control

TABLE OF CONTENTS

LIST OF FIGURES

Figure Number	Title	Page
START HERE		

LIST OF TABLES

Table Number	Title	Page
START HERE		

LIST OF ABBREVIATIONS

START HERE	

CHAPTER 1

Introduction

In this chapter, the background and motivation of the research are presented, together with the problem statement, project objectives, scope, and expected contributions of the study. This chapter also introduces the significance of the proposed project in addressing current limitations in school transportation management. Finally, the outline of the thesis is provided to give an overview of the structure and organisation of the subsequent chapters.

1.1 Project Background

School transportation is an essential support service in the education sector because it ensures that students travel safely and punctually between home and school. In many schools, transportation management is still conducted using conventional methods such as manual attendance recording, phone calls, and informal communication between parents, drivers, and school administrators [1] . Although these practices are commonly used, they often lead to several operational limitations, including delayed information sharing, inaccurate attendance records, and the absence of real-time monitoring of school bus movements. Therefore, parents may experience anxiety regarding their children's safety, while schools and drivers may face difficulties in maintaining efficient and accountable transportation operations [2].

Recent developments in mobile computing, cloud-based systems, Global Positioning System (GPS) technology, and Near Field Communication (NFC) have created opportunities to improve school transportation management. A mobile-based platform integrated with GPS can provide real-time bus location tracking, enabling parents and school administrators to monitor journeys more effectively [3] . At the same time, NFC-based attendance recording can automate student check-in and check-out processes, thereby reduce human error and improve data accuracy [4] . Cloud services further support instant data synchronization and notification delivery, allowing important updates to be shared immediately with relevant users [2] . These technologies offer clear benefits to users and administrators by improving transparency, reducing manual workload, strengthening communication, and supporting safer transportation management [1].

This project proposes SchoolWay as a mobile-based school transportation monitoring system, implemented through a Flutter mobile application for school bus tracking and student attendance management. The proposed system integrates real-time GPS tracking, NFC-based attendance recording, and cloud-based data synchronization to provide a unified platform for parents, drivers, and school administrators. The significance of the system benefits in its potential to replace inefficient manual practices with a more reliable and systematic digital solution. The expected outcome is a school transportation management system that reduces manual attendance errors through NFC-based check-in/check-out, enables real-time bus location visibility via GPS tracking, and provides immediate status updates through cloud-synchronized notifications. These functions improve parents' ability to verify student boarding status, support administrators in monitoring route and attendance records, and help drivers coordinate pickups and drop-offs more accurately.

1.2 Problem Statement and Motivation

School transportation is an essential support service within the education sector, as it ensures that students are conveyed safely, punctually, and consistently between home and school. In parallel with the advancement of digital technologies, parents and school authorities now place greater emphasis on service transparency, timely information, and operational reliability. Nevertheless, many school transportation services continue to depend on conventional and largely manual practices, resulting in a mismatch between parents' expectations and the actual level of service provided. This situation underscores the need to examine the limitations of existing school bus management practices and to justify the development of a more systematic and technology-driven solution [5] .

A major issue in current school transportation management is the absence of real-time visibility of school bus movements. In many cases, parents and school administrators rely on phone calls or text-based communication to obtain updates about bus location, arrival times, or unexpected delays. Such methods are inefficient, inconsistent, and highly dependent on the immediate availability of the driver. This lack of accurate and timely tracking information can create anxiety among parents, reduce confidence in the transportation service, and limit the ability of school administrators to monitor daily operations effectively [6], [7]. If this issue remains unresolved, the safety and accountability of student transportation may be adversely affected. Therefore, a real-time bus tracking mechanism is necessary to improve transparency, responsiveness, and user trust.

Another significant problem is the continued use of manual methods for recording student attendance during boarding and drop-off. Practices such as handwritten attendance lists or verbal confirmation are susceptible to human error, incomplete documentation, and delayed updates [8]. Therefore, uncertainty may arise regarding whether a student has successfully boarded the bus, arrived at school, or returned home. Such inaccuracies may compromise student safety and lead to misunderstandings between parents, drivers, and school management. If left unaddressed, these weaknesses may undermine confidence in the transportation process and increase administrative difficulties in verifying student movement. Hence, an automated attendance mechanism is important to ensure greater accuracy, reliability, and efficiency in student monitoring.

A further concern is the lack of an integrated communication platform connecting parents, drivers, and school administrators. Existing communication practices are often fragmented, informal, and dependent on individual initiative rather than a centralised system. Consequently, important updates such as route changes, delays, emergencies, or student boarding status may not be communicated promptly or uniformly to all relevant parties [4]. This communication gap can result in confusion, delayed decision-making, and reduced service effectiveness. If this condition persists, coordination among parents and school administrators will remain inefficient, and parental confidence in school transportation services may continue to decline. For this reason, a mobile-based platform with centralised information sharing and instant notifications is essential to support timely communication and more effective coordination.

In summary, the lack of real-time bus tracking, the reliance on manual attendance recording, and the absence of efficient communication mechanisms represent critical weaknesses in existing school transportation management [5]. If these issues are not addressed, student safety may be placed at risk, parents may continue to experience uncertainty and anxiety, and school authorities may face ongoing operational inefficiencies. These concerns collectively justify the need for a digital and integrated solution. Accordingly, the proposed SchoolWay mobile application is intended to address these limitations by providing real-time tracking, automated attendance management, and improved communication among the users, thereby enhancing the safety, efficiency, and reliability of school transportation services [8].

1.3 Project Objectives

Based on the background and problem statement discussed in the preceding sections, this mobile application establishes a set of objectives to guide the development of a systematic solution for the identified shortcomings in school transportation management. The issues previously highlighted, namely inefficient information sharing among parents and school administrators, dependence on manual student attendance recording, and the lack of real-time visibility of school bus movements, indicate the need for a mobile-based system that can improve operational efficiency, monitoring capability, and student safety [5]. In line with the overall aim of the system, the following objectives have been formulated to provide clear direction for the design and implementation of SchoolWay.

The first objective of this project is to design and develop a mobile-based school transportation monitoring system, through a Flutter mobile application with parent, driver, and administrator modules to support student bus activity monitoring and efficient information sharing among users [9]. This objective is significant because it establishes the core platform through which the needs of the three main user groups can be addressed within a unified mobile application. By providing dedicated modules for parents, drivers, and administrators, the application is expected to improve information accessibility and support real-time communication through centralized in-app updates and instant notifications on delays, route changes, and student boarding status, thereby strengthening stakeholder coordination. The successful achievement of this objective can be demonstrated through the completion of a functional mobile-based school transportation monitoring system, through a Flutter mobile application containing the specified user modules and essential features within the defined project scope.

The second objective of this project is to implement and validate NFC-based automated student identification for real-time boarding and exiting event logging [10]. This objective directly addresses the limitation of manual attendance recording, which is often associated with inaccurate, incomplete, or delayed records. Through the application of NFC technology, student identification can be performed in a more efficient and reliable manner, thereby enabling the automatic recording of boarding and exiting events during daily transportation operations. The accomplishment of this objective may be measured through the system's ability to capture and store attendance events accurately, as well as through validation testing to verify the effectiveness and consistency of the NFC-based identification mechanism.

The third objective of this project is to develop and integrate a GPS-based real-time bus tracking module into the mobile application, providing continuous location updates to parents and administrators [5]. This objective is essential in addressing the lack of real-time visibility of school bus movements, which often contributes to parental uncertainty and difficulties in administrative supervision. By integrating GPS tracking functionality into the application, the mobile application is expected to provide timely and continuous bus location information, thereby enhancing transparency and improving the overall safety and accountability of school transportation services. The achievement of this objective can be evaluated through the successful implementation of live location updates and the accurate display of bus movement information within the application interface.

In conclusion, the objectives of this project provide a clear and structured framework for the development of SchoolWay as a mobile-based school transportation management solution. Collectively, these objectives address the principal operational and safety issues identified in the earlier sections and reflect the intended contribution of the mobile application to supporting real-time bus location tracking, automated NFC-based attendance logging for student boarding and exiting, and timely communication among stakeholders through instant notifications. Furthermore, the objectives are designed to be specific, measurable, achievable, relevant, and realistic within the allocated mobile application duration, thereby ensuring their suitability for the final year project report.

1.4 Project Contribution

The proposed SchoolWay mobile application contributes to the improvement of school transportation management by introducing a mobile-based digital solution that addresses the key limitations of existing manual practices. In particular, the mobile application is intended to enhance communication among users, improve the accuracy of student attendance recording, and provide real-time visibility of school bus movements [6]. By aligning its contributions with the identified problem statements and project objectives, SchoolWay is expected to offer practical value to parents, drivers, and school administrators while supporting safer and more efficient transportation operations.

The first contribution of this mobile application is the provision of an integrated mobile application with dedicated parent, driver, and administrator modules. This contribution directly addresses the problem of inefficient information sharing and fragmented communication among users [5]. Through a unified platform, relevant users can access student bus activity

information in a more organised and timelier manner. As a result, communication delays can be reduced, coordination can be improved, and parent and school administrator can rely on a more systematic method of receiving and sharing transportation-related updates. This contributes to greater operational efficiency and strengthens confidence in the management of school bus services.

The second contribution of the mobile application is the implementation of NFC-based automated student identification for real-time boarding and exiting event logging [11]. This contribution is significant in addressing the reliance on manual attendance recording, which is often associated with inaccuracies, incomplete records, and difficulties in verification. By automating the identification process, the mobile application can generate more reliable attendance data and reduce the likelihood of human error during daily transit operations. The expected benefit of this contribution is improved accuracy in student movement records, which in turn supports better safety monitoring, easier record validation, and reduced administrative burden for drivers and school staff [6].

The third contribution of the mobile application is the integration of a GPS-based real-time bus tracking module into the mobile application. This contribution responds directly to the problem of limited real-time visibility of school bus locations [12]. Through continuous location updates, parents and school administrators can monitor bus movements more effectively and obtain timely information regarding travel progress, delays, or route status. This improves transparency in school transportation services and helps reduce parental anxiety concerning student travel. In addition, real-time tracking supports stronger accountability and enables more effective oversight of transportation operations by school authorities [5].

In summary, the contributions of SchoolWay include improvements in the communication between parents and school administrators through an integrated mobile platform, increasing the reliability of attendance management through NFC-based automation, and enhancing transportation visibility through GPS-based real-time tracking. Collectively, these contributions address the main weaknesses identified in the problem statement and support the overall aim of the mobile application, which is to provide a safer, more efficient, and more transparent school transportation management system [11]. Through these contributions, the mobile application is expected to deliver both practical and academic value within the context of mobile-based school transport solutions.

1.5 Project Scope

The project scope defines the boundaries of the proposed mobile application by identifying what is included and excluded from development. A clear scope ensures the mobile application remains focused, achievable within the allocated timeline, and aligned with the project objectives. For SchoolWay, the scope is centred on a mobile-based school transportation monitoring system developed using Flutter. The mobile application addresses key issues such as inefficient communication between parents and school administrators, manual student attendance recording, and limited visibility of school bus movement.

Inclusions

1. This project develops a Flutter-based mobile application as the primary platform for the SchoolWay system.
2. The mobile application provides three user roles, namely parent, driver, and administrator, with role-based access and module-specific functions.
3. The application supports student transportation activity monitoring, including attendance and trip-related status information.
4. The system includes an NFC-based student identification workflow for recording boarding and alighting events within the application data flow.
5. The mobile application supports GPS-based bus location tracking with real-time location display for relevant users through Firebase-backed updates.
6. The system enables real-time information sharing among users through a centralized cloud-connected mobile platform.
7. The system stores and manages data digitally, including student records, boarding event logs, route sessions, emergency alerts, and GPS tracking data.
8. The system includes functional testing and validation through defined test guides and checklists to verify core module behaviour.

Exclusions

1. The mobile application does not include online payment gateway integration, as financial transaction processing is outside the core transportation monitoring objective.
2. The mobile application does not include AI-based prediction or analytics features such as route optimization, forecasting, or delay prediction.

3. The mobile application does not include web deployment as a target deliverable; the system scope is limited to mobile use despite framework-generated web scaffolding.
4. The mobile application does not include multi-school or large-scale fleet management; it is limited to a prototype context suitable for the system setting.
5. The mobile application does not include advanced communication features such as in-app chat, chatbot support, or voice-based communication.
6. The mobile application does not include custom hardware development and relies on existing mobile device capabilities such as NFC and GPS.
7. The mobile application does not cover full live production rollout; the emphasis is on design, development, integration, and prototype-level validation within the project duration.

In summary, SchoolWay is scoped as a mobile-based school transportation monitoring system, implemented through a Flutter mobile application with parent, driver, and administrator modules, NFC attendance workflow, and GPS tracking support. The scope is intentionally constrained to core monitoring and attendance objectives while excluding advanced, non-essential, or large-scale production features to keep development manageable and aligned with the project timeline.

1.6 Report Organization

CHAPTER 2

Literature Reviews

2.1 Review of Technologies

This section reviews the technologies and technical approaches relevant to the development of **SchoolWay**, a mobile-based school transportation monitoring system implemented through a Flutter mobile application. The review covers the application domain, mobile platforms and development framework, attendance and identification technologies, real-time tracking methods, database and backend services, and map and notification services. Each technology is introduced and evaluated in terms of its strengths and weaknesses to justify the selections made for the proposed mobile application. Through this review, the chapter establishes the technical foundation of SchoolWay and explains why specific technologies are more suitable than their alternatives for the intended project scope.

2.1.1 Field of Study: School Transportation Management

School transportation management refers to the planning, monitoring, and administration of student transport services between home and school. It is a critical operational domain because it directly affects student safety, punctuality, and the overall confidence of parents and school administrators in school transport services [13]. In many educational environments, especially where students depend on buses or vans for daily travel, transportation is not merely a logistical service but an extension of the school's duty of care. Effective transportation management contributes to timely arrivals, reduced parental uncertainty, improved route coordination, and better accountability in student movement [14].

Traditionally, school transportation management has relied heavily on manual practices. Parents often depend on phone calls or informal messaging to confirm whether a bus has arrived, while drivers or attendants may record attendance manually using paper-based lists or verbal checks. These approaches are simple to implement, but they are also limited in accuracy, speed, and transparency [15]. Manual records are vulnerable to miss and human error, and conventional communication methods do not provide real-time visibility into bus location or student boarding status [16]. As a result, delays, uncertainty, and poor coordination can arise, particularly when the transport schedule changes unexpectedly.

The growing availability of smartphones, mobile internet access, and cloud-based systems has created a strong case for digital transformation in this field. A mobile-based school transportation system can centralize communication, automate attendance capture, and provide live location updates to relevant users [16]. Such a solution is particularly valuable because it serves the needs of multiple user groups simultaneously, including parents, drivers, and school administrators. Therefore, the field of school transportation management provides an important and practical context for the development of SchoolWay as a digital mobile application that addresses operational inefficiencies and enhances student safety.

2.1.2 Mobile Platforms and Development Framework

Mobile applications are typically delivered through operating systems such as Android and iOS or built using cross-platform frameworks that allow a single codebase to target multiple operating systems. The choice of platform and framework significantly affects project cost, development speed, hardware access, user reach, maintainability, and long-term scalability. Since SchoolWay is intended to provide mobility, real-time updates, and interaction with device features such as GPS and NFC, the selection of an appropriate mobile platform and development framework is a critical architectural decision.

Native development on Android or iOS offers deep access to device features and high platform-specific performance. However, building separate native applications for each platform often increases the development workload and introduces duplication in implementation and maintenance. Cross-platform frameworks, by contrast, aim to reduce development overhead while keeping acceptable performance and access to native capabilities [17]. In the context of a university project with a defined scope and duration, a cross-platform framework is often more practical if it can support the required device integrations.

2.1.2.1 Android

Android is the most widely adopted mobile operating system globally and is known for its open ecosystem, broad device support, and strong integration with mobile hardware features. From a development perspective, Android provides direct access to sensors and communication capabilities such as GPS, Bluetooth, NFC, and notifications, making it well suited for location-aware and identification-based applications.

A major strength of Android is its flexibility. Developers can deploy applications across a wide range of device types and price points, which improves accessibility for both

institutional users and individual users. Android also offers comparatively fewer restrictions on hardware access than some competing platforms, which is beneficial for applications that rely on real-time tracking and NFC interactions. In addition, many lower-cost smartphones used in operational settings, such as devices assigned to drivers, are Android-based.

Despite these strengths, Android also presents weaknesses. Device fragmentation is a persistent challenge because different manufacturers and operating system versions may behave differently, increasing testing and maintenance effort [18]. User interface consistency can vary across devices, and application behaviour may be affected by battery optimization settings or manufacturer-specific restrictions. As a result, Android is highly capable but can require careful compatibility management [19].

2.1.2.2 iOS

iOS is Apple's mobile operating system and is recognized for its high performance, polished user experience, strong security model, and consistent device ecosystem. For application developers, iOS offers stable platform behaviour, predictable hardware-software integration, and high standards of usability and interface quality [20].

One of the main strengths of iOS is platform consistency. Since Apple controls both the hardware and operating system, applications can be optimized for a relatively limited range of devices. This reduces fragmentation and can simplify quality assurance. iOS also has a strong reputation for security and privacy, which is valuable in applications handling student-related data and location information [21].

However, iOS is less flexible than Android in several respects. The ecosystem is closed, development and testing require Apple-specific tools and hardware, and access to certain device functions may be more tightly controlled. In addition, iOS devices are often more expensive, which can limit accessibility for deployment scenarios that depend on dedicated operational devices. For features such as NFC, support is available but can be more constrained depending on the use case and the device environment. Consequently, iOS is robust and secure, but it may not be the most practical primary target in a cost-sensitive project requiring broad hardware flexibility.

2.1.2.3 Flutter

Flutter is a cross-platform application development framework developed by Google. It enables developers to build mobile applications using a single codebase, primarily in the Dart

programming language, and deploy them across platforms such as Android and iOS. Flutter has gained popularity because it combines cross-platform productivity with strong interface rendering performance and an expressive widget-based design model [22].

The primary strength of Flutter is development efficiency. A shared codebase reduces duplication across platforms, shortens implementation time, and simplifies maintenance. This is particularly helpful in a final year project, where development time is limited and the emphasis is on delivering a functional system prototype. Flutter also offers consistent UI rendering across devices and a rich library of widgets that support the development of modern, responsive user interfaces. In addition, Flutter can access device capabilities such as GPS, networking, notifications, and NFC through plugins and platform channels [23].

Its weaknesses are mainly related to ecosystem dependence and feature-specific integration. Although Flutter supports many mobile capabilities, some advanced device functions may depend on third-party plugins or platform-specific adjustments. Application size can also be larger compared to some native implementations, and troubleshooting platform-specific issues may still be necessary when using hardware-dependent features. Nevertheless, these limitations are generally manageable within the scope of SchoolWay.

2.1.2.4 Summary

The comparison of Android, iOS, and Flutter shows that each option offers distinct advantages. Android is highly flexible and provides broad access to hardware features, but device fragmentation increases maintenance complexity. iOS offers strong consistency, security, and performance, but it is more restrictive and can raise development and deployment costs. Flutter, while not an operating system, provides a practical cross-platform framework that allows a single codebase to serve multiple mobile environments while still supporting the required features through available plugins and integrations.

Table 2.1 Comparison of Android, iOS, and Flutter

Technology	Type	Strengths	Weaknesses
Android	Mobile operating system	Broad market reach, flexible hardware access, strong support for GPS and NFC, cost-effective device ecosystem	Device fragmentation, inconsistent behaviour across manufacturers, higher compatibility testing effort
iOS	Mobile operating system	Stable ecosystem, strong security, consistent performance, polished user experience	Closed ecosystem, higher device cost, more restricted hardware flexibility in some cases
Flutter	Cross-platform framework	Single codebase, faster development, consistent UI, good performance, plugin support for mobile features	Reliance on plugins, platform-specific tuning may still be needed, larger app size in some cases

In summary, **Flutter** is selected for SchoolWay because it offers the best balance between development efficiency, cross-platform capability, maintainability, and functional suitability [24] . Since the system requires a unified application with parent, driver, and administrator modules, Flutter supports rapid development of a single mobile solution without the need to maintain separate native applications. It also provides sufficient integration capability for GPS, Firebase, notifications, and NFC-based attendance features, making it the most appropriate framework for the system.

2.1.3 Student Attendance and Identification Technologies

Student attendance and identification are central to the SchoolWay mobile application because the project aims to automate the recording of boarding and exiting events [4]. In transportation settings, an effective attendance technology must be accurate, fast, easy to use, and practical in real-world boarding conditions. Several approaches can be considered, ranging from purely manual methods to contactless digital identification mechanisms such as QR codes, RFID, and NFC [7].

The selection of an identification technology should consider more than technical functionality alone. It should also reflect operational constraints such as scanning speed, physical proximity, reliability in mobile environments, accidental misreads, ease of deployment, and suitability for school-aged users. For this reason, a comparison of alternative attendance approaches is necessary before selecting the most appropriate method for SchoolWay.

2.1.3.1 Manual Attendance Recording

Manual attendance recording typically involves checking student names from a printed list, marking attendance by hand, or using verbal confirmation during boarding and drop-off. This method has been widely used because it is simple, requires no specialized equipment, and can be implemented immediately without software infrastructure [4].

Its primary strength lies in low cost and ease of use. Since it does not depend on network access, mobile devices, or scanning technology, manual attendance can be performed in nearly any environment. It also allows human judgment in situations where student identity needs to be confirmed directly.

However, the weaknesses of manual attendance are large. This approach is prone to human error, illegible records, delayed updates, and inconsistent information [4] . It is also difficult to share in real time with parents or administrators and creates unnecessary administrative workload for drivers or transport staff. In the context of school transportation, where timely and accurate confirmation is important, manual attendance is inadequate for scalable and accountable operations [4], [7].

2.1.3.2 QR Code-Based Attendance

QR code-based attendance uses printed or digital QR codes that are scanned by a mobile device camera to identify a student and record an event [15]. This method is widely used in educational and event management systems because QR codes are inexpensive to generate and easy to distribute [15].

The strengths of QR code systems include low implementation cost, minimal hardware requirements beyond a smartphone camera, and ease of integration into software applications. QR codes can also encode unique identifiers and work well for check-in workflows where controlled scanning conditions are available [15].

Nonetheless, QR-based attendance has practical weaknesses in transportation environments. Camera-based scanning may be slower than tap-based approaches, especially when lighting conditions are poor, camera focus is inconsistent, or printed codes are damaged [15]. Students may also need to position their code correctly, which can create queueing during boarding. In addition, QR codes can be shared or duplicated more easily than short-range tap interactions, which may reduce security in some situations. Therefore, while QR codes are practical in many settings, they are not always ideal for fast and consistent bus boarding workflows.

2.1.3.3 RFID-Based Attendance

RFID-based attendance relies on radio frequency identification tags and readers to detect a student's presence or identity [7]. RFID systems are widely used in access control, inventory management, and automated check-in scenarios. In school environments, RFID cards or tags can be assigned to students and detected by a reader mounted near an entrance [7].

A key strength of RFID is its speed and contactless operation. It can support automated identification with minimal user interaction and is effective in environments where repeated, routine scanning is required. Depending on the type of RFID system used, tags can be read quickly and from varying distances [7].

However, RFID also has limitations. Dedicated RFID readers are typically required, increasing hardware cost and integration complexity [7]. Longer reading ranges, while useful in some contexts, may lead to unintended reads if multiple tags are detected at once. This can reduce precision in transportation use cases where intentional confirmation of boarding or exiting is preferred [7]. RFID systems may therefore be effective but less practical for a mobile-

app-centred system that aims to leverage standard smartphone capabilities rather than additional hardware infrastructure.

2.1.3.4 NFC-Based Attendance

NFC-based attendance uses Near Field Communication to identify a student when an NFC-enabled tag or card is tapped against a compatible mobile device [25], [26]. NFC is a short-range communication technology designed for close-proximity interaction, which makes it suitable for intentional, secure, and quick event logging [26].

Its strengths are closely aligned with the needs of SchoolWay. NFC interactions are fast, simple, and user-friendly, requiring only a short intentional tap [25] . The short communication range reduces the likelihood of accidental reads and supports clearer event confirmation than longer-range methods [26]. NFC can also be implemented using widely available Android devices equipped with NFC capability, which reduces the need for separate dedicated readers in a prototype environment [25]. In a boarding process, this makes NFC efficient and operationally suitable [25].

The weaknesses of NFC include dependency on compatible devices and the need to manage tags or cards assigned to students. Some mobile platforms or device models may impose restrictions on NFC usage, and deployment must account for hardware compatibility. Even so, in a system focused on controlled attendance capture during transport, NFC provides a strong balance between usability, accuracy, and implementation feasibility [26].

2.1.3.5 Summary

The comparison of attendance technologies shows that manual attendance is the least efficient and least reliable option, despite its simplicity. QR code-based attendance is inexpensive and easy to deploy, but camera-based interaction can slow down boarding and is affected by environmental conditions. RFID offers fast contactless identification but often requires dedicated reader hardware and may be less precise if tags are read unintentionally. NFC provides a more deliberate short-range interaction, making it especially suitable for recording bus boarding and exiting events in a controlled, intentional manner.

Table 2.2 Comparison of Manual, QR, RFID, and NFC attendance methods

Method	Strengths	Weaknesses	Cost or Complexity
Manual Attendance	Very low cost, simple to implement, no hardware dependency	Error-prone, slow, not real-time, poor transparency, difficult to share instantly	Low
QR Code	Low cost, easy to generate, smartphone camera compatible, simple software integration	Slower scanning, affected by lighting and print quality, easier to duplicate/share	Low to Moderate
RFID	Fast, contactless, efficient for repetitive identification	Requires dedicated readers, higher hardware cost, possible unintended reads	Moderate to High
NFC	Fast tap-based interaction, short-range precision, low accidental read risk, suitable for mobile integration	Requires NFC-capable device and student tags/cards, platform compatibility considerations	Moderate

In summary, **NFC** is selected for SchoolWay because it offers the most appropriate balance of speed, intentionality, accuracy, and integration practicality. Unlike manual recording, it automates event capture. Unlike QR codes, it does not depend on camera conditions. Unlike broader-range RFID approaches, it supports controlled close-range interactions that are well suited to boarding and exiting workflows. For these reasons, NFC is the most suitable attendance and identification technology for the proposed system.

2.1.4 Real-Time Tracking Technologies

Real-time tracking technologies are essential to SchoolWay because one of the main objectives of the mobile application is to provide continuous bus location updates to parents and administrators [14]. An effective tracking technology must be capable of supporting route-scale movement visibility, timely updates, and integration with mobile applications. Since school transportation occurs primarily in outdoor road environments, tracking must also remain meaningful over relatively large geographical areas rather than short indoor distances.

Several technologies can be used to monitor movement or location, including GPS-based tracking, GSM or SMS-assisted tracking, and proximity-based methods such as Wi-Fi or Bluetooth. These technologies differ in accuracy, communication model, infrastructure requirements, and suitability for real-time monitoring in transportation scenarios [8].

2.1.4.1 GPS-Based Tracking

GPS-based tracking uses satellite positioning to determine the geographic location of a device. It is widely used in navigation, fleet management, transportation monitoring, and ride-sharing services [14]. In mobile applications, GPS can be combined with network connectivity to transmit device coordinates to a backend service, where the location can then be visualized on a map for end users [10].

The major strength of GPS is its suitability for outdoor location tracking across large distances [8]. It can provide continuous position updates as a bus move along its route and supports map visualization, route awareness, and estimated movement monitoring. Since smartphones commonly include GPS hardware, implementation is practical in mobile-based systems without specialized external tracking devices [4].

Its weaknesses include battery consumption, signal limitations in tunnels or dense urban environments, and dependence on internet connectivity for real-time transmission [27]. GPS may also require location permission handling and optimization to balance accuracy against battery usage. Despite these considerations, GPS remains the standard and most suitable approach for route-scale outdoor transport tracking [10].

2.1.4.2 GSM / SMS-Based Tracking

GSM or SMS-based tracking generally uses cellular communication to send location-related information or status updates from a device to another system or user [7]. In some implementations, GSM is combined with GPS, where GPS obtains coordinates and GSM or SMS is used as the communication channel to transmit them [5].

The strength of GSM or SMS-based tracking is its ability to operate in environments where internet-based data services may be limited or unreliable. SMS notifications can be useful for simple event-based status alerts, such as notifying a parent that a bus has departed or reached a checkpoint [7]. This method can also be easier to understand in low-bandwidth settings.

However, GSM or SMS alone is not ideal for rich, continuous real-time tracking. It does not inherently provide map-based live visualization and is less efficient for frequent updates. Costs may accumulate if message-based communication is used repeatedly, and the user experience is less interactive than that of an application displaying live movement on a map. Therefore, while GSM or SMS is useful as a communication support mechanism, it is less suitable as the primary tracking model for SchoolWay [5].

2.1.4.3 Wi-Fi / Bluetooth-Based Proximity Tracking

Wi-Fi and Bluetooth-based proximity tracking are commonly used to determine whether a device is near a known access point, beacon, or another device [28]. These approaches are well suited to indoor localization, room-level presence detection, or checkpoint-based monitoring in controlled environments.

Their strengths include low-power operation in some configurations and usefulness for short-range proximity detection. Bluetooth beacons, for example, can help determine whether a device has entered or left a specific zone [28]. In limited spaces, these technologies can support localized presence awareness.

Their weaknesses make them unsuitable as primary bus route tracking technologies. Proximity-based systems do not provide continuous geographic coordinates over long outdoor routes and usually depend on additional infrastructure such as beacons or known networks. They are better suited to location approximation near fixed points than to full route-scale transport visibility [28]. Consequently, their role in SchoolWay, if any, would be secondary rather than central.

2.1.4.4 Summary

The comparison of tracking technologies shows that GPS-based tracking is the most appropriate method for outdoor, route-based transportation monitoring. GSM or SMS can support simple event transmission but does not provide an adequate experience for live tracking. Wi-Fi and Bluetooth are useful for proximity detection in controlled spaces but are not designed for continuous road-based monitoring. Since SchoolWay requires parents and administrators to view ongoing bus movement in real time, GPS clearly provides the strongest functional fit.

Table 2.3 Comparison of GPS, GSM/SMS, and Wi-Fi/Bluetooth tracking methods

Technology	Strengths	Weaknesses	Best Use Case
GPS-Based Tracking	Accurate outdoor positioning, supports map visualization, widely available on smartphones, suitable for continuous movement tracking	Battery usage, signal limitations in some environments, requires network transmission for real-time sharing	Route-scale outdoor transport monitoring
GSM / SMS-Based Tracking	Can operate in basic communication environments, useful for event-based alerts, simple status delivery	Not ideal for live map-based tracking, less interactive, repeated message costs may apply	Simple alerts and low-bandwidth updates
Wi-Fi / Bluetooth-Based Proximity Tracking	Useful for short-range detection, suitable for checkpoints or indoor zones, can be low power	Not suitable for long-distance tracking, depends on local infrastructure, does not provide full route coordinates	Indoor or fixed-point proximity monitoring

In summary, **GPS** is selected for SchoolWay because it is the most suitable technology for real-time school bus monitoring over road networks. It provides precise outdoor location data, supports continuous tracking, and integrates naturally with mobile maps and cloud backends. These strengths directly address the system requirement for providing continuous location updates to parents and administrators.

2.1.5 Database and Backend Services

Database and backend services are essential to SchoolWay because the mobile application must store user data, attendance logs, bus location records, and notification-related information [4], [10]. The backend also plays a critical role in synchronization among multiple users, especially when parents, drivers, and administrators need to access updated information at different times and from different devices [5], [12]. The choice of database architecture therefore affects not only storage but also responsiveness, scalability, and ease of integration [14].

For a mobile-based system such as SchoolWay, the backend should support structured data storage, secure access, and timely synchronization [10]. The technologies considered in this review are SQLite, MySQL, and Firebase, each of which reflects a different architectural approach to application data management.

2.1.5.1 SQLite

SQLite is a lightweight relational database engine commonly embedded within mobile or desktop applications [29]. It is designed for local data storage and is often used when an application needs offline persistence without a separate database server [29], [30].

Its main strength is simplicity. SQLite is easy to deploy, fast for local operations, and suitable for storing app data directly on a user's device [29]. It is also efficient for caching, offline data retention, and local transactional storage [30]. Because it does not require a network connection or external server, it is useful for device-level persistence [29].

However, SQLite is not designed to function as a multi-user real-time backend [30]. It does not provide native cloud synchronization, centralized access for multiple users, or built-in authentication [29]. In the context of SchoolWay, where parents, drivers, and administrators require shared access to updated records, SQLite alone is insufficient as the primary data platform [10], [29].

2.1.5.2 MySQL

MySQL is a widely used relational database management system that supports structured data storage, server-side access, and multi-user applications [30]. It is commonly used in web and enterprise systems where a centralized database server is required [30].

Its strengths include maturity, reliability, strong relational modelling, and support for structured queries [30]. MySQL is suitable for systems with clearly defined schemas and can

serve as the backend foundation for complex applications when combined with a server-side application layer [30]. It also supports data consistency and administrative control effectively.

Its weaknesses for SchoolWay relate mainly to integration complexity. Using MySQL typically requires a separate backend application or API layer for mobile clients to access and update data securely [30]. This increases development effort and infrastructure setup compared to serverless or integrated backend solutions [30]. Real-time synchronization must also be built explicitly rather than being available by default [30], [31]. Therefore, MySQL is powerful but may be less convenient for a system emphasizing rapid development and real-time mobile interaction [30].

2.1.5.3 Firebase

Firebase is a cloud-based backend platform that provides services such as real-time databases, authentication, cloud messaging, analytics, and file storage [31]. It is widely used in mobile application development because it reduces the need to build and maintain a traditional custom backend from scratch [31].

Its strengths are highly relevant to SchoolWay. Firebase supports real-time data synchronization, which is valuable for updating attendance and bus location data across multiple users [10], [31]. It also provides integrated authentication services, cloud messaging for notifications, and a managed infrastructure that simplifies deployment [31]. Because it is designed to work closely with mobile applications, it reduces backend complexity and accelerates development [31].

Its weaknesses include dependence on a third-party ecosystem, possible cost growth as usage increases, and a less traditional relational data model compared with SQL databases [31]. Developers must also design data access carefully to avoid inefficient reads or writes [31]. Despite these limitations, Firebase provides a strong practical advantage for a prototype mobile application requiring real-time interaction among mobile users [10], [31].

2.1.5.4 Summary

SQLite is effective for local storage but does not meet the centralized multi-user requirements of SchoolWay. MySQL is robust and structured, but it generally requires additional backend services and more implementation effort to support real-time mobile synchronization. Firebase offers direct advantages in real-time updates, authentication, push integration, and rapid development, making it especially appropriate for a mobile transportation management system.

Table 2.4 Comparison of SQLite, MySQL, and Firebase

Technology	Strengths	Weaknesses	Best Use Case
SQLite	Lightweight, fast local storage, simple integration, supports offline persistence	Not centralized, no native real-time sync, unsuitable as shared multi-user backend	Device-level local storage
MySQL	Mature relational model, strong query support, centralized storage, reliable for structured systems	Requires separate backend/API layer, more setup effort, real-time features need additional implementation	Traditional client-server applications
Firebase	Real-time synchronization, integrated authentication, cloud messaging support, managed infrastructure, rapid mobile integration	Vendor dependence, cost scaling considerations, less suited to complex relational models	Real-time mobile and cloud-connected applications

In summary, **Firebase** is selected for SchoolWay because it best satisfies the project's need for centralized storage, real-time synchronization, user authentication, and notification support within a mobile-first architecture. Its managed backend services reduce implementation complexity while enabling the project to focus on core application features such as attendance logging, communication among the user, and bus tracking.

2.1.6 Map and Notification Services

Map and notification services are important supporting components in the development of **SchoolWay** because the mobile application requires both visual representation of bus location and timely communication with users [4]. Since one of the core functions of the proposed application is to provide real-time visibility of school bus movement, an appropriate mapping service is necessary to display location data clearly and accurately [5]. At the same time, a notification service is needed to inform parents and administrators of important transportation events such as student boarding, exiting, and possible delays [10]. The selection of these services must therefore consider integration capability, cost, usability, and suitability for a mobile-based school transportation system [8].

In this project, map service selection is particularly significant because it directly affects the practicality and sustainability of the tracking feature. While commercial mapping solutions offer rich functionality, open-source and flexible alternatives may provide greater affordability and customization for academic and prototype-based systems [32]. Similarly, push notification services must be reliable enough to support real-time communication between the system and its users. For these reasons, this section reviews **Google Maps API**, **OpenStreetMap**, and **push notification services** before justifying the final choice for SchoolWay.

2.1.6.1 Google Maps API

Google Maps API is one of the most widely used mapping platforms for mobile and web applications. It provides interactive maps, geolocation support, markers, route visualization, and location-based services that can be integrated into digital systems. Due to its popularity, Google Maps API is often considered a standard solution for applications requiring map display and navigation-related features [5].

One of its major strengths is its accuracy and maturity. Google Maps API offers reliable map data, familiar visual presentation, and extensive documentation for developers. It also supports real-time map interaction, making it suitable for applications that require users to monitor moving vehicles such as school buses [12]. In addition, the widespread familiarity of Google Maps may improve usability, as many users are already accustomed to its interface and map conventions.

However, Google Maps API also has several limitations. The service is commercial in nature and may involve usage limits, billing requirements, and API quota restrictions, particularly when location requests become frequent. This may create challenges for student

projects or small-scale systems that aim to remain cost-effective. Furthermore, its level of customization may be more restricted when compared to some open-source alternatives. Although Google Maps API is technically strong, these limitations make it less attractive for a project that prioritizes affordability and flexibility.

2.1.6.2 OpenStreetMap

OpenStreetMap (OSM) is a collaborative, open-source mapping platform that provides freely accessible geographic data for use in web and mobile applications [32]. Unlike proprietary map providers, OpenStreetMap allows developers to use and customize map data with greater flexibility, making it an attractive option for educational, research, and prototype-based projects. It has become a widely adopted alternative in systems that require map-based visualization without the cost implications of commercial platforms.

The primary strengths of OpenStreetMap are its openness, cost-effectiveness, and adaptability [33]. Since the platform is community-driven and openly licensed, it is well suited for projects that require a sustainable and low-cost mapping solution. OpenStreetMap also allows greater customization in map styling and data usage, which can benefit systems such as SchoolWay where only specific transport-related visualization is needed. In addition, OpenStreetMap can be integrated into Flutter applications through compatible mapping libraries, making it practical for mobile-based development [24].

Nevertheless, OpenStreetMap also has certain weaknesses. The quality and completeness of mapping data may vary depending on the region, as the data is maintained collaboratively by contributors rather than a single centralized provider [33]. In some locations, it may offer less polished visualization or fewer advanced built-in services compared to commercial platforms. Furthermore, implementation may require more configuration effort depending on the libraries and tools selected. Despite these limitations, OpenStreetMap remains highly suitable for projects that value flexibility, affordability, and open access.

2.1.6.3 Push Notification Services

Push notification services enable applications to deliver real-time alerts and updates directly to users' devices. In the context of SchoolWay, such services are essential because parents and administrators need to be informed promptly about important events, including student boarding, bus departure, route updates, and student drop-off [2]. Without a notification

mechanism, users would need to repeatedly open the application to check for updates, which would reduce convenience and the effectiveness of the mobile application.

A commonly used push notification solution is **Firebase Cloud Messaging (FCM)**. This service supports the delivery of notifications to mobile devices and is widely integrated into modern mobile applications [31]. Its strengths include reliable message delivery, ease of integration with Firebase-based backends, and support for event-triggered communication. This makes it suitable for SchoolWay, especially since the project already adopts Firebase for data storage and synchronization [31]. Through FCM, the mobile application can send immediate alerts when attendance events are recorded or when transport status changes occur.

The main limitations of push notification services are not in their functionality, but in their implementation and management. Notifications must be designed carefully to avoid excessive or unnecessary alerts, which may lead users to ignore them [34]. In addition, permission handling must be managed properly so that users allow the application to receive notifications. Despite these considerations, push notification services remain an essential component for improving user awareness and communication efficiency in a school transportation mobile application.

2.1.6.4 Summary

The comparison of mapping and notification services indicates that both Google Maps API and OpenStreetMap can support location visualisation in mobile applications, while Firebase Cloud Messaging is suitable for real-time alert delivery. Google Maps API offers accurate map information, a familiar interface, extensive documentation, and strong integration support for mobile applications. In addition, it can calculate road-based routes and provide a more accurate Estimated Time of Arrival (ETA) by considering current traffic conditions. However, Google Maps API introduces usage limits, billing requirements, and dependency on a commercial service provider. OpenStreetMap, on the other hand, provides a flexible, open-source, and cost-effective alternative, although additional configuration and external routing services may be required to support route calculation and traffic-aware ETA estimation.

Table 2.5 Comparison of map and notification service options

Service	Category	Strengths	Weaknesses
Google Maps API	Mapping	Accurate maps, familiar interface, strong mobile integration, rich documentation, road-based route calculation, and traffic-aware ETA estimation	Usage limits and billing considerations, third-party dependence
OpenStreetMap	Mapping	Open-source, flexible, customisable, and potentially lower dependence on a commercial provider	May require additional configuration and external services for route calculation and traffic-aware ETA estimation
Firebase Cloud Messaging	Notifications	Real-time push delivery, strong Firebase integration, and broad mobile support	Requires notification permission management and an appropriate notification strategy

In summary, **Google Maps API** is selected as the mapping and route-calculation service for SchoolWay because it provides accurate location visualisation, road-based route information, and a more reliable ETA based on current traffic conditions. These capabilities are important for enabling parents and administrators to monitor school bus journeys and obtain more realistic arrival-time information. Although Google Maps API involves usage quotas and billing considerations, its accuracy, comprehensive documentation, and strong Flutter integration make it suitable for the requirements of SchoolWay. At the same time, **Firebase Cloud Messaging** is selected as the push-notification service because it complements the Firebase backend and enables the timely delivery of important transportation events, such as student boarding and alighting updates, route-session information, and emergency alerts, to authorised parents and administrators.

2.2 Review of Existing Systems / Applications

This section reviews existing systems and applications relevant to school transportation tracking, student attendance monitoring, and integrated transport management. Examining real-world applications is important because it reveals current industry practices, highlights strengths that can inform the proposed design, and identifies limitations that SchoolWay can address. The review focuses on three systems: a school bus tracking application, a student attendance monitoring system, and an integrated school transport management platform. The purpose is not only to describe these systems but also to identify the functional gaps that justify the development of SchoolWay.

2.2.1 Existing System A: School Bus Tracking Application

A suitable example of a school bus tracking application is **SafeStop**, a mobile-focused platform designed to provide parents and schools with visibility into school bus movement and arrival status [35]. SafeStop is known for enabling parents to track bus routes and receive updates associated with daily transportation [35].

The main features of SafeStop include live bus location monitoring, route awareness, estimated arrival support, and status updates through a user-accessible mobile interface [35]. From a user perspective, the system is valuable because it reduces uncertainty around bus arrival and allows parents to make better decisions regarding pickup or readiness [5]. The platform reflects a strong emphasis on real-time transport visibility [35].

Its strengths lie in usability and direct problem relevance [10]. SafeStop addresses one of the most common pain points in school transportation, namely the lack of location transparency [5]. By focusing on route tracking and updates, it improves the parent experience and reduces the need for repeated phone-based inquiries [35]. Its mobile-centred approach also aligns with the practical expectation that transport information should be accessible on demand [10].

However, SafeStop also presents limitations when considered against the broader objectives of SchoolWay. Its functional emphasis is primarily on tracking rather than integrated student attendance management [35]. It does not inherently resolve the challenge of verifying whether a specific student has boarded or exited the bus through an identification mechanism such as NFC [25]. In addition, the system may emphasize parent-facing visibility more than full tri-role coordination among parents, drivers, and administrators within a unified application workflow [10].

The gap revealed by this system is therefore clear: while real-time tracking is valuable, tracking alone does not fully solve the accountability problem in student transportation. SchoolWay extends beyond this model by combining bus location visibility with student event logging and role-based coordination [26].

2.2.2 Existing System B: Student Attendance Monitoring System

A relevant example of a student attendance monitoring system is **SchoolPass**, a platform widely associated with attendance, dismissal, and student movement management in school environments [36]. Although its primary context is broader campus workflow rather than school bus-only operations, it provides a useful reference for digital student status management [36].

The main features commonly associated with SchoolPass include digital attendance processes, dismissal coordination, guardian-related student release workflows, and administrative oversight of student status changes [36]. Its value lies in digitizing workflows that are often handled manually in many institutions [15]. By replacing paper-based records or fragmented procedures with a more structured digital process, the system improves traceability and administrative efficiency [15].

The strengths of SchoolPass include centralization of student status information, improved record visibility, and support for school operational workflows beyond simple manual logging [36]. It demonstrates how digital attendance-related systems can strengthen institutional control, reduce errors, and improve communication around student movement events [15]. The administrative perspective is particularly strong, as the system helps schools standardize how attendance and dismissal information is handled [36].

Its weaknesses, when viewed from the perspective of SchoolWay, are related to transport-specific functionality [10]. A campus attendance and dismissal platform does not necessarily provide real-time bus tracking or route visualization [5]. It may also lack a dedicated driver-oriented operational module or a direct mechanism for recording transportation-specific boarding and exiting events through NFC-based tap interactions [25]. In other words, it improves institutional attendance processes but does not fully address school transport monitoring as an integrated mobile activity [15].

The gap identified here is that attendance monitoring systems may digitize student status effectively but remain disconnected from transport-specific real-time tracking [26].

SchoolWay seeks to address this gap by linking identification-based event capture directly with bus movement visibility and user modules [10], [25].

2.2.3 Existing System C: Integrated School Transport Management System

A strong example of an integrated school transport management system is **Zonar Systems**, which is widely recognized in student transportation and fleet monitoring contexts [37]. Zonar offers solutions related to fleet visibility, route operations, driver support, and transport oversight for educational institutions and other fleet-based environments [37].

The main features associated with Zonar include fleet tracking, route monitoring, operational dashboards, driver-related tools, and transportation management capabilities that support institutional oversight [37]. This makes it more comprehensive than a simple consumer-facing tracking application [14]. It demonstrates how transport technology can support large-scale coordination, visibility, and accountability at the operational level [37].

The strengths of Zonar include broad transport management functionality, strong institutional orientation, and the ability to support administrative oversight across vehicle operations [14]. It is more comprehensive than systems focused solely on parent-facing visibility and shows the value of combining tracking data with operational decision support [37]. Its design perspective is especially relevant to school transportation providers and administrators.

Its weaknesses, relative to SchoolWay, stem from system complexity and focus. Enterprise-grade transport platforms can be more operationally heavy and may not prioritize a lightweight unified mobile experience for parents, drivers, and administrators within a single project-scale application [14]. They may also not focus on simple NFC-based boarding and exiting confirmation at the student level as a core mobile workflow. For a university project targeting direct user communication and student-level event logging, such enterprise systems can be functionally broad but not necessarily optimized for the same interaction model [10].

The gap shown by Zonar is that strong fleet management capability does not automatically imply a simple, student-centric, mobile-integrated system with attendance and parent communication features combined in one coherent application [14]. SchoolWay addresses this by focusing on a narrower but more integrated set of features tailored to daily student transport monitoring [26].

2.2.4 Proposed System and Summary

The review of existing systems demonstrates that each solution addresses an important part of the school transportation problem space, but none fully aligns with the integrated feature set proposed for SchoolWay [10]. SafeStop emphasizes real-time bus tracking and improves parent visibility, but it does not adequately address automated student attendance capture [35]. SchoolPass demonstrates the value of digitized student status and attendance workflows, but it is not primarily designed as a transport tracking system with route-level real-time monitoring [36]. Zonar Systems provides broad transport management capabilities, yet its enterprise orientation does not directly translate into a lightweight mobile application centred on the combined needs of parents, drivers, and administrators at the student-event level [37].

SchoolWay is proposed as a mobile-based system that integrates the strengths of these approaches while addressing their limitations [10], [25]. It includes real-time GPS bus tracking, NFC-based student boarding and exiting logs, role-based modules for parents, drivers, and administrators, push notifications, and centralized data storage [10], [25], [26]. This integration is important because it connects transport visibility, student accountability, and communication among the user within a single mobile application architecture [5]. Rather than solving only one part of the problem, SchoolWay is designed to support a more complete daily transport workflow.

Table 2.6 Comparison of existing systems and the proposed SchoolWay system

System	Mobile-based	Parent module	Driver module	Admin module	NFC attendance	Real-time GPS tracking	Push notification	Centralized data storage
SafeStop	✅	✅	⚠️	⚠️	❌	✅	✅	✅
SchoolPass	✅	⚠️	❌	✅	❌	❌	✅	✅
Zonar Systems	✅	⚠️	✅	✅	❌	✅	✅	✅
SchoolWay (Proposed)	✅	✅	✅	✅	✅	✅	✅	✅
✅ Yes ❌ No ⚠️ Limited								

In summary, SchoolWay builds upon the practical strengths observed in existing systems while addressing the gaps that remain across them [10]. The proposed system combines mobile

accessibility, student-level event logging, user-specific modules, location visibility, and centralized information handling in a single platform [26]. This integrated approach constitutes the main contribution of the proposed system at the design level and justifies its development within the context of school transportation management [14].

2.3 Chapter Summary

This chapter has reviewed the main technologies and existing systems relevant to the development of SchoolWay. The review began with the field of school transportation management, highlighting the limitations of traditional manual practices and the need for a digital mobile solution. It then examined mobile platforms and development frameworks, identifying Flutter as the most suitable choice due to its cross-platform efficiency, consistent interface design, and practical support for the required application features.

The chapter also reviewed attendance and identification technologies, where NFC was selected because it offers fast, intentional, and reliable student event logging for boarding and exiting activities. In the area of real-time tracking, GPS was identified as the most appropriate technology for monitoring bus movement over outdoor routes. For backend services, Firebase was chosen because it provides centralized cloud storage, authentication, and real-time synchronization suitable for a mobile multi-user application. Finally, Google Maps API and Firebase Cloud Messaging were identified as suitable support services for map visualization and push notifications respectively.

The review of existing systems showed that current applications often address only part of the problem space. Some systems focus on bus tracking, others on attendance or campus workflows, and others on broader transport operations. However, the combination of role-based mobile access, NFC attendance logging, real-time GPS tracking, push notifications, and centralized data management remains insufficiently integrated in a single lightweight application. These gaps justify the development of SchoolWay as a focused and practical digital solution for school transportation management.

CHAPTER 3

Proposed Method/Approach

3.1 System Development Models

System development models are important because they provide a structured approach for planning, implementing, and validating software projects. A suitable model helps reduce technical and managerial risks, improves communication among stakeholders, and ensures that deliverables are produced in a controlled and measurable manner throughout the project lifecycle [38], [39].Since different projects have different requirement stability, risk levels, and user involvement needs, this study compares three development models before selecting the most appropriate one for the SchoolWay mobile application.

3.1.1 System Development Model 1: Waterfall Model

The Waterfall model is a linear and phase-based methodology where each phase, such as requirements, design, implementation, testing, and deployment, is completed before moving to the next [38]. It emphasizes strong documentation and clear phase boundaries. Feedback loops are limited, so changes are costly once development has progressed.

Table 3.1.1: Strengths and Weaknesses of Waterfall Model

Strengths	Weaknesses
Clear structure and documentation at every phase.	Low flexibility when requirements change mid-project.
Easier to manage for projects with stable and well-defined requirements.	User feedback usually arrives late, often during testing or deployment.
Milestones and deliverables are straightforward to monitor.	Risk of mismatch between user expectations and final product.

Suitability for this project

Waterfall is only partially suitable because SchoolWay includes evolving requirements such as, role workflows, live tracking behaviour, and UI refinements, which require iterative feedback.

Figure of Model

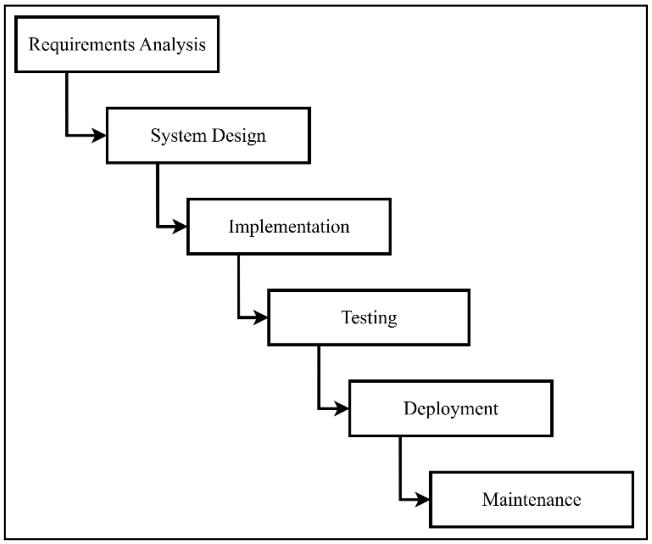


Figure 3.1.1: Waterfall Model Process Flow

3.1.2 System Development Model 2: Agile Model

Agile is an iterative and incremental model where development is delivered in short cycles (sprints), with frequent stakeholder feedback and continuous adaptation [40], [41]. Requirements can be refined during development. Working software is prioritized early and often.

Table 3.1.2: Strengths and Weaknesses of Agile Model

Strengths	Weaknesses
High adaptability to changing requirements.	Requires consistent stakeholder participation and fast decision-making.
Frequent stakeholder involvement improves requirement accuracy.	Scope creep may occur if backlog control is weak.
Early and continuous delivery reduces project risk.	Documentation can become lightweight unless explicitly managed.

Suitability for this project

Agile is suitable because SchoolWay contains multiple user roles such as admin, driver, parent and features that benefit from iterative validation and continuous refinement.

Figure of Model

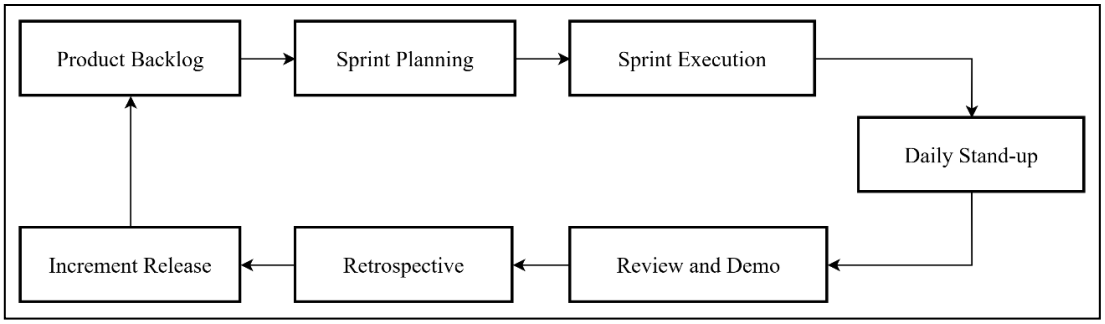


Figure 3.1.2: Agile Iterative Sprint Cycle

3.1.3 System Development Model 3: Prototype Model

The Prototype model develops an early working version of the system to gather user feedback before full-scale implementation [38], [39].The prototype may be revised multiple times until core requirements become clearer. This is useful when user expectations are not fully known at the beginning.

Table 3.1.3: Strengths and Weaknesses of Prototype Model

Strengths	Weaknesses
Improves requirement clarity through early user interaction.	Can increase development time if prototype revisions are excessive.
Reduces risk of building unusable interfaces or workflows.	Users may confuse prototype quality with final system quality.
Particularly effective for UI-intensive or workflow-heavy systems.	May lead to weak architecture if quick fixes are overused.

Suitability for this project

Prototype is highly suitable for SchoolWay because dashboard layout, map-based interactions, and role-specific user experience require repeated usability feedback.

Figure of Model

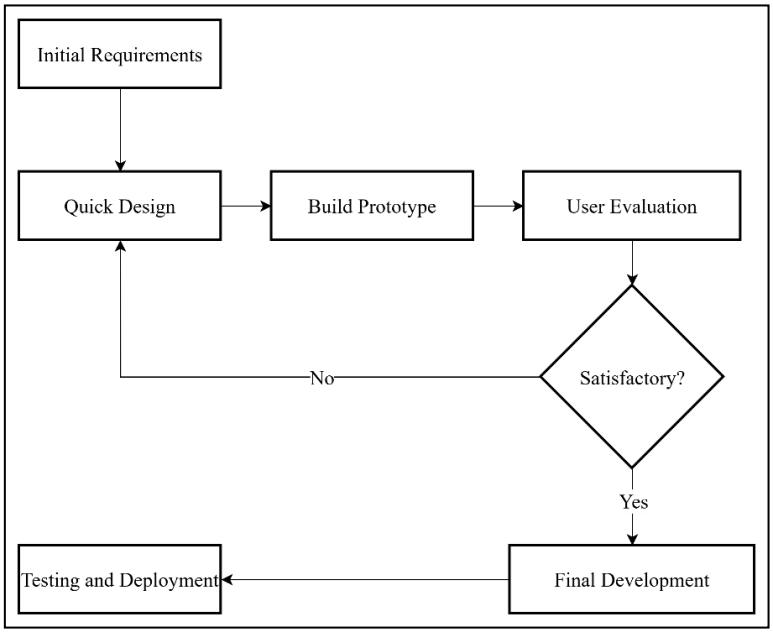


Figure 3.1.3: Prototype Development Cycle

3.1.4 Selected Model: Prototype-Driven Agile Development

The selected approach for this project is a Prototype-driven Agile model. This is because Agile iterations supported by early and repeated prototyping.

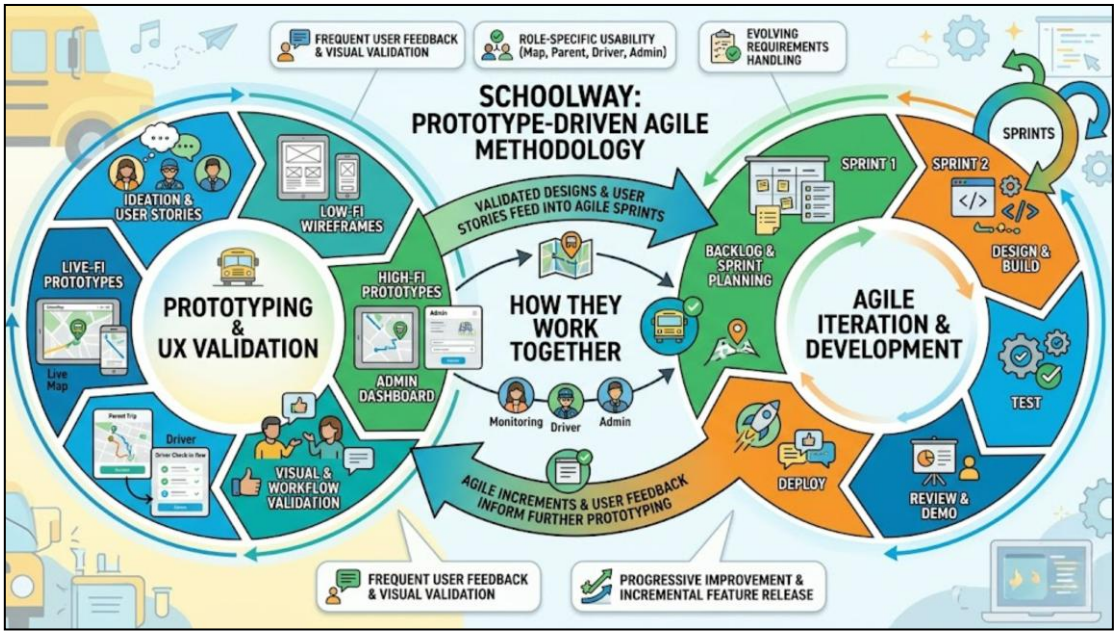


Figure 3.1.4: Prototype-Driven Agile Development

Justification

SchoolWay is a user-facing mobile system with multiple actors and interaction-heavy modules such as live map views, parent trip monitoring, driver check-in flows, and administrative control screens. These characteristics require frequent user feedback, visual validation, and progressive improvement, which are strongly supported by both prototyping and Agile iteration [40], [41].

Compared with this approach, Waterfall is less suitable because it assumes stable requirements and delayed feedback, which does not match a real-time transport monitoring application. Pure Agile is strong for iteration, but adding explicit prototyping further improves UI/UX clarity and workflow validation early in development. Therefore, Prototype-driven Agile best matches project needs which include evolving requirements, role-specific usability concerns, and incremental feature release.

3.2 System Requirement

3.2.1 Introduction

The successful development and deployment of SchoolWay require well-defined system requirements. These requirements are categorized into hardware and software components to ensure the project can be developed, tested, and operated effectively. Hardware requirements ensure sufficient processing, memory, connectivity, and device capability, while software requirements provide the tools, frameworks, and cloud services needed to implement secure authentication, real-time tracking, notifications, and data management.

3.2.2 Hardware Components

The hardware requirements define the essential devices and equipment needed for SchoolWay, including a development computer, Android phone, GPS-enabled device, NFC-capable phone, and stable internet connection. These components provide computing power, connectivity, and user interaction to ensure smooth development, live tracking, and reliable system communication.

Table 3.2.2: Summary of Hardware Components and Minimum System Requirements

Component	Minimum Requirement	Recommended Requirement	Purpose
Developer Laptop/PC	Intel i5 (or equivalent), 8 GB RAM, 20 GB free storage	Intel i7/Ryzen 7, 16 GB RAM, SSD 256 GB+	Used for Flutter development, build, emulator usage, and testing.
Android Test Device	Android 8.0+, 3 GB RAM, GPS support	Android 11+, 4 GB RAM+, NFC + GPS	Used to test parent/driver/admin mobile flows and location features.
Internet Connectivity	Stable 10 Mbps	30 Mbps+	Required for Firebase authentication, Firestore/Realtime Database sync, and push notifications.
GPS Sensor (mobile)	Built-in GPS	High-accuracy GPS + mobile data	Required for real-time bus location update and map rendering.
NFC-capable Phone (driver use case)	Android with NFC hardware	Android with NFC + secure element support	Needed for NFC-based attendance/check-in features.

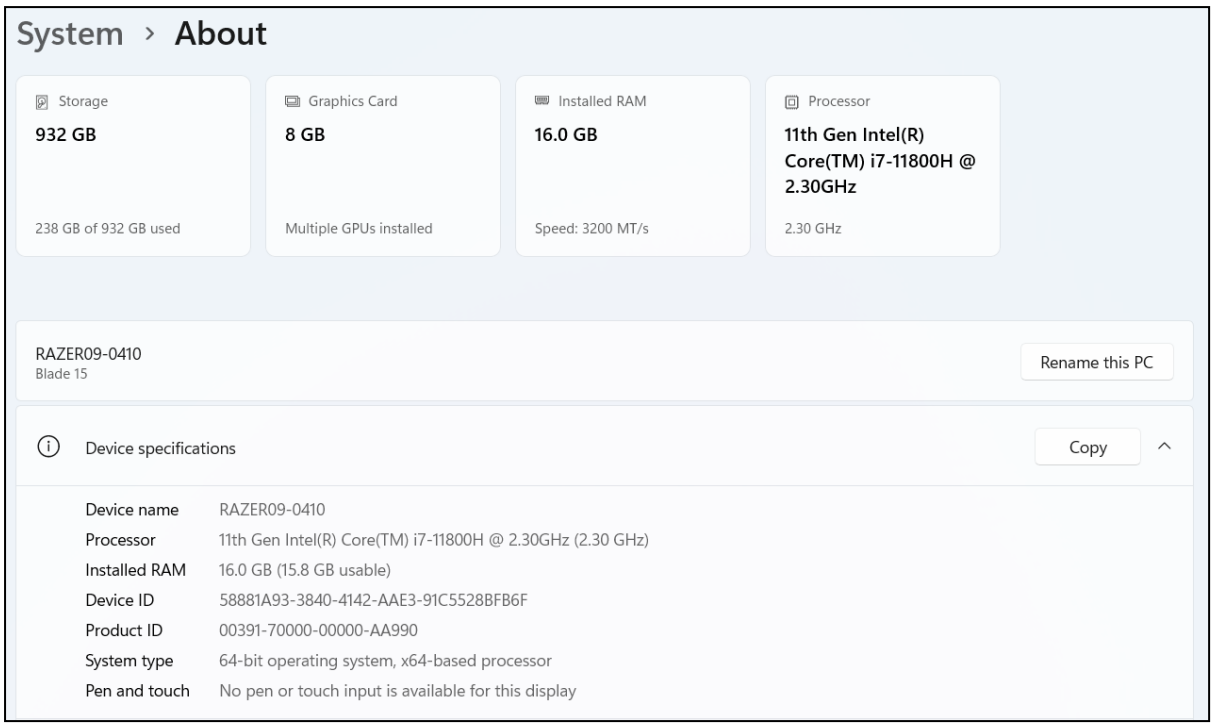


Figure 3.2.2.1: Developer device specifications

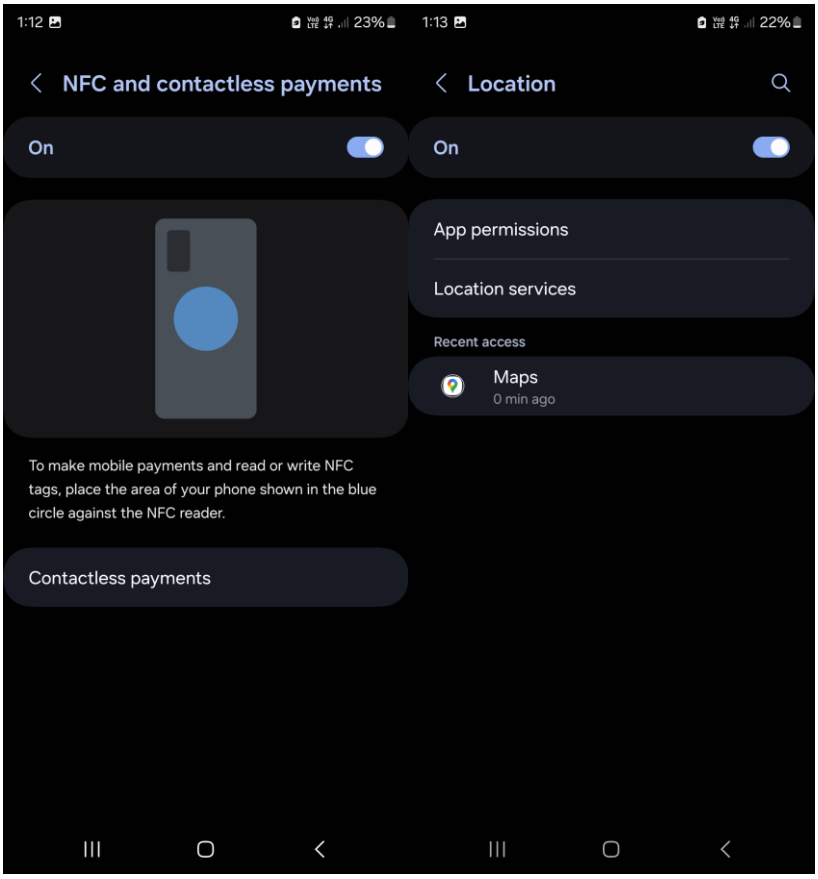


Figure 3.2.2.2: Android device settings showing GPS and NFC enabled

Table 3.2.2 outlines the hardware requirements needed to support the SchoolWay mobile application. The development laptop or PC functions as the primary platform for writing code, performing debugging activities, and executing the Flutter project effectively. The Android test device is required to evaluate the application under real operating conditions, particularly for the parent- and driver-related functions. In addition, stable internet connectivity is essential because the system depends on Firebase services for authentication, cloud data storage, real-time synchronisation, and notification delivery. A GPS-enabled device is also necessary to facilitate location-based tracking, while an NFC-capable phone is required to support driver attendance and check-in operations. Collectively, these hardware components ensure that the application can be developed, tested, and operated in a dependable and efficient manner.

Figure 3.2.2.1 presents the specifications of the developer's device, including the processor, memory, storage capacity, and system type. These specifications indicate that the laptop possesses adequate computational resources to support Flutter development, emulator execution, and debugging processes.

Figure 3.2.2.2 shows that the Android device has both GPS and NFC functions enabled. This configuration is significant because GPS is required for live bus tracking, whereas NFC is used in the driver module for tap-based check-in and attendance recording.

Overall, the table and figures confirm that the project is supported by suitable hardware for both development and operational testing.

3.2.3 Software Components

The software requirements define the applications, platforms, and tools needed for SchoolWay, including Flutter, Dart, Firebase, Android Studio, and related packages. These components support app development, testing, authentication, database management, real-time updates, and notification services, ensuring the system runs efficiently and reliably.

Table 3.2.3: Software Requirements for the SchoolWay Mobile Application

Software	Version / Platform	Purpose
Flutter SDK	Stable channel (3.x)	Cross-platform mobile UI development framework.
Dart SDK	Matching Flutter SDK	Main programming language for business logic and UI.
Android Studio / VS Code	Current stable	IDE and emulator/device debugging support.
Firebase Core Services	Firebase project setup	Backend integration entry point for authentication and data services.
Firebase Authentication	Cloud service	Secure login and account management (email/password, role-gated flows)
Cloud Firestore	Cloud NoSQL database	Stores users, routes, notifications, and operational records
Firebase Realtime Database	Cloud database	Supports low-latency updates for live location/state synchronization.
Firebase Cloud Messaging	Cloud push service	Sends app notifications and alerts.
flutter_map + map tiles	Package-based map stack	Displays route and live bus movement in map interface.
Geolocator	Flutter package	Device location collection and permission handling.
Hive	Local storage package	Local/offline data persistence and queue buffering.

Minimum installation requirements for Android emulator:

1) Android version: 8.0 (Oreo) or later
2) Free storage: at least 200 MB
3) RAM: at least 3 GB
4) Active internet connection
5) Location permission enabled
6) Notification permission enabled
7) NFC enabled (driver use case only)

Table 3.2.3 presents the software requirements and development tools used in the SchoolWay mobile application. It defines the core development stack, including Flutter SDK, Dart SDK, and Android Studio or Visual Studio Code, which together provide the environment for interface development, coding, debugging, and testing. The table also identifies backend and supporting packages required for authentication, database operations, real-time updates, map rendering, location access, and offline storage, thereby establishing the technical foundation needed for reliable system implementation.

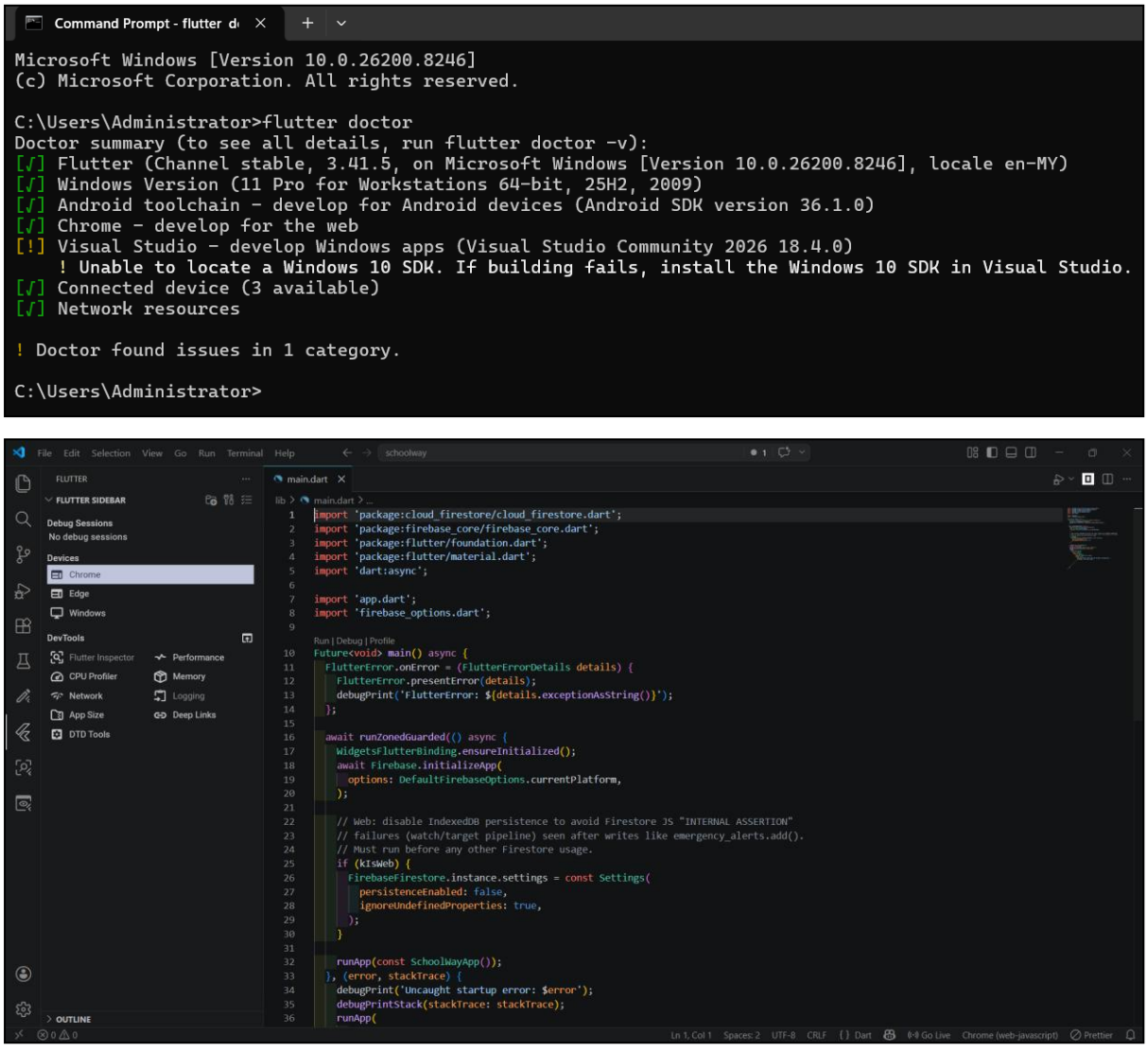


Figure 3.2.3.1: Flutter SDK and IDE setup

Figure 3.2.3.1 shows the local development environment used for implementation. The Flutter diagnostic output indicates that the essential toolchain is installed and functional, while the IDE interface confirms that project source code is being developed and managed in a proper

coding workspace. This figure supports the practicality of the software setup listed in Table 3.2.3 and demonstrates that the project can be built, executed, and debugged in a controlled development environment.

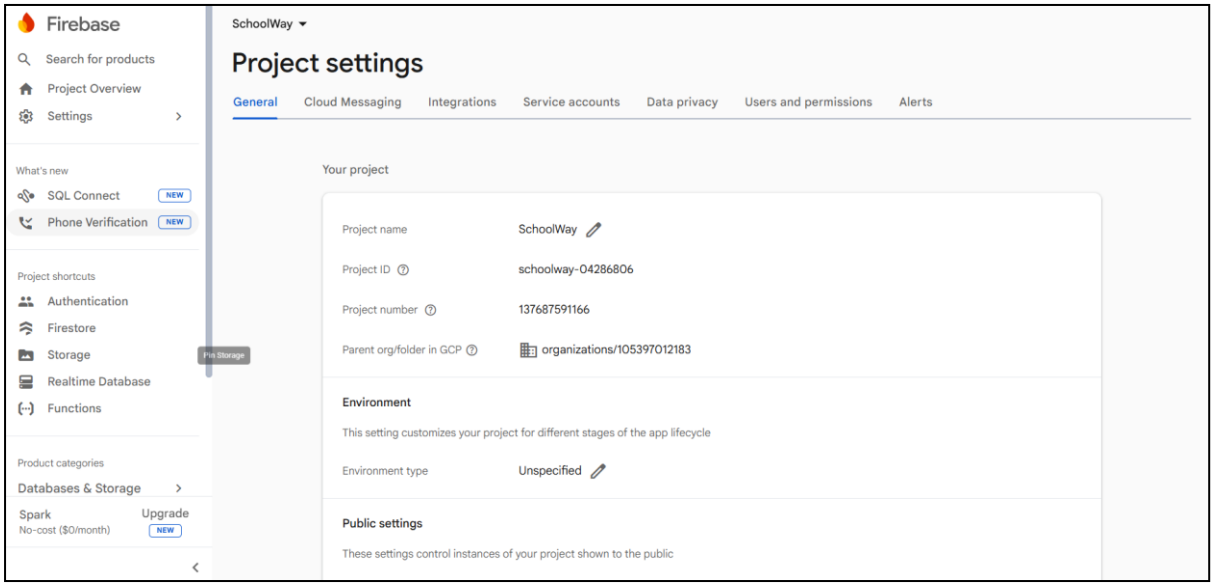


Figure 3.2.3.2: Firebase project console

Figure 3.2.3.2 shows the Firebase project console with configured services. This figure provides evidence that the cloud backend environment is established and ready to support system operations. It verifies that the project is connected to a valid Firebase workspace and that the required cloud modules can be integrated for authentication, data storage, synchronization, and notification delivery.

Firebase services form the backend core of SchoolWay. Firebase Authentication is used for secure user login and account management, Cloud Firestore stores structured operational data such as users, buses, routes, and notifications, Firebase Realtime Database supports low-latency synchronization for live tracking updates, and Firebase Cloud Messaging enables real-time alert and notification delivery. These services are essential because the system relies on continuous cloud communication and role-based data access across parent, driver, and admin modules [42].

Other services and packages support application-specific features. The flutter_map package is used for route visualization and live map display [43]. Geolocator provides device positioning and permission handling for tracking features, and Hive supports local persistence and queue buffering when connectivity is unstable [44]. Together with the core development

tools, these services improve performance, usability, and resilience of the application during both development and deployment.

In summary, Table 3.2.3 and Figure 3.2.3.1 to Figure 3.2.3.2 collectively confirm that SchoolWay is supported by a complete and operational software ecosystem. The integration of development tools, Firebase backend services, and supporting packages provides a stable platform for secure authentication, real-time monitoring, data management, and dependable mobile application performance.

3.3 Functional and Non-Functional Requirements

3.3.1 Function Requirement

Functional requirements describe the behaviours that SchoolWay must provide to satisfy the project objectives. The requirements focus on the core workflows of the parent, driver, and administrator modules, including authenticated access, operational data management, NFC attendance recording, GPS-based bus monitoring, notifications, and historical reporting. Each requirement is linked to an implemented user or system action so that it can later be evaluated through a corresponding test case.

ID	Functional requirement	Requirement description
FR-01	Authentication and account provisioning	The system shall allow provisioned users to authenticate using email and password or Google sign-in. An authenticated account without an assigned role shall remain on the pending-assignment screen until an administrator completes provisioning.
FR-02	Role-based access control	The system shall route authenticated users to the Parent, Driver, or Admin module according to the authoritative role stored for their Firebase Authentication UID.
FR-03	Profile and saved-account management	An authenticated user shall be able to view permitted profile information, update permitted personal fields, sign out, and manage locally

ID	Functional requirement	Requirement description
		saved accounts without changing protected role or account-status fields.
FR-04	Student and parent management	An administrator shall be able to create, edit, import, assign, activate, or deactivate student and parent records, including parent-child links and student NFC identifiers.
FR-05	Bus and driver management	An administrator shall be able to create and update bus records, assign an authorised driver, record capacity and operational status, and associate a bus with a planned route.
FR-06	Route planning and assignment	An administrator shall be able to define ordered service stops, obtain saved road geometry through the backend Google Routes integration, and assign buses and students to a planned route.
FR-07	Driver trip operation	An assigned driver shall be able to start and complete a route session, select the trip direction, view service stops, and perform trip-related actions only for the assigned bus and session.
FR-08	Real-time bus tracking	During an active route session, the system shall capture the driver's GPS position and display authorised bus-location updates on Google Maps for the driver, linked parents, and administrators.
FR-09	Boarding and alighting recording	The driver module shall read a registered NFC identifier, match it to an assigned student, determine a boarding or alighting event, and store the event with the student, driver, bus, session, time, and available location data.
FR-10	Notification delivery	The system shall register a permitted device for Firebase Cloud Messaging and provide persistent in-app notifications for relevant transport, attendance, and emergency events.

ID	Functional requirement	Requirement description
FR-11	Emergency and incident management	An assigned driver shall be able to create an emergency alert or incident for the active bus, while an administrator shall be able to monitor and resolve authorised records.
FR-12	Reports and history	Authorised users shall be able to review relevant route history, boarding events, notifications, attendance summaries, bus activity, driver activity, and emergency history.
FR-13	CSV-based administration	An administrator shall be able to download supported templates, validate CSV records, preview row-level errors, and import valid users, buses, routes, and student-related data in a controlled order.

The functional requirements collectively support the three project objectives. FR-01 to FR-07 establish the integrated role-based mobile platform and its administrative and trip workflows. FR-09 addresses automated student identification and attendance event logging through NFC. FR-08 provides the GPS-based tracking workflow required for live bus visibility, while FR-10 to FR-12 strengthen communication, monitoring, and accountability. FR-13 reduces repetitive administrative data entry and supports efficient prototype setup.

3.3.2 Non-Functional Requirements

Non-functional requirements define the measurable quality attributes expected from SchoolWay. They describe how effectively, securely, reliably, and consistently the implemented functions should operate rather than introducing additional business functions. The requirements below are stated as evaluation targets; a target should only be reported as achieved when Chapter 6 contains corresponding evidence.

ID	Quality attribute	Measurable requirement
NFR-01	Performance	Under a stable validated Wi-Fi or mobile-data connection, primary user actions such as sign-in,

ID	Quality attribute	Measurable requirement
		role routing, dashboard retrieval, and normal navigation should complete within 3 seconds in at least 95% of ten repeated trials. A new GPS record should appear on an authorised monitoring screen within 5 seconds in at least 95% of measured updates.
NFR-02	Security	All protected data access shall require Firebase Authentication. Authorization shall use the Firebase Authentication UID, role, bus assignment, or parent-child relationship. Users shall not be able to change their own role, UID, active status, or another user's protected profile, and privileged notification or account-provisioning operations shall remain server controlled.
NFR-03	Reliability	A transient connectivity failure shall not crash the application. Eligible unsynchronised attendance actions shall be placed in local persistent storage and retried, while unrecoverable failures shall show meaningful recovery information.
NFR-04	Usability and accessibility	Core interfaces shall remain readable and operable at phone, tablet, and desktop widths, in light and dark themes, with meaningful semantics, visible interaction states, and supported reflow at 200% text scaling.
NFR-05	Compatibility	The Android application shall support API level 24 or later. GPS functions shall operate on a location-capable device, and NFC functions shall operate on an Android device with enabled NFC hardware. Unsupported or denied capabilities shall be handled without an application crash.

ID	Quality attribute	Measurable requirement
NFR-06	Availability and recovery	Cloud-dependent screens shall present a recoverable loading, stale-data, offline, or error state when Firebase, maps, or network services are temporarily unavailable. Local preferences and eligible queued actions shall remain available across an application restart.
NFR-07	Scalability	The data design shall use scoped queries, composite indexes, server-side privileged operations, and bounded caches. Before production use, concurrent GPS writes, authorised listeners, and notification fan-out shall be load-tested at the intended fleet size without exceeding the response targets in NFR-01.

3.4 Project Milestone

3.4.1 Project I Timeline

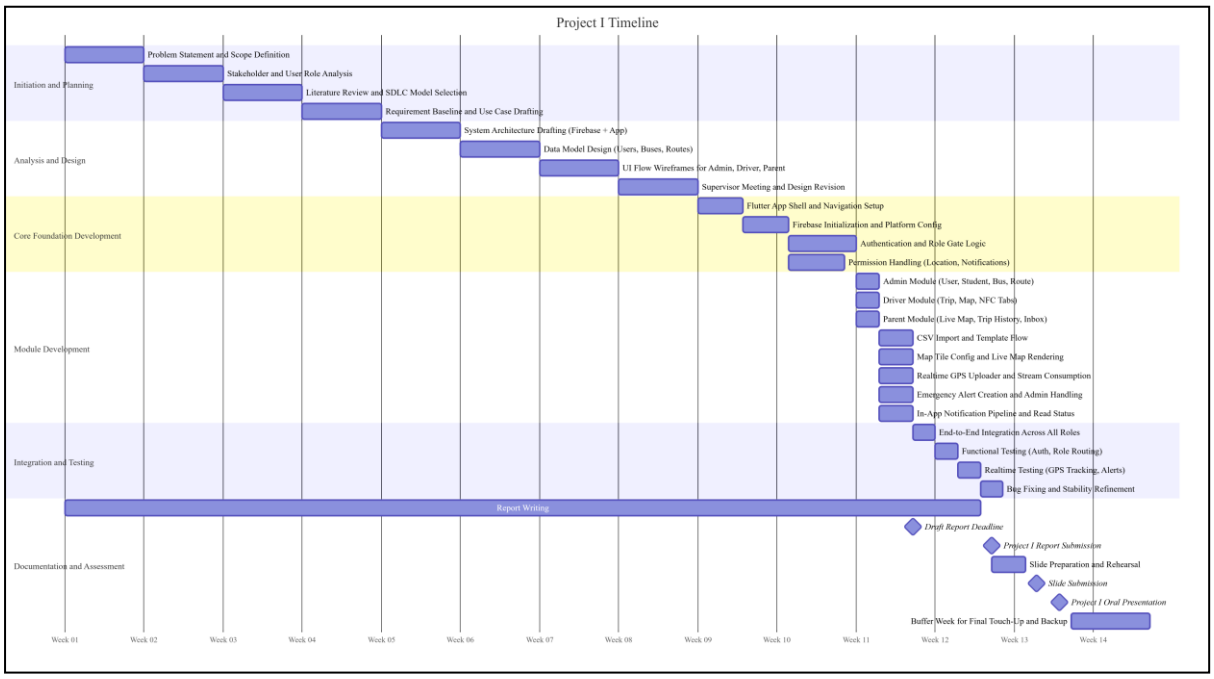


Figure 3.4.1: Project I Timeline in Gantt Chart

3.4.2 Project II Timeline

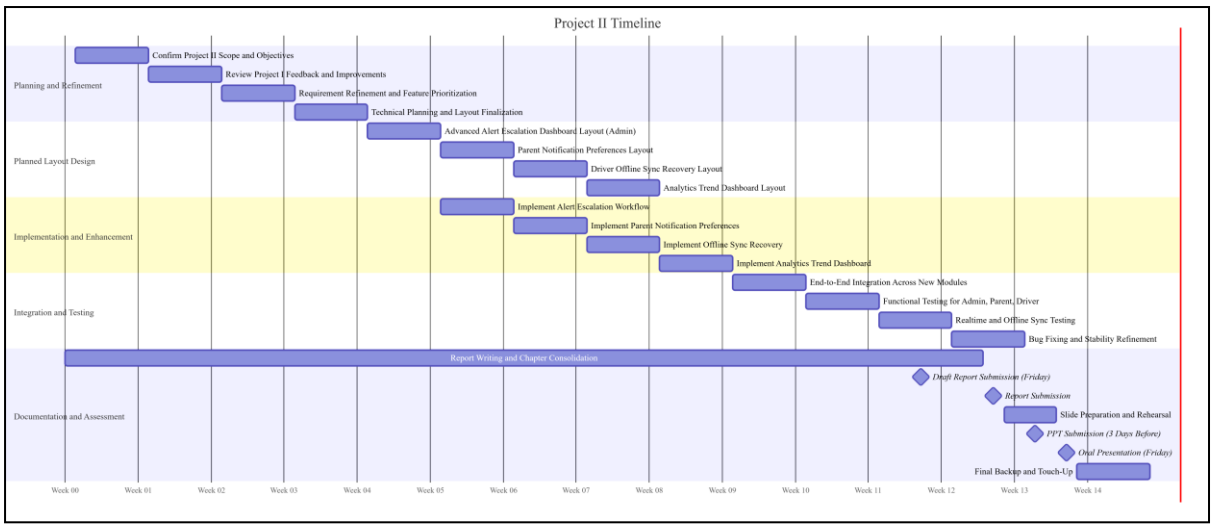


Figure 3.4.2: Project II Timeline in Gantt Chart

3.5 Estimated Cost

The prototype was developed without a separate software-purchase cost because Flutter, Dart, Visual Studio Code, Android development tools, Firebase command-line tools, and the libraries used by the project are available for download without an upfront licence fee. Development and testing used an existing Windows computer, Android phone, Internet connection, and NFC capability; therefore, no new hardware was purchased specifically for the prototype. Nevertheless, the operational system should not be described as permanently cost-free. Firebase, Google Maps Platform, Google Routes API, cloud storage, network usage, and application distribution may incur charges when free quotas are exceeded or the system is deployed at a larger scale. Consequently, the estimated direct development cost for this academic prototype was RM0, excluding existing hardware, electricity, Internet service, NFC cards, and any future usage-based cloud charges.

3.6 Concluding Remark

This chapter established the development approach, technical environment, functional scope, quality expectations, milestone plan, and cost basis of SchoolWay. The requirements translate the project objectives into specific behaviours for authenticated role-based access, NFC attendance recording, GPS bus tracking, notification delivery, and operational administration. The non-functional requirements provide measurable targets for performance, security, reliability, usability, compatibility, availability, and scalability. Together, these requirements provide the traceability baseline for the system design in Chapter 4, implementation in Chapter 5, and evaluation in Chapter 6.

CHAPTER 4

System Design

This chapter presents the design of SchoolWay before discussing its detailed implementation. It describes the high-level architecture, actor interactions, major activity flows, and database structure that support the parent, driver, and administrator modules. The design is aligned with the functional and non-functional requirements in Chapter 3 so that each major component and workflow has a clear purpose within the overall system.

4.1 System Architecture

4.1.1 Introduction

The system architecture of SchoolWay is designed to provide a scalable, secure, and responsive platform for school transportation monitoring and coordination. The architecture organizes front-end mobile modules, cloud backend services, and communication channels into a modular structure so each subsystem can be developed and maintained independently while still working cohesively. The overall architecture is illustrated in Figure 3.3.1.

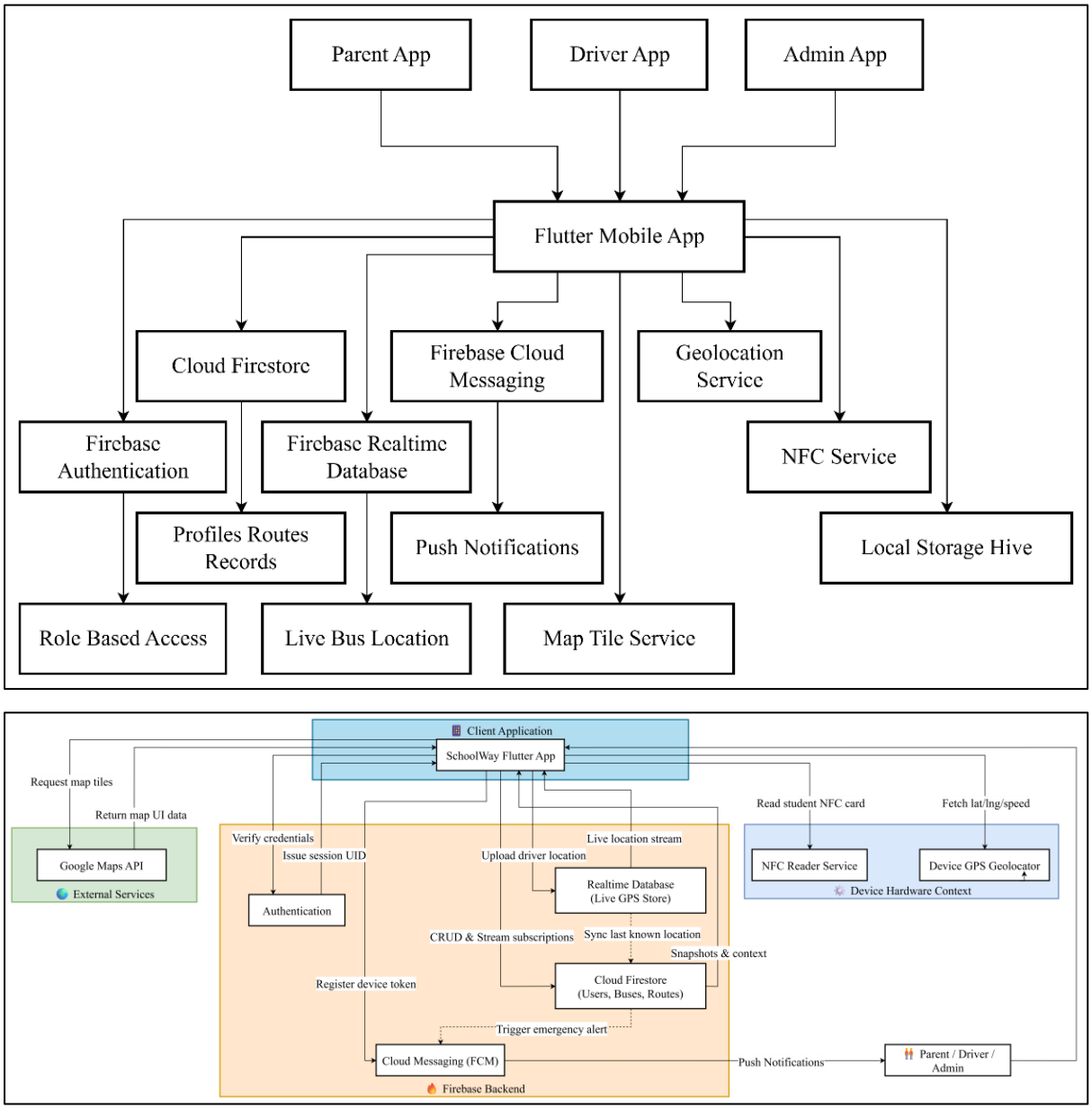


Figure 4.1.1: System Architecture Diagram

4.1.2 High-Level Architecture Components

1. **Presentation Layer (Mobile Application)**

   The mobile frontend is built using Flutter, enabling a unified codebase with role-based interfaces for Admin, Driver, and Parent users. Each role has dedicated screens and permissions, improving usability and access control.

2. **Authentication and Authorization Layer**

   Firebase Authentication manages user sign-up, sign-in, and session handling. Role-based access is enforced by storing role metadata and routing users to appropriate modules after authentication.

3. **Data Management Layer**

   Cloud Firestore stores structured entities such as user profiles, student records, route planning data, notifications, and attendance-related events. Firebase Realtime Database is used for low-latency, frequently changing data such as live bus location and movement updates.

4. **Location and Tracking Layer**

   GPS data from driver devices is periodically captured and synchronized to backend services. Parent and admin views retrieve this data to visualize live fleet movement and trip progress.

5. **Notification and Alert Layer**

   Firebase Cloud Messaging delivers push notifications for important operational events, such as emergencies, trip updates, and transport alerts.

6. **Integration Layer**

   External package integrations include map rendering, geolocation, NFC reading (driver workflow), and CSV-based import/export for administrative operations.

7. **Security and Reliability Layer**

   Security rules in Firebase services, role checks, and controlled permissions protect data access. Error handling and local persistence mechanisms improve resilience during unstable connectivity.

4.1.3 Data Flow Summary

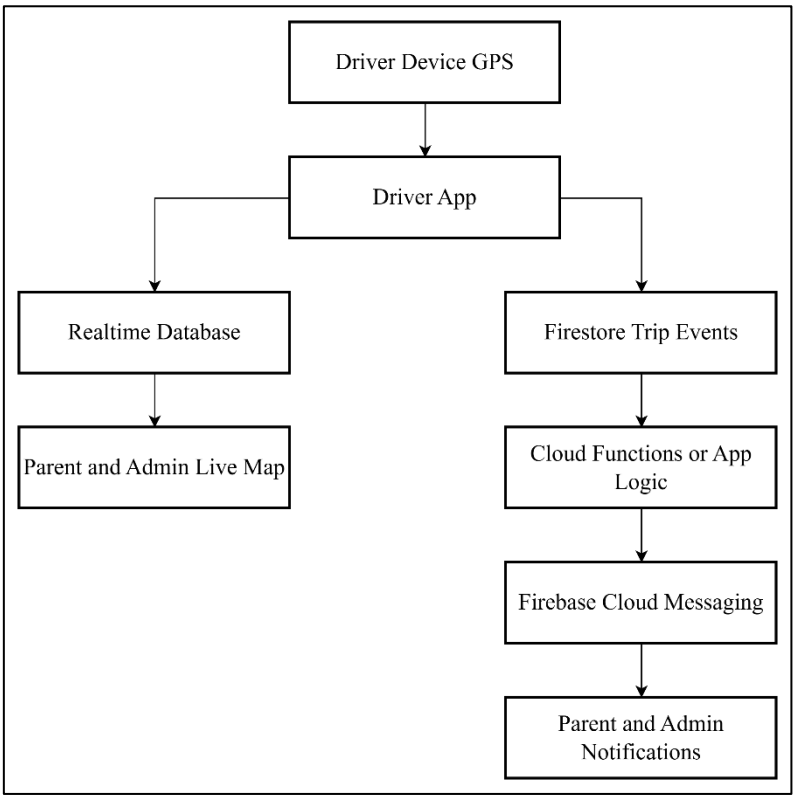


Figure 4.1.3: Data Flow Diagram for location and notifications

As shown in Figure 4.1.3, the SchoolWay data flow illustrates how authentication, real-time tracking, and notification services are integrated to support continuous communication between drivers, parents, and administrators.

1. User authenticates through Firebase Authentication.
2. System identifies user role and loads corresponding interface.
3. Driver location and event data are uploaded to cloud services.
4. Parent/admin modules subscribe to updates and display live status.
5. Notifications are dispatched via cloud messaging when trigger conditions are meet.

4.1.4 Conclusion of Design

In conclusion, the SchoolWay architecture establishes a robust foundation for secure user access, real-time tracking, role-based operations, and scalable cloud-backed data services. Its modular design supports maintainability and future enhancements such as analytics, route optimization, and expanded platform support.

4.2 Use Case Diagram

The use case diagram shows the system users, also known as actors, and their interactions with SchoolWay. In this project, there are three main actors: **Parent, Driver, and Administrator**. Each actor must authenticate before accessing the functions assigned to their role. The role is obtained from the user's authorised account profile; therefore, users cannot select or modify their own role. Parents use the system to monitor their linked children and school buses, drivers manage route sessions and student transportation activities, while administrators maintain the operational data and monitor the overall transportation service. Figure 4.2 presents the use cases available to each actor.

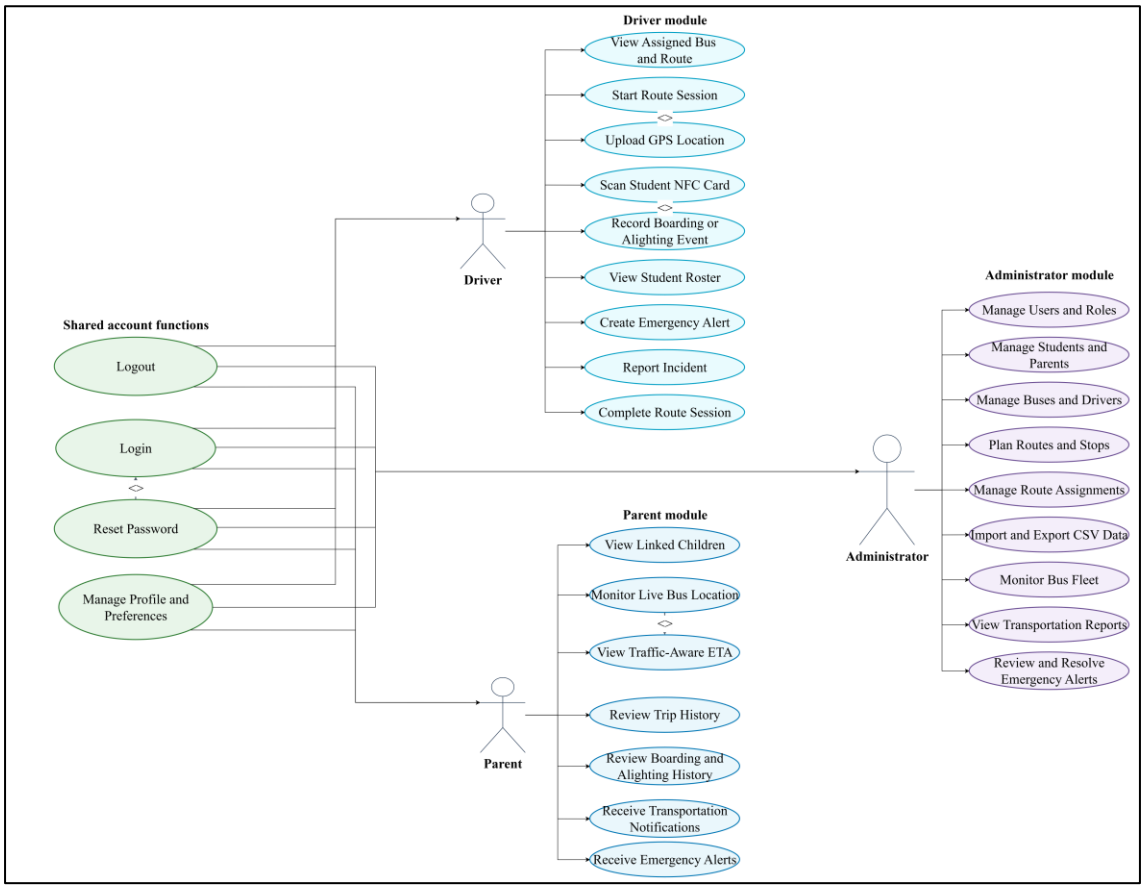


Figure 4.2: Use case diagram for the Parent, Driver, and Administrator modules of SchoolWay.

Table 4.1: Description of SchoolWay use cases

Actor	Use case	Description
Parent, Driver, and Administrator	Login	Allows a registered user to authenticate using a SchoolWay account. After successful authentication, the system retrieves the user profile and directs the user to the appropriate module based on the assigned role.
	Reset Password	Allows a user who has forgotten the account password to request a password-reset email. This use case extends the Login use case when the user cannot complete the normal authentication process.
	Manage Profile and Preferences	Allows an authenticated user to view or update permitted profile information and application preferences, including theme, language, motion, and notification settings. Protected attributes such as the authentication identity and role cannot be modified by the user.
	Logout	Allows an authenticated user to securely end the current session and return to the authentication screen.
Parent	View Linked Children	Allows a parent to view children linked to the parent's authenticated account. The parent can only access information belonging to authorised linked children.

Actor	Use case	Description
	Monitor Live Bus Location	Allows a parent to view the latest authorised location of the child's assigned school bus on Google Maps. Access is determined by the selected child's bus and route assignment.
	View Estimated Time of Arrival	Allows a parent to view the estimated arrival time of the assigned school bus. SchoolWay uses road-based route information and current traffic conditions to provide a more realistic ETA.
	Review Trip History	Allows a parent to review previous route sessions and transportation information associated with a linked child.
	Review Boarding and Alighting History	Allows a parent to review the child's recorded boarding and alighting events. These records are normally created when the driver scans the student's NFC card.
	Receive Transportation Notifications	Allows a parent to receive notifications about transportation events, including student boarding, student alighting, and relevant route updates.
	Receive Emergency Alerts	Allows a parent to receive emergency notifications when an emergency involving the child's assigned bus is reported. Emergency messages remain available in the in-application notification inbox.
Driver	View Assigned Bus and Route	Allows a driver to review the assigned bus, planned route, trip direction, route stops, and student roster.

Actor	Use case	Description
	Start Route Session	Allows an authorised driver to begin a transportation session using the assigned bus and route. The system verifies the driver-bus assignment before activating the route session.
	Upload GPS Location	Allows the application to upload the assigned bus location while a route session is active. The location information supports live bus monitoring by authorised parents and administrators.
	Scan Student NFC Card	Allows a driver to scan a student's registered NFC card during an active route session. The system identifies the assigned student and determines whether the scan represents a boarding or alighting event.
	Record Boarding or Alighting Event	Allows SchoolWay to record the student, bus, driver, route session, event type, time, and location after a valid NFC scan. If the card is not registered, the system displays a warning and does not create an invalid transportation record.
	View Student Roster	Allows a driver to view students assigned to the current bus and review their latest boarding or alighting status.
	Create Emergency Alert	Allows an authorised driver to report an emergency involving the assigned bus. The system records the driver, bus, route session, time, and available location before notifying authorised parents and administrators.
	Report Incident	Allows a driver to record a non-emergency transportation incident, including its category, severity, description, bus, and occurrence time.
	Complete Route Session	Allows a driver to end an active route session after completing the journey. Completing the session stops session-bound GPS tracking and records the session's completion status.
Administrator	Manage Users and Roles	Allows an administrator to create and maintain user accounts and assign the appropriate Parent, Driver, or Administrator role. Ordinary users cannot assign or modify their own roles.

Actor	Use case	Description
	Manage Students and Parents	Allows an administrator to create and update student information, link students to authorised parents, register NFC card identifiers, and maintain transportation assignments.
	Manage Buses and Drivers	Allows an administrator to maintain bus information, assign drivers, configure operational statuses, and manage the relationships between buses, drivers, and planned routes.
	Plan Routes and Stops	Allows an administrator to create planned routes and arrange their stops according to the required service order.
	Calculate Route Geometry	Allows the system to calculate road-based route geometry through the Google Routes API. The resulting geometry is stored with the planned route and used for route visualisation and ETA calculation.
	Manage Route Assignments	Allows an administrator to assign drivers, buses, students, and planned routes according to the required transportation arrangement.
	Import and Export CSV Data	Allows an administrator to import supported operational data from CSV files and export selected system data. The system validates the file structure and field values before committing valid records.
	Monitor Bus Fleet	Allows an administrator to monitor active buses, route sessions, current locations, and operational statuses
	View Transportation Reports	Allows an administrator to review NFC activity, boarding and alighting records, route-session history, emergency history, incidents, and attendance corrections.
	Review and Resolve Emergency Alerts	Allows an administrator to view active emergency alerts, inspect the available bus and location information, and record a resolution after the emergency has been handled.

4.3 Activity Diagram

4.3.1 Authentication and Role Routing Activity

This activity diagram outlines the process by which provisioned users authenticate into the SchoolWay mobile application and are routed to their designated role modules. In compliance with **FR-01** (Authentication and Account Provisioning), **FR-02** (Role-Based Access Control), **FR-03** (Profile Management), and **NFR-02** (Security & Least-Privilege Access), user authorization is strictly tied to the authoritative authUid and role metadata in Cloud Firestore rather than client-selected identities. This guarantees that unprovisioned accounts are held at a secure pending screen and cannot escalate privileges.

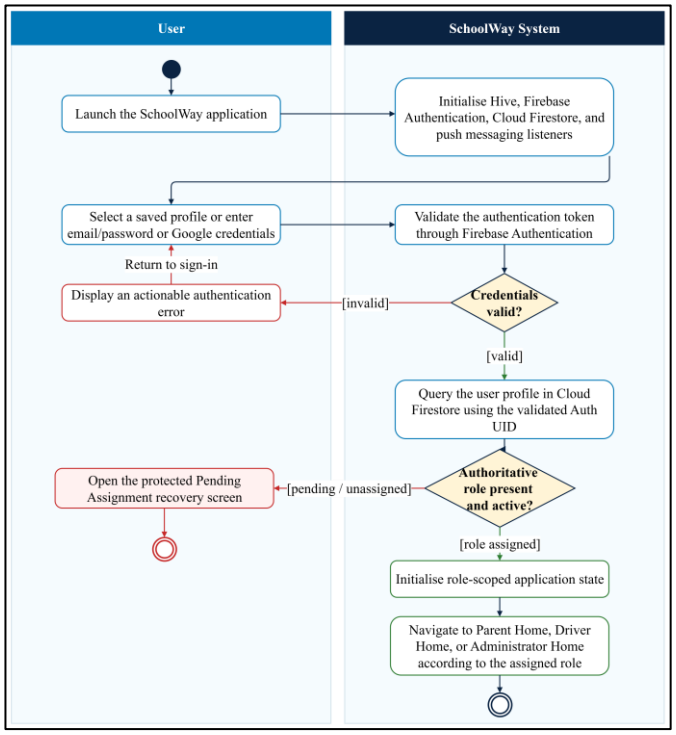


Figure 4.3.1: Authentication and role-routing activity for provisioned SchoolWay users.

The process begins at the "Start" node in the User swimlane when the user launches the SchoolWay application. The System executes its initialization gate, preparing local storage (Hive), Firebase Authentication, Firestore, and push messaging listeners. The user selects an existing saved profile or inputs credentials using email/password or Google OAuth. The system transmits the authentication token to Firebase Authentication for validation.

If the credentials are invalid, the system displays an actionable authentication error on the interface and returns the user to the sign-in prompt. If the credentials are valid, the system queries Cloud Firestore using the validated authUid to retrieve the user's profile document.

A decision point evaluates whether an authoritative role (Parent, Driver, or Admin) is present and active:

- If a valid role is assigned ([role assigned]), the system initializes role-scoped state and navigates the user to their respective dashboard (Parent Home, Driver Home, or Admin Home).
- If no role is assigned or the profile is unprovisioned ([pending / unassigned]), the system redirects the user to the Pending Assignment recovery screen, preventing access to protected operational features until an administrator provisions the account. The flow terminates at the "End" node.

4.3.2 Administrator Data and Route Preparation Activity

This activity diagram illustrates the workflow through which an administrator prepares the transportation data, including student profiles, parent-child links, vehicle fleet data, driver assignments, and planned route geometry. This activity directly satisfies **FR-04** (Student & Parent Management), **FR-05** (Bus & Driver Management), **FR-06** (Route Planning & Assignment), and **FR-13** (CSV-based Administration), while adhering to **NFR-02** (Authorization) and **NFR-03** (Data Validation & Integrity).

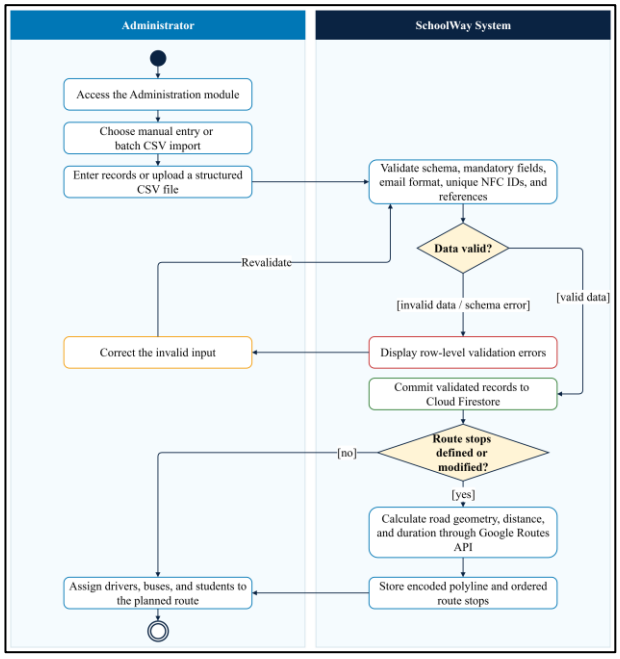


Figure 4.3.2: Administrator data validation, route preparation, and transportation assignment activity.

The process begins at the "Start" node in the Administrator swimlane when the administrator accesses the Administration module. The administrator chooses between manual record entry or batch CSV importation for users, students, buses, or routes. The administrator inputs the dataset or uploads the structured CSV file.

The SchoolWay system performs strict schema and relational validation with checking mandatory fields, email formatting, unique NFC card UIDs, and foreign key references.

If validation fails [invalid data / schema error], the system displays row-level error summaries, allowing the administrator to correct the inputs.

Once validated [valid data], the system commits the records to Cloud Firestore. When defining or modifying route stops, the system invokes the backend Google Routes API integration to calculate actual road-based polyline geometry and realistic distance/duration metrics. The system stores the encoded polyline geometry alongside the ordered stop array. Finally, the administrator assigns drivers, buses, and registered students to the planned route, establishing the relational mapping required for operational trips. The workflow terminates at the "End" node.

4.3.3 Driver Route, GPS, and NFC Activity

This activity diagram models the operational trip workflow executed by an authorized driver, covering trip initiation, background GPS location broadcasting, student NFC attendance verification, and trip conclusion. This process fulfills **FR-07** (Driver Trip Operation), **FR-08** (Real-Time Bus Tracking), and **FR-09** (Boarding & Alighting Recording), in alignment with **NFR-01** (Low-Latency GPS Streaming), **NFR-03** (Offline Queue Reliability), and **NFR-05** (Hardware Compatibility).

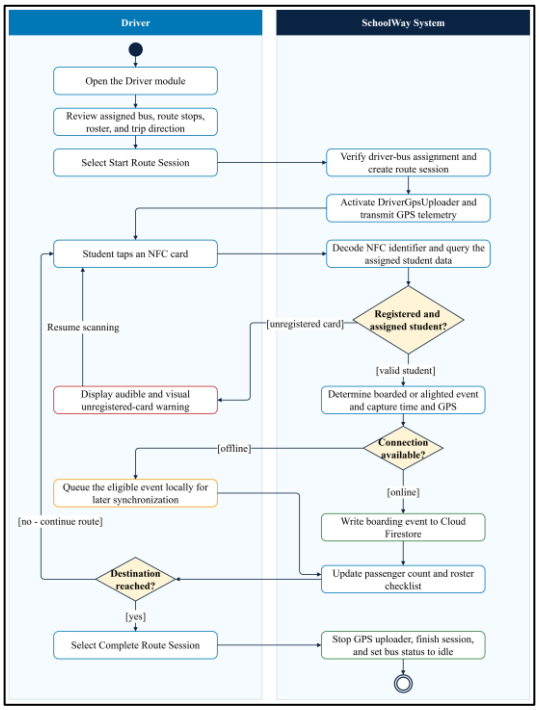


Figure 4.3.3: Driver route-session, GPS tracking, and student NFC boarding and alighting activity.

The process begins at the "Start" node when the driver opens the Driver module. The driver reviews the assigned bus, route stops, student roster, and selects the trip direction (Pickup or Drop-off). The driver clicks "Start Route Session". The system validates the driver-bus assignment, creates a new route_sessions document, and activates the background DriverGpsUploader service. The device periodically samples GPS fixes and writes real-time telemetry (coordinates, speed, heading) to the gps_tracking collection.

As the bus progresses along the route, students tap their physical NFC smart tags on the driver's device at designated stops.

The system's NFC reader decodes the identifier and queries the student database:

- If the tag is not recognized ([unregistered card]), the system issues an immediate audible/visual warning on the driver's screen without writing invalid records, and scanning resumes.
- If recognized ([valid student]), the system determines the context-aware event type (boarded or alighted), captures current timestamp and GPS coordinates, and writes a boarding_events record to Firestore (or queues it locally in the offline queue if connectivity is interrupted).

The driver UI updates the live passenger count and roster checklist in real time. Upon reaching the destination, the driver clicks "Complete Route Session". The system shuts down the GPS uploader, marks the session as finished, and updates the bus status to idle. The process terminates at the "End" node.

4.3.4 Parent Monitoring and Notification Activity

This activity diagram models how authenticated parents monitor their linked children's school bus in real time and receive instant operational notifications. Synchronized with **FR-08** (Real-Time Bus Tracking), **FR-09** (Attendance Monitoring), **FR-10** (Notification Delivery), and **FR-12** (Reports & History), the activity adheres to **NFR-01** (Under 5-second Map & Notification Refresh), **NFR-02** (Strict Parent-Child Access Scoping), and **NFR-06** (Graceful Stale/Offline Recovery).

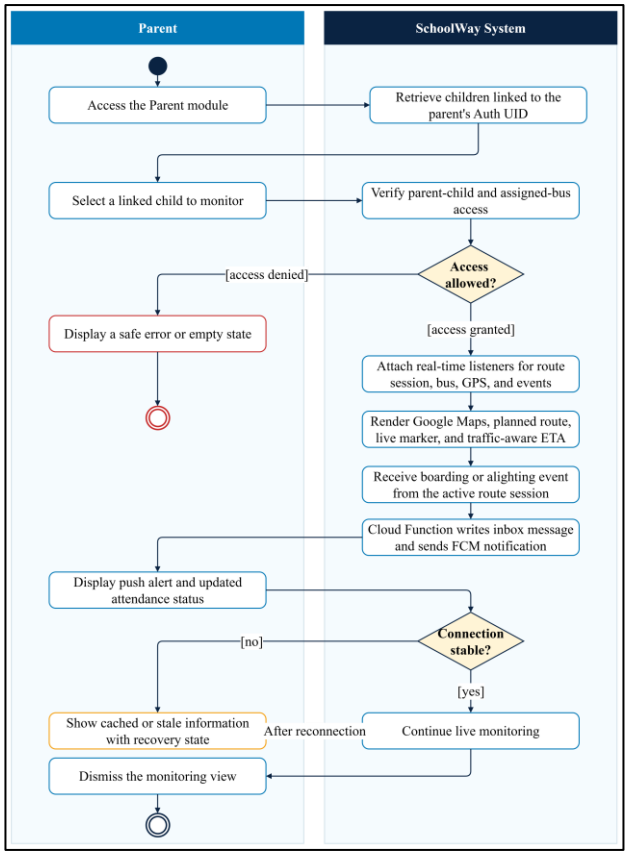


Figure 4.3.4: Parent live bus monitoring, traffic-aware ETA, and transportation notification activity.

The process begins at the "Start" node when the parent accesses the Parent module. The system retrieves the list of children linked specifically to the parent's authUid. The parent selects a child to monitor. The system verifies access permissions via Firestore security rules:

- If access is unauthorized or denied ([access denied]), the system renders a safe error/empty state and stops.
- If authorized ([access granted]), the system attaches real-time Firestore stream listeners to the child's assigned bus, active route session, and gps_tracking updates.

The system renders Google Maps displaying the live vehicle location marker, planned route geometry polyline, and dynamically calculated traffic ETA. In parallel, when the child's NFC card is scanned by the driver, a Cloud Function triggers a high-priority Firebase Cloud Messaging (FCM) push notification and writes an in-app message to the parent's notifications inbox. The parent's device displays the push alert and updates the live attendance status (e.g., "Boarded at 7:15 AM"). If connectivity fluctuates, the UI transitions gracefully to a cached/stale state without crashing. The process concludes at the "End" node when the parent dismisses the view.

4.3.5 Emergency Alert Activity

This activity diagram depicts the critical workflow for reporting, propagating, and resolving in-transit school bus emergencies. Built to satisfy **FR-11** (Emergency and Incident Management) and **FR-10** (Urgent Notification Fan-out), this process operates under **NFR-01** (Rapid Real-Time Alert Delivery) and **NFR-02** (Strict Authorization & Traceable Audit History) to ensure immediate situational awareness and safety response.

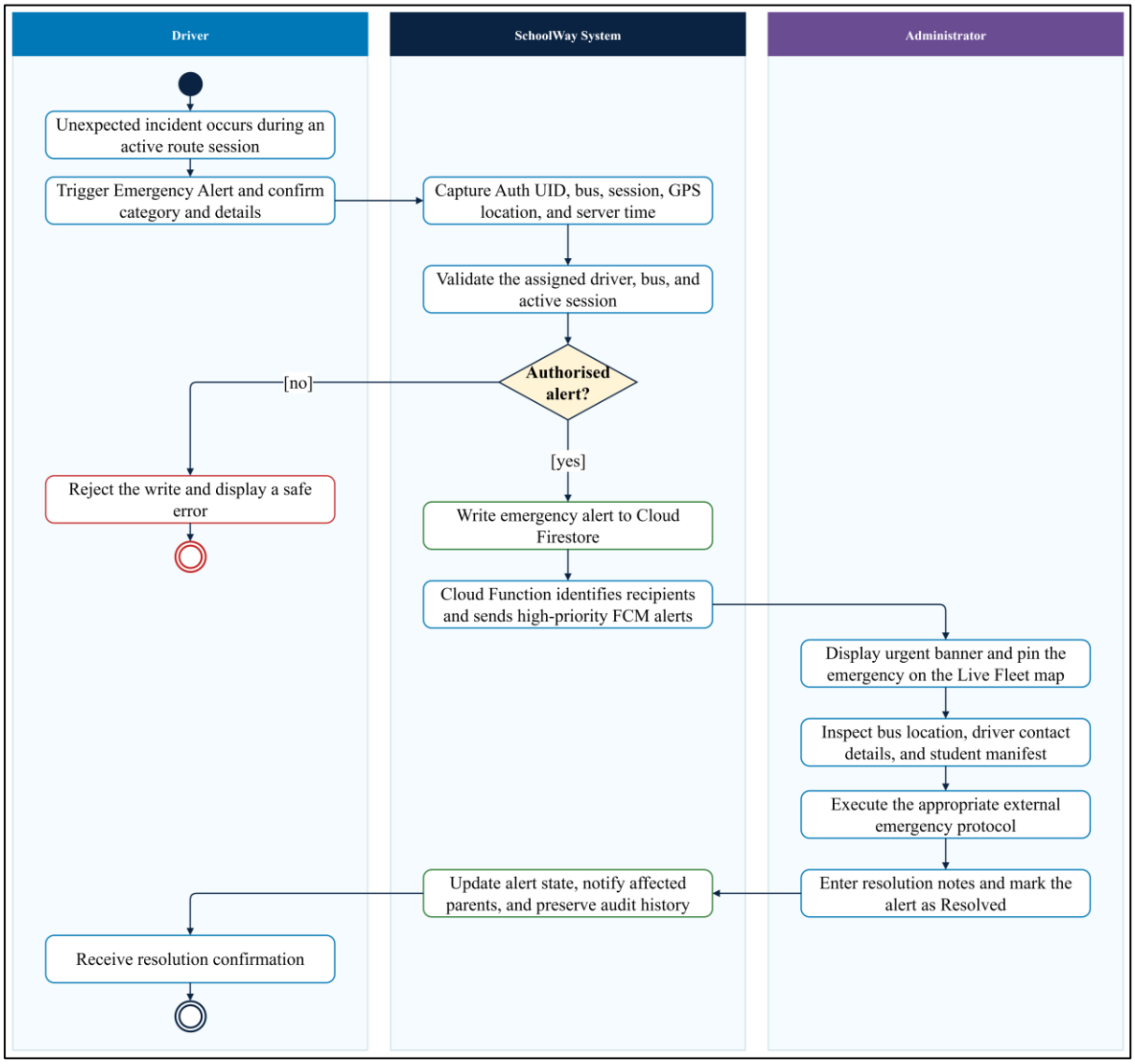


Figure 4.3.5: Emergency-alert reporting, notification, response, and resolution activity.

The process begins at the "Start" node in the Driver swimlane during an active route session when an unexpected incident occurs (e.g., vehicle breakdown, medical emergency, accident). The driver triggers the "Emergency Alert" action and confirms the alert category and details.

The system captures the driver's authUid, assigned busId, active sessionId, high-accuracy GPS coordinates, and server timestamp. The system writes an emergency record to the emergency_alerts collection. A Cloud Function / background trigger validates the incident and immediately broadcasts high-priority push notifications to all administrators and parents whose children are on the affected bus.

The Admin module immediately displays an urgent visual banner and pins the emergency on the Live Fleet map. The administrator inspects the bus location, driver contact details, and student manifest, and executes external emergency protocols. Once the situation is handled, the administrator inputs resolution notes and marks the alert as "Resolved". The system updates the Firestore alert state to resolved, pushes a resolution update to affected parents, and archives the event for auditing. The flow terminates at the "End" node.

4.4 Database Design

SchoolWay uses Cloud Firestore as its primary operational database. Firestore was selected because its document model and real-time streams are suitable for role-based mobile workflows, live transport updates, and event history. Collections are linked through explicit identifiers such as authUid, studentId, busId, routeId, sessionId, driverUid, and recipientUid. The design avoids treating a username as an authorization identity; Firebase Authentication UID is used for ownership and access decisions.

Collection	Important fields	Purpose and relationship
users	authUid, username, firstName, lastName, email, role, isActive	Stores the application profile for an authenticated UID. Role and active status are protected authorization attributes.
students	studentId, firstName, lastName, nfcCardId, parentAuthUids, assignedBusId, assignedRouteId, pickupStopIndex, dropoffStopIndex, status, classId, createdAt	Links a student to one or more parents, a bus, a planned route, service-stop positions, and an NFC identifier.
buses	busNumber, licensePlate, status, currentDriverUid, assignedRouteId, capacity, lastLocation, createdAt	Represents a fleet vehicle and its current driver and route assignments.
planned_routes	name, stops, waypoints, expectedDurationMinutes, googleGeometry, createdAt, updatedAt	Stores ordered service stops and backend-generated Google road geometry used for route display and ETA calculations.
route_sessions	driverUid, busId, routeId, startTime, endTime, status, gpsTrackingEnabled,	Represents one active or completed driver trip and binds trip operations to a specific driver and bus.

Collection	Important fields	Purpose and relationship
	tripDirection, serviceStopRouteIndices	
gps_tracking	latitude, longitude, speed, heading, accuracy, timestamp, sessionId, driverUid, isSimulated	Uses the bus ID as the document ID to expose the latest authorised vehicle position.
boarding_events	studentId, busId, driverUid, sessionId, eventType, location, timestamp, nfcCardId, synced, isManual, manualNote, correction fields	Stores immutable transport attendance evidence, with controlled administrative correction or deletion.
notifications	recipientUid, title, body, type, category, read, createdAt, studentIds, relatedAlertId, isCritical, location	Stores server-created messages for an authorised recipient and permits the recipient to update only the read state.
emergency_alerts	driverUid, busId, sessionId, location, timestamp, status, resolvedBy, resolvedAt, notification summary	Records a driver emergency and the later administrative response.
incidents	driverUid, busId, description, severity, resolved, timestamps	Stores non-emergency operational incidents for driver and administrator review.
attendance_corrections	correctedByUid and correction details	Provides a controlled audit record for manual attendance corrections.
bus_access	busId, member authUid	Materialises which parents may access a particular bus, supporting security-rule evaluation.

The relationships are enforced at both the application and security-rule layers. For example, a driver can create a boarding event only when the driver UID matches the authenticated user, the driver is assigned to the bus, the student belongs to that bus, and the route session matches both the driver and bus. A parent can read a student only when the parent's UID is listed in

parentAuthUids and can access bus data only through the corresponding membership record. Notifications cannot be created directly by a client, and protected routing-cache records are inaccessible to clients.

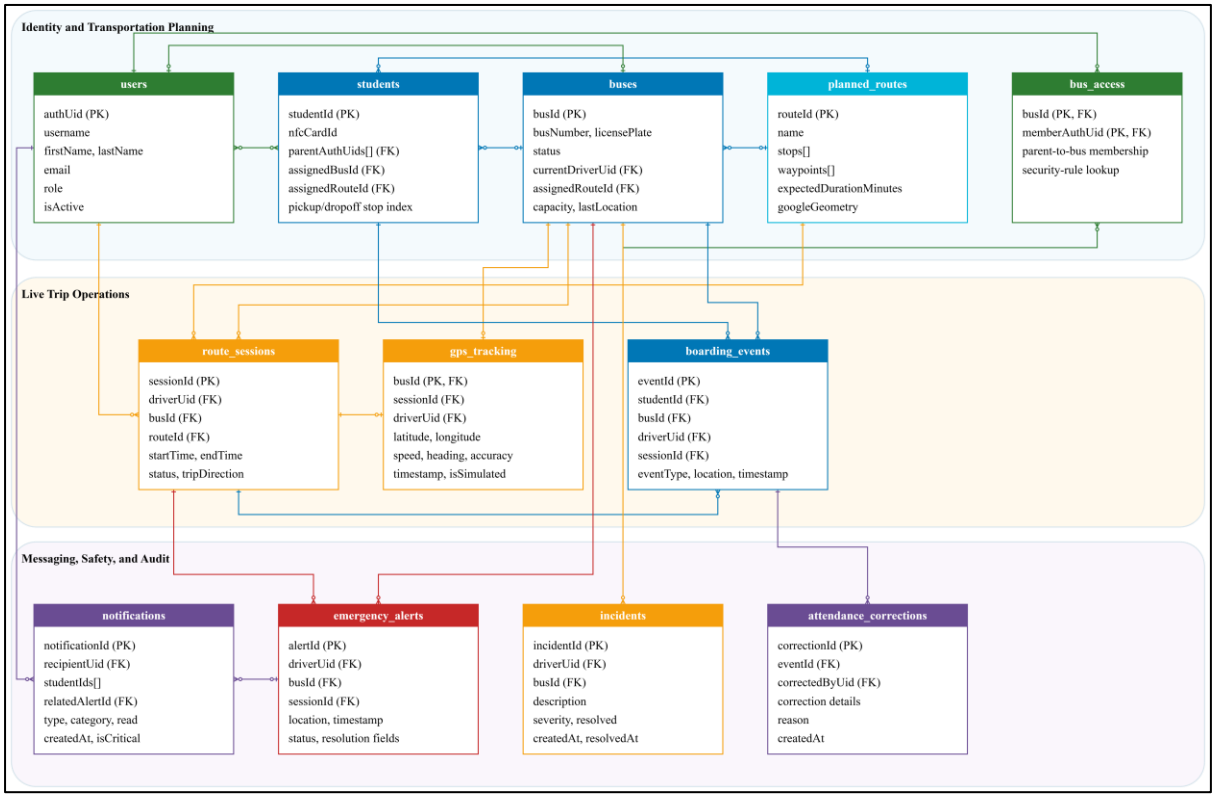


Figure 4.4: Cloud Firestore collections and identifier-based relationships used by SchoolWay.

4.5 Conclusion Remark

This chapter described the design of SchoolWay through its layered architecture, role-based use cases, principal activity flows, and Firestore data model. The design connects device GPS and NFC capabilities with authenticated Flutter modules, local persistence, managed Firebase services, and backend Google mapping operations. Explicit relationships among users, students, buses, routes, sessions, events, and notifications allow the system to support real-time monitoring while maintaining role and ownership boundaries. These designs provide the blueprint for the implementation presented in Chapter 5 and the traceable evaluation presented in Chapter 6.

CHAPTER 5

System Implementation

5.1 Software/Hardware Setup and Configuration

This section explains the software environment setup, third-party integration, cloud and database configuration, deployment preparation, and monitoring setup for the SchoolWay mobile application. The focus is on installation and configuration procedures.

5.1.1 Environment Preparation

The SchoolWay development environment was prepared on Windows using Flutter, Dart, Visual Studio Code, Android SDK tools, and Firebase tooling. The software was downloaded from official sources to ensure version reliability and compatibility.

5.1.1.1 Flutter and Dart SDK Setup

Flutter SDK was downloaded from the official Flutter website: https://docs.flutter.dev/get-started/install/windows.

Installation was performed using the default package, then the Flutter bin directory was added to the system Path environment variable. No custom installation parameters were required.

Verification was performed using:

- flutter --version
- dart --version
- flutter doctor

```
C:\Users\Administrator>flutter --version
Flutter 3.41.5 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 2c9eb20739 (5 weeks ago) • 2026-03-17 16:14:01 -0700
Engine • hash c1db59d880ca73dd86cec08a6663f287522d9f39 (revision 052f31d115)
(1 months ago) • 2026-03-17 20:29:11.000Z
Tools • Dart 3.11.3 • DevTools 2.54.2

C:\Users\Administrator>dart --version
Dart SDK version: 3.11.3 (stable) (Tue Mar 17 01:06:16 2026 -0700) on "windo
ws_x64"

C:\Users\Administrator>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[√] Flutter (Channel stable, 3.41.5, on Microsoft Windows [Version
    10.0.26200.8246], locale en-MY)
[√] Windows Version (11 Pro for Workstations 64-bit, 25H2, 2009)
[√] Android toolchain - develop for Android devices (Android SDK version
    36.1.0)
[√] Chrome - develop for the web
[!] Visual Studio - develop Windows apps (Visual Studio Community 2026
    18.4.0)
    ! Unable to locate a Windows 10 SDK. If building fails, install the
      Windows 10 SDK in Visual Studio.
[√] Connected device (3 available)
[√] Network resources

! Doctor found issues in 1 category.
```

Figure 5.1.1.1: Futter and Dart SDK Setup Verification

5.1.1.2 Visual Studio Code setup

Visual Studio Code was downloaded from the official website: https://code.visualstudio.com/. Installation used default settings. After installation, the Flutter and Dart extensions were installed from the VS Code Extensions Marketplace.

Verification was performed by checking:

- code –version
- Flutter and Dart extensions shown as Installed in VS Code

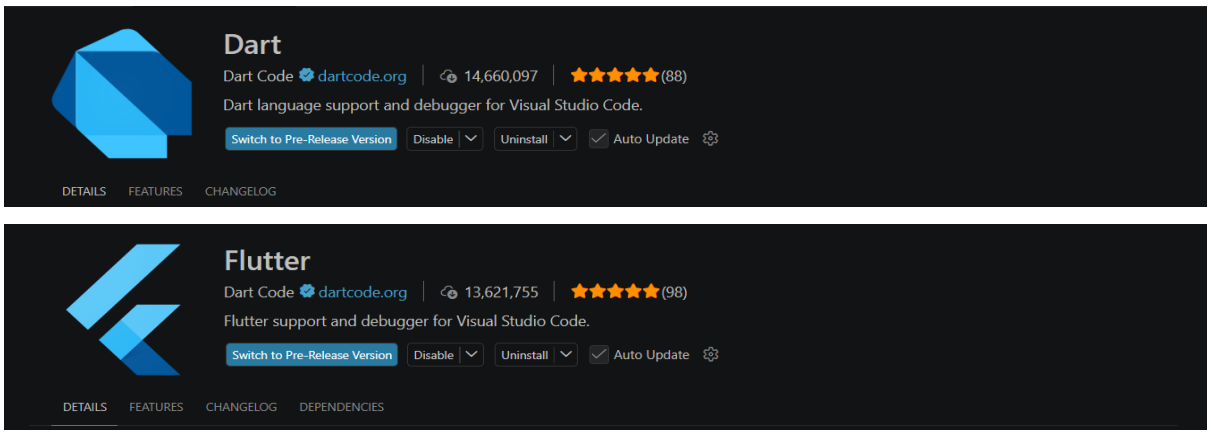


Figure 5.1.1.2: Flutter and Dart extensions from VS Code Marketplace

5.1.1.3 Android SDK and Device Tooling Setup

Android Studio was downloaded from: https://developer.android.com/studio.

Installation used default settings, with required SDK components installed through SDK Manager (Android SDK Platform, Build-Tools, Platform-Tools, and Emulator). For debugging, either an Android Emulator or USB-debugging-enabled physical device was configured.

Verification was performed using:

- flutter doctor (Android toolchain section)
- adb devices

Installation was considered successful when flutter doctor marked Android toolchain as available and adb devices detected an emulator or device.

```
C:\Users\Administrator>adb devices
List of devices attached
emulator-5554   device
```

Figure 5.1.1.3: Android SDK Setup Verification

5.1.1.4 Firebase CLI Setup

Firebase CLI setup followed the official documentation: https://firebase.google.com/docs/cli. Node.js LTS was installed first from https://nodejs.org/, then Firebase CLI was installed globally. Default installation settings were used.

Verification was performed using:

- firebase --version
- firebase login

Installation was successful when the CLI returned a valid version and authentication completed without errors.

```
C:\Users\Administrator>firebase --version
15.15.0

C:\Users\Administrator>firebase login
Already logged in as iamboonnnnn@gmail.com
```

Figure 5.1.1.4: Firebase CLI Setup verification

5.1.1.5 FlutterFire CLI and Firebase Project Linking

FlutterFire CLI setup followed: https://firebase.google.com/docs/flutter/setup. The tool was installed with default settings and then used to connect the project to Firebase. The SchoolWay Firebase project was configured using the project identifier schoolway-04286806.

Verification was performed using:

- flutterfire --version
- flutterfire configure

Installation and configuration were successful when Firebase options were generated, and platform configurations were linked correctly.

```
D:\UTAR\schoolway>flutterfire --version
1.3.2

D:\UTAR\schoolway>flutterfire configure
? You have an existing `firebase.json` file and possibly already configured
your project for Firebase. Would you prefer to reuse the values in your exis
? You have an existing `firebase.json` file and possibly already configured
your project for Firebase. Would you prefer to reuse the values in your exis
✔You have an existing `firebase.json` file and possibly already configured
your project for Firebase. Would you prefer to reuse the values in your exis
ting `firebase.json` file to configure your project? · yes
```

Figure 5.1.1.5: FlutterFire CLI Setup verification

5.1.1.6 Project Dependency Installation and Build Validation

After environment preparation, project dependencies were retrieved and the build process was validated.

Verification commands:

- flutter pub get
- flutter analyze
- flutter run

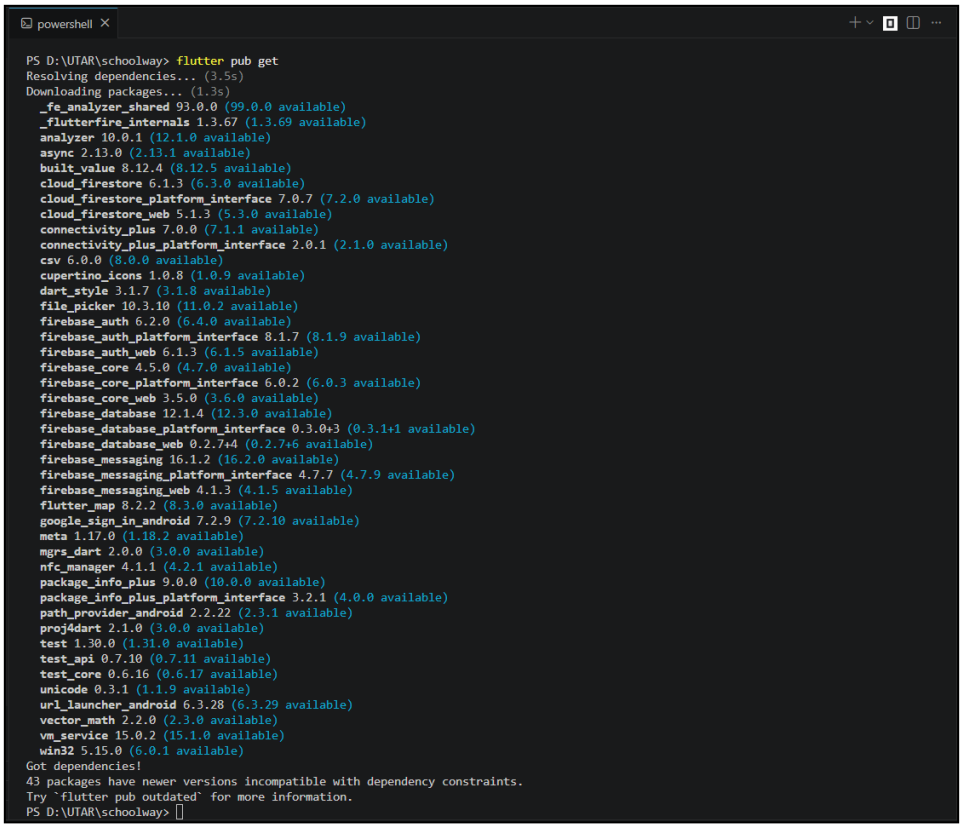


Figure 5.1.1.6: Dependency installation completion in project terminal.

Setup was confirmed as successful when dependencies were resolved, static analysis completed without blocking issues, and the application launched on the selected target platform.

Overall, the environment preparation process ensured a consistent and reproducible baseline for implementation, testing, and integration of SchoolWay modules.

5.1.2 Integration of External Services

SchoolWay integrates several external services to support authentication, communication, real-time tracking, and operational attendance workflows. These services include Firebase Authentication, Firebase Cloud Messaging, map and location services, and NFC services. Their integration ensures that system access is secure, transport updates are timely, and operational records are captured accurately.

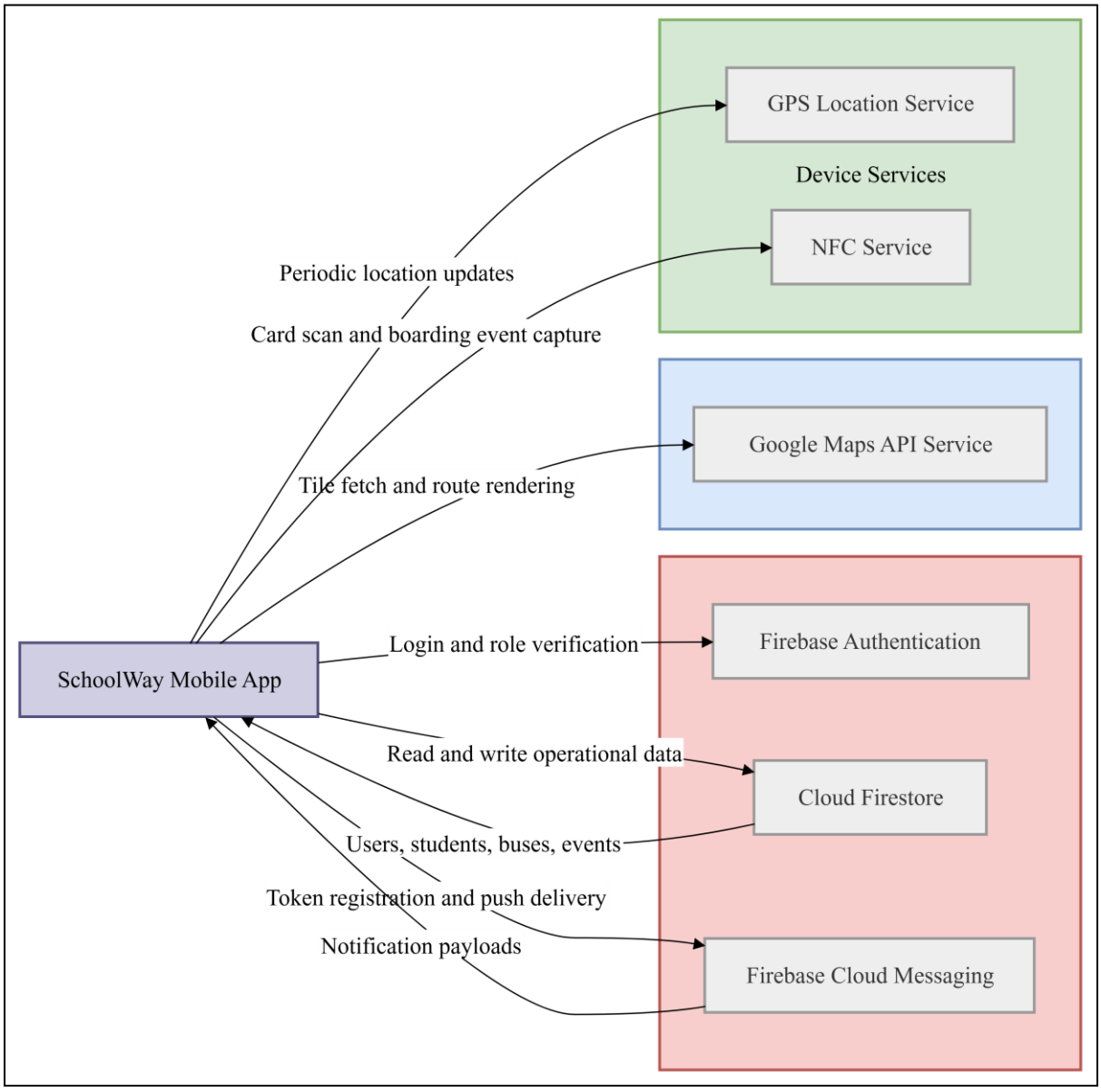


Figure 5.1.2: External service architecture for authentication, messaging, map, location, and NFC

5.1.2.1 Authentication Service Integration

Firebase Authentication is configured for account-based login and Google sign-in. Platform credentials are configured for Android. After successful sign-in, user profile verification and role assignment are performed before navigation.

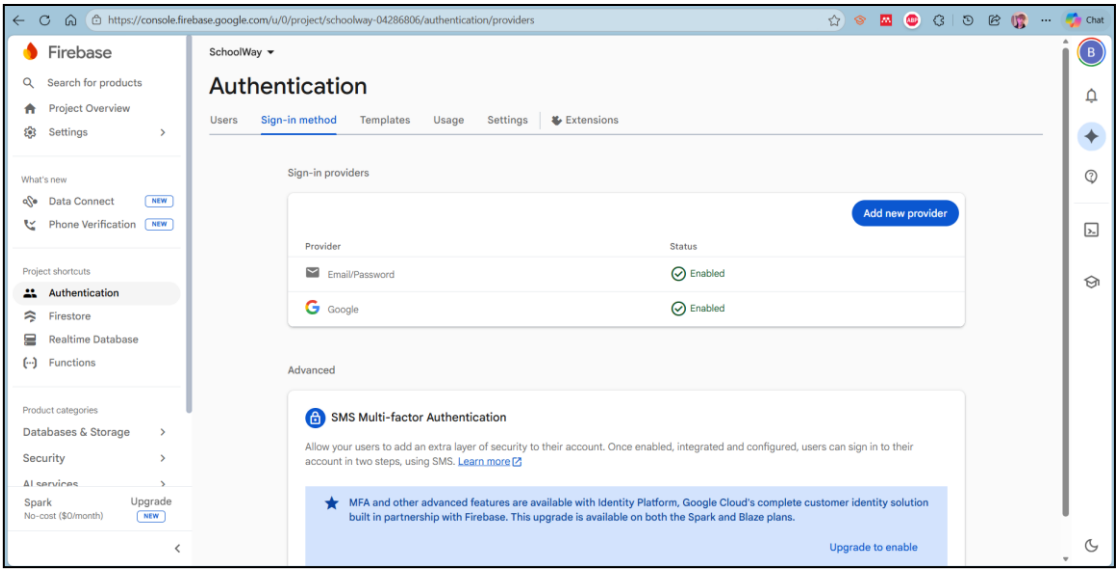


Figure 5.1.2.1: Firebase Authentication provider setup screen.

Installation and setup:

Firebase Authentication was prepared by creating and configuring a Firebase project in the Firebase Console, then enabling the required sign-in providers under the Authentication module. Email and password authentication was enabled for account-based access, and Google sign-in was enabled for federated login. The Android platform registration was completed in project settings so authentication traffic can be associated with the correct application identity.

Integration in application:

The authentication SDK was integrated into SchoolWay so users can sign in through either email credentials or Google sign-in. After successful authentication, the application retrieves the user identity and checks the corresponding profile record before routing the user to the correct module. This integration ensures that only authenticated users proceed to operational screens.

Configuration details:

Authentication configuration includes provider enablement, platform registration, and OAuth-related values for Google sign-in. These values are stored in the project configuration and should not be disclosed in full in documentation.

Verification:

Successful integration was verified by executing sign-in tests using valid credentials and confirming that authentication sessions were created correctly. Additional checks confirmed that signed-in users were routed according to their role information and that invalid or unregistered accounts were handled safely. This verification confirms that the authentication service is active and correctly integrated with user access flow.

5.1.2.2 Push Notification Service Integration

Firebase Cloud Messaging is initialized during startup. On successful authentication, the device token is obtained and synchronized with the user profile for push delivery readiness.

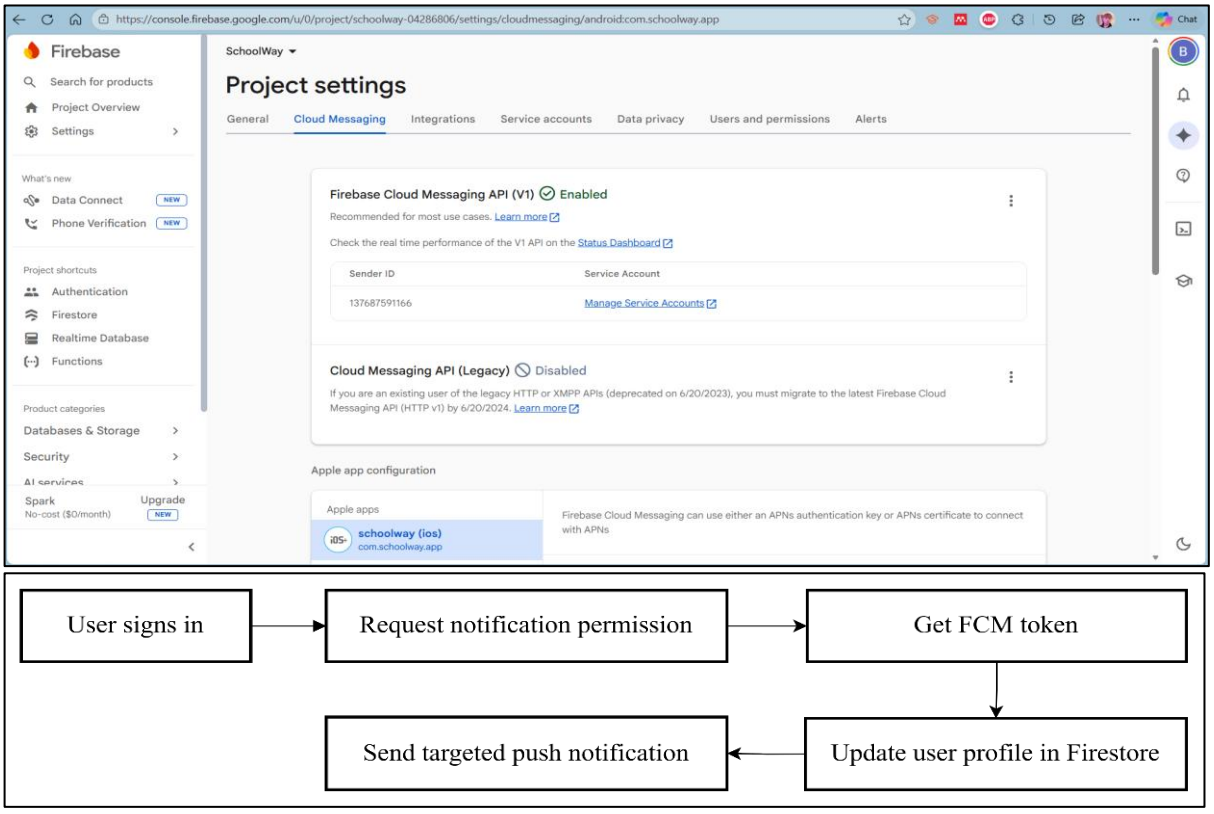


Figure 5.1.2.2: Cloud Messaging configuration and token registration flow.

Installation and setup:

Firebase Cloud Messaging was enabled in the same Firebase project through the Cloud Messaging module in Firebase Console. Notification support was prepared for Android so the application can request permission and receive push messages at runtime. The messaging service setup was linked with the application as part of Firebase integration.

Integration in application:

The messaging SDK was integrated into SchoolWay during startup initialization, so notification services are prepared before normal operation. After user authentication, the system retrieves a device token and synchronizes it with the corresponding user profile record. This enables targeted message delivery based on authenticated user identity.

Configuration details:

Configuration includes notification permission behaviour, token registration flow, and platform messaging parameters. Any token or key referenced in documentation should be masked to protect security-sensitive values while still explaining how token-based delivery works.

Verification:

Integration was verified by confirming successful permission prompts, token retrieval, and token synchronization into the user profile. Test push notifications were then sent to validate end-to-end delivery behaviour. Results confirmed that messaging services are correctly configured and operational.

5.1.2.3 Map and Location Service Integration

Map display uses Google Maps API tiles through a Google Maps Flutter plugin. Location services are initialized with runtime permission checks and high-accuracy position retrieval for driver tracking workflows.

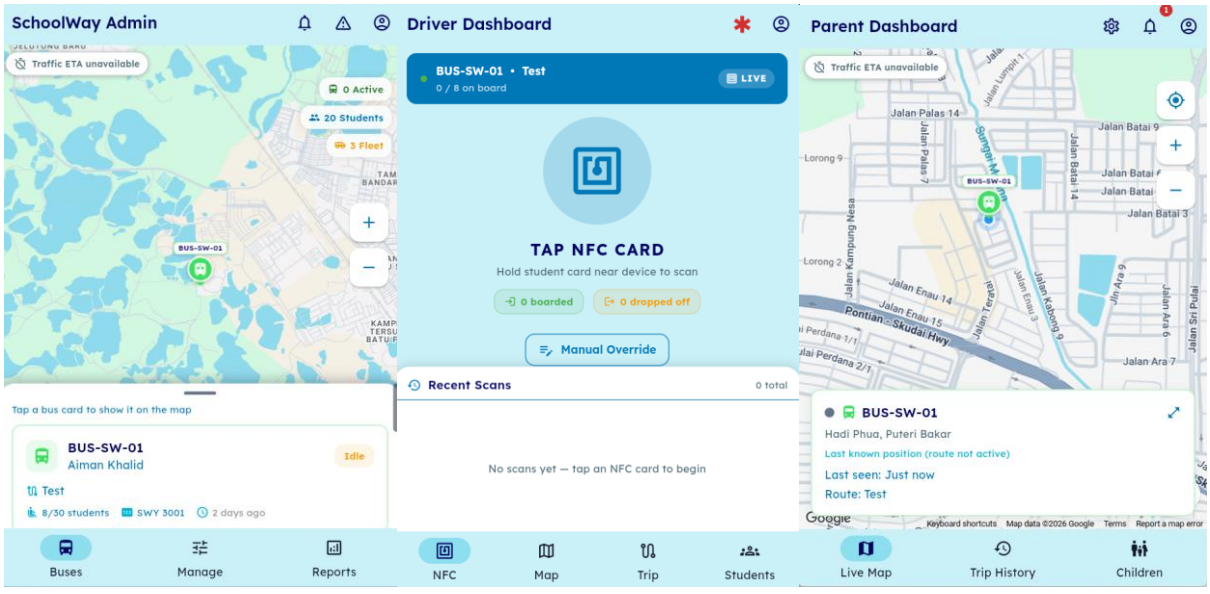


Figure 5.1.2.3: Google Maps through Google Maps Flutter plugin

Installation and setup:

Map and location functions were prepared by adding map rendering and geolocation packages to the project dependency configuration. Google Maps API was selected as the tile provider for map visualization, and location access requirements were prepared to allow runtime permission handling. Android location readiness was configured to support GPS-based tracking workflows.

Integration in application:

Map rendering was integrated into SchoolWay to display route context and vehicle positions in role-specific views. Location retrieval was integrated for high-accuracy tracking during driver operation, and position updates were reflected in live monitoring interfaces. This integration supports real-time visibility for both administrators and parents.

Configuration details:

Configuration includes map tile endpoint settings, location accuracy parameters, and runtime permission checks. The current map implementation using OpenStreetMap does not require a private API token. If a token-based map provider is adopted in future, key storage and masking requirements should be documented as part of security controls.

Verification:

Integration was verified by checking that map tiles loaded correctly, location permission prompts appeared as expected, and GPS coordinates were retrieved after user consent. Live map views were then observed to confirm that position updates were reflected during active operation. These checks verified that map and location services were functioning correctly.

5.1.2.4 NFC Service Integration

NFC access is configured for supported platforms to read student card identifiers. Reader integration is used in the driver workflow for boarding and alighting records.

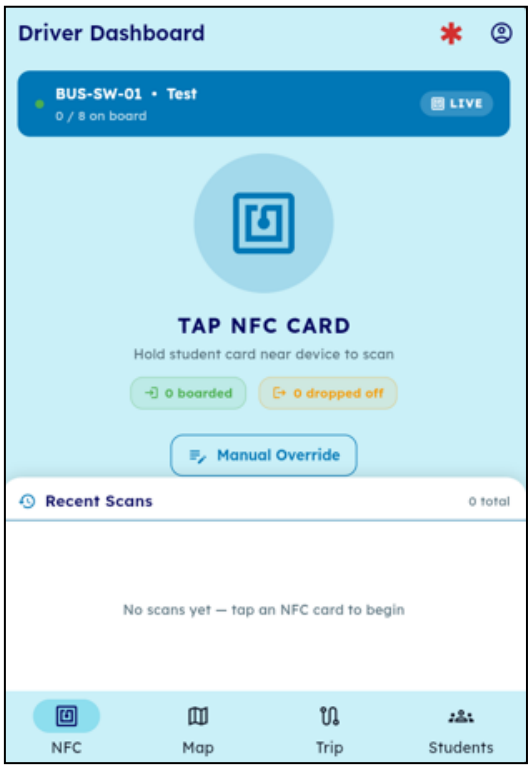


Figure 5.1.2.4: NFC access is usable on supported platforms

Installation and setup:

NFC support was prepared by adding NFC-related libraries and enabling NFC capability on supported mobile test devices. Setup focused on ensuring that NFC reading could be initiated in compatible runtime environments. This provided the required hardware-software base for card scanning workflows.

Integration in application:

NFC scanning was integrated into the driver workflow so card scans can be captured during boarding and alighting. The scanned value is decoded and matched against student data before event records are created. This integration enables attendance events to be captured as part of real-time bus operations.

Configuration details:

NFC configuration focuses on platform support checks, scan handling behaviour, and identifier matching logic. No external cloud API token is required for NFC scanning in the current

implementation. Because NFC capability depends on device hardware, unsupported environments must use fallback behaviour without scan execution.

Verification:

Integration was verified by performing scans with both valid and invalid NFC cards. Valid scans were confirmed through successful student matching and event recording, while invalid scans correctly triggered warning behaviour without incorrect data writes. Immediate UI updates after scan events further confirmed successful NFC workflow integration.

5.1.3 Database Configuration

SchoolWay uses Cloud Firestore as the primary operational database. The data model is organized into collections for users, buses, students, route sessions, boarding events, GPS tracking, emergency alerts, notifications, and planned routes. Composite indexes were configured to support filtered and ordered queries for dashboards, trip history, and activity reports. Firestore security rules were configured to require authenticated access across core collections. During development, permissive authenticated write rules were used to accelerate module integration and testing. Seed CSV datasets were prepared for buses, drivers, parents, students, and route assignments to speed up functional validation.

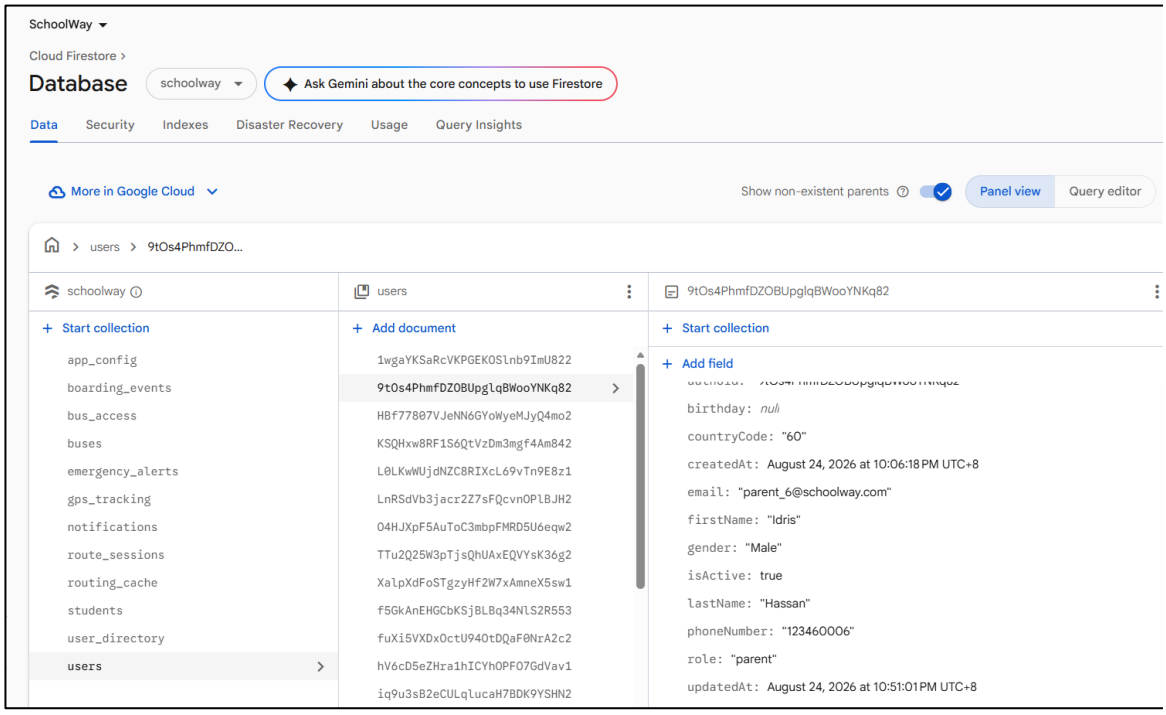


Figure 5.1.3.1: Firestore collection structure used by SchoolWay.

Figure 5.1.3.1 illustrates the Cloud Firestore database structure used in SchoolWay. The upper panel presents the actual Firestore console view, while the lower panel summarizes the logical relationships among core collections, including users, students, buses, route sessions, boarding events, GPS tracking, emergency alerts, notifications, and planned routes. This structure supports role-based operation, trip monitoring, and event recording by linking operational entities through identifiers such as busId, sessionId, and user-related references. The dual presentation provides both implementation evidence and conceptual clarity of the data model used in the system.

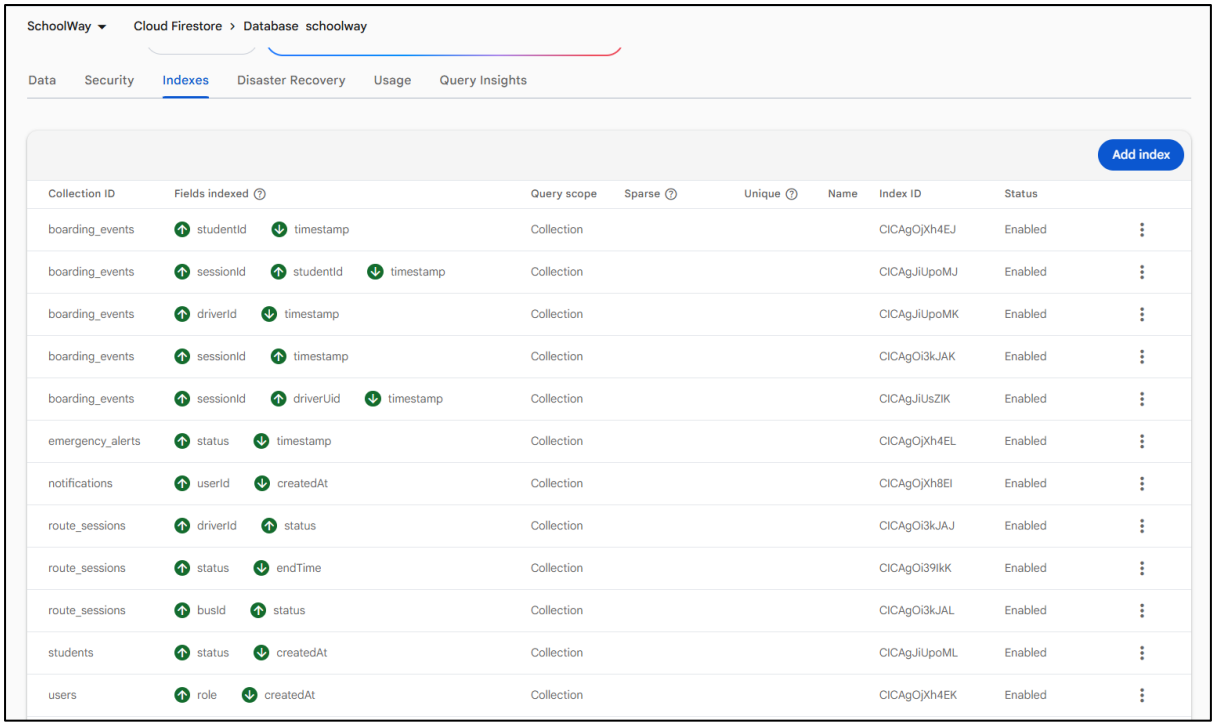


Figure 5.1.3.2: Composite index configuration for operational queries.

Figure 5.1.3.2 shows the composite index configuration defined for operational query performance in SchoolWay. These indexes are required to support filtered and ordered queries across high-usage datasets, especially for boarding events, route sessions, notifications, user lists, and emergency records. By predefining composite indexes, the system can execute dashboard and report queries efficiently without runtime index creation errors. This configuration is important for maintaining responsive data retrieval in live monitoring and historical analysis screens.

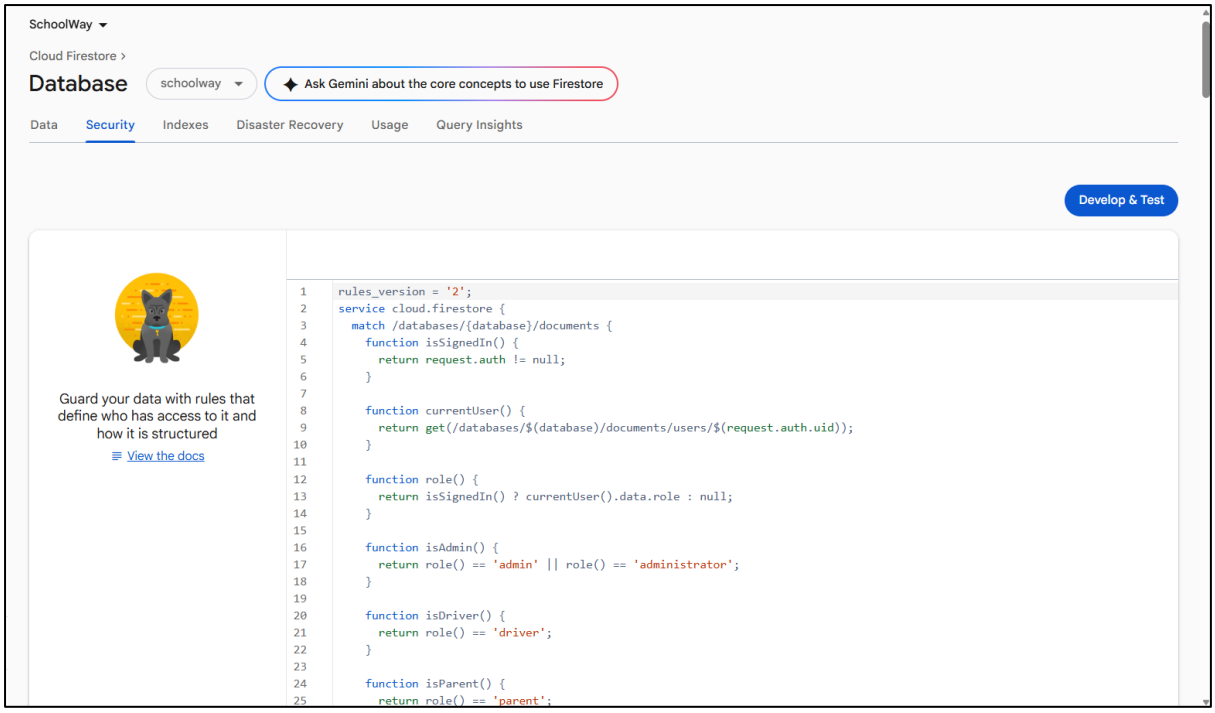


Figure 5.1.3.3: Firestore security rule setup in project console.

Figure 5.1.3.3 presents the Firestore security rule configuration used during the development stage. The rules enforce authenticated access for core collections so only signed-in users can read or write operational data. For faster feature integration and testing, write permissions were temporarily configured in a permissive authenticated mode, while destructive actions such as selected delete operations remain restricted. This setup provides a practical balance between development agility and baseline access control, and it forms the foundation for future production hardening with stricter role-based authorization.

5.1.4 Cloud Services

Cloud service setup for SchoolWay was implemented through a single Firebase project that serves as the central backend for authentication, configuration, and runtime service connectivity. Platform-specific registration was completed for Android, and the generated Firebase app identifiers and configuration values were integrated into the Flutter application so cloud services can initialize correctly at startup. In addition to project linking, startup flow design includes cloud bootstrapping, essential permission handling, and push messaging initialization before users enter operational modules.

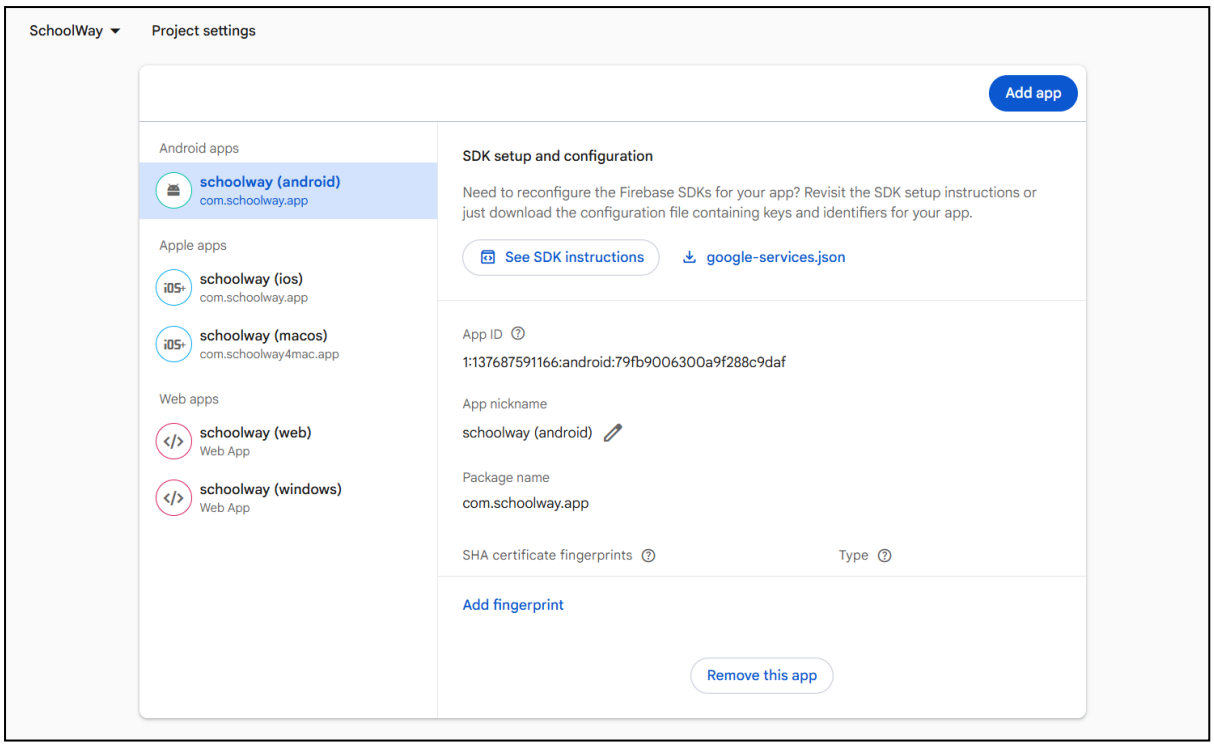


Figure 5.1.4.1: Firebase project overview and platform app registration.

Figure 5.1.4.1 shows the Firebase project overview and platform application registration used in SchoolWay. The figure confirms that the Android application is registered in the Firebase project with its platform identity and package mapping. This registration is required so Firebase services can correctly associate authentication, messaging, and other cloud interactions with the deployed mobile app. The figure therefore provides configuration evidence that cloud-platform linkage was completed before runtime integration.

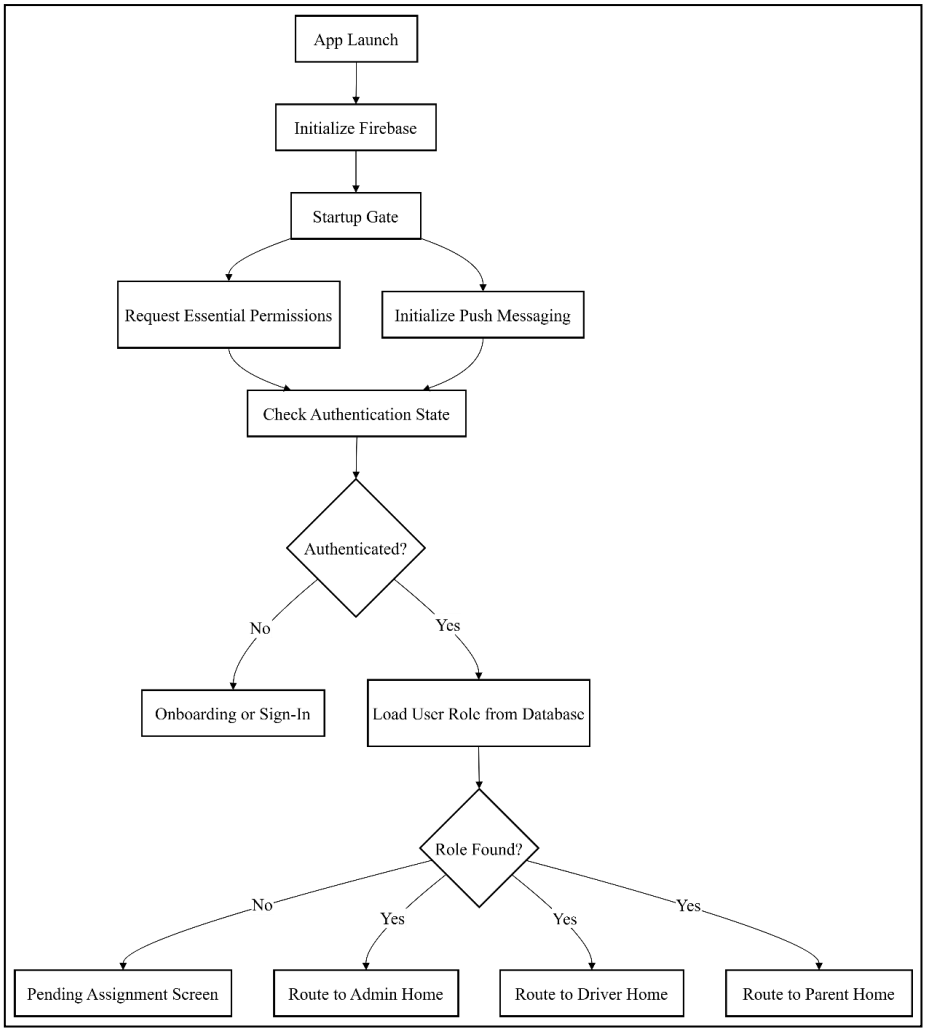


Figure 5.1.4.2: Application startup sequence with cloud initialization gate.

Figure 5.1.4.2 illustrates the application startup sequence with the cloud initialization gate. As shown in the flow, the app launches, initializes Firebase services, executes startup checks, requests essential permissions, and initializes push messaging before proceeding to authentication and role routing. This gated startup approach ensures that cloud-dependent features are prepared before user-facing operations begin, reducing runtime errors and improving initialization reliability across different user roles.

5.1.5 Deployment and Hosting

For Android deployment, the SchoolWay application was prepared for release distribution after the signing configuration and build environment were finalized. The deployment process began with build verification on the target platform, followed by dependency cleanup and package retrieval to ensure the project was in a stable state before release generation. As shown in Figure 5.1.5, the application was successfully built in release mode using the Android build pipeline, and the generated APK confirmed that the release artifact was produced correctly. This release build checkpoint verifies that the application can be packaged properly for Android deployment after configuration and dependency validation.

The deployment process also includes validation of authentication settings and cloud endpoint connectivity to ensure the application can connect correctly to Firebase services during runtime. After the release build is generated, the APK can be uploaded to a hosting or distribution channel for installation testing or production deployment. Post-deployment smoke testing is then carried out to confirm that sign-in, map loading, and core module navigation operate correctly on the target device. These checks ensure that the release package is functionally ready for distribution and that the application behaves as expected after deployment.

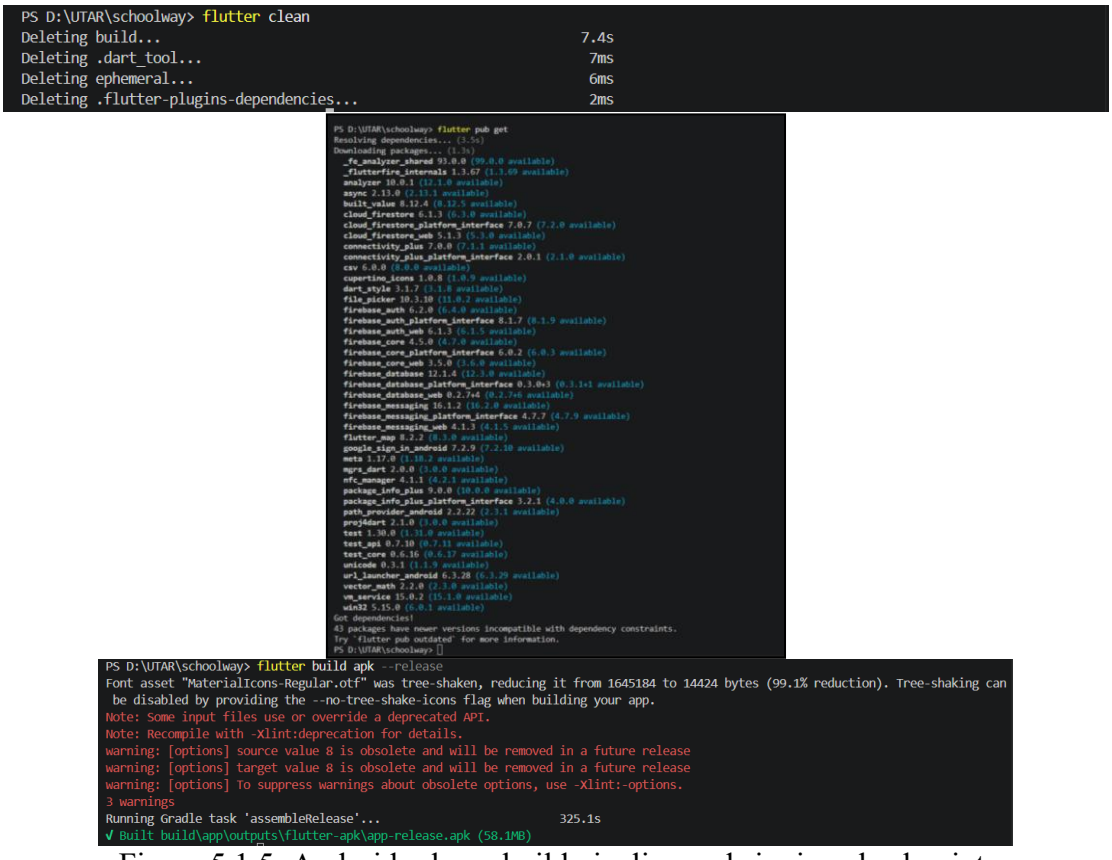


Figure 5.1.5: Android release build pipeline and signing checkpoint.

5.1.6 Monitoring and Analytics

System monitoring in SchoolWay currently relies on runtime error logging, operational dashboards, and Firestore-based activity reporting. The application captures key operational events and makes them accessible through role-specific monitoring interfaces. Administrative report screens summarize attendance events, bus activity, driver activity, and emergency records, providing practical operational oversight during development and pilot deployment phases.

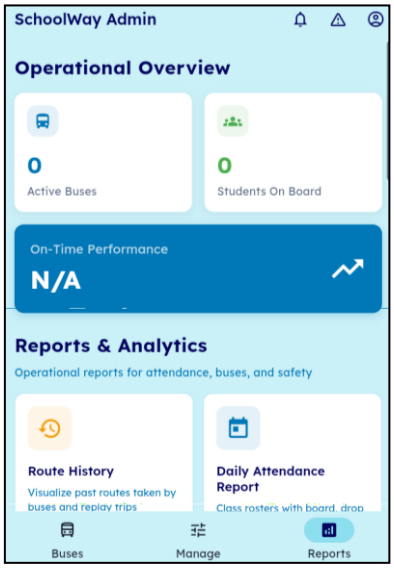


Figure 5.1.6: Admin reporting interface used for operational monitoring.

Figure 5.1.6 shows the Admin reporting interface used for operational monitoring, presenting four report cards that focus on specific monitoring areas: the Daily Attendance Report displays class rosters with student boarding, drop-off, or absence status, allowing administrators to tap the card to scan time-based data and verify attendance patterns across classes; Bus Activity summarizes boarding and drop-off NFC scan counts for each bus over the last seven days, enabling administrators to review transport volume and identify anomalies in bus usage; Driver Activity shows NFC scans attributed to each driver over the last seven days, helping administrators assess driver engagement and verify that attendance events are being recorded during operation; and Emergency Alert History displays recent driver-initiated emergency alerts with timestamps and resolution status, allowing administrators to monitor safety incidents and track response actions.

At the current development stage, a dedicated analytics software development kit (SDK) is not integrated into the system. This is identified as future enhancement work for supporting deeper user behaviour analysis, event funnel tracking, and trend-based insights. The current implementation focuses on real-time operational reporting rather than longitudinal analytics, which is appropriate for the pilot stage of development.

5.2 Implementation Details

This section focuses on technical realization of features, including module-level behaviour, data flow, and runtime processing.

5.2.1 Authentication and Role Routing

The authentication implementation supports both account-based login which is email and password and Google-based federated authentication.

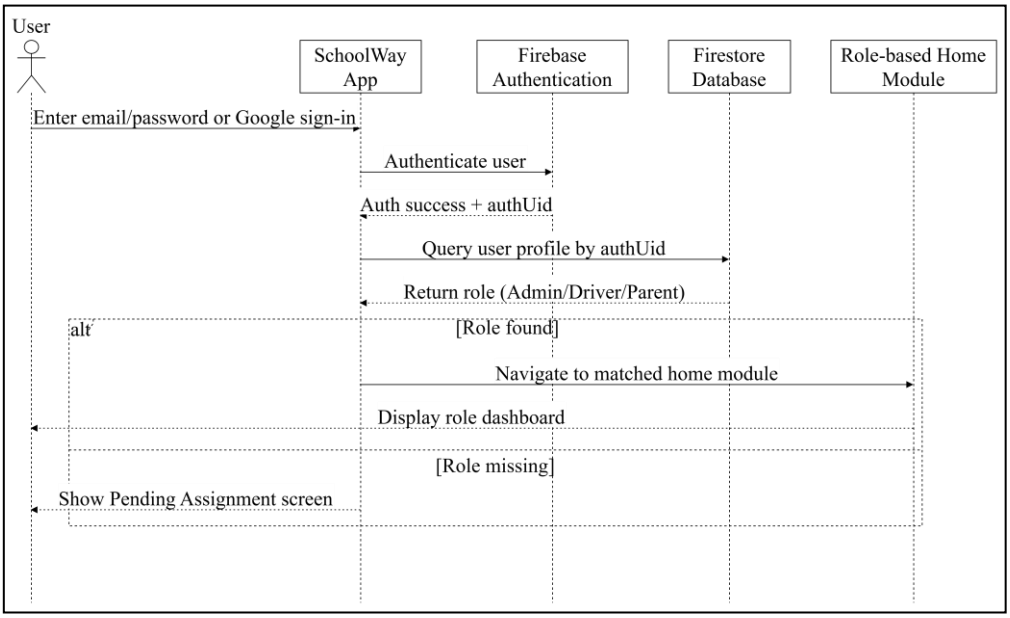


Figure 5.2.1: Authentication sequence from credential submission to role-based navigation.

Figure 5.2.1 illustrates the authentication sequence from credential submission through role-based navigation. When a user enters their credentials or initiates Google sign-in, the SchoolWay app submits these credentials to Firebase Authentication for validation. Upon successful authentication, Firebase returns an authentication token and a unique authUid. The application then queries the Firestore database to retrieve the user profile associated with that authUid. From this profile record, the system extracts the user's role metadata such as Admin, Driver, or Parent. With the role determined, the application navigates to the corresponding home module and displays the role-specific dashboard. If the role metadata is missing or has not yet been assigned, the system displays a Pending Assignment screen, preventing access to role-restricted features until administrative provisioning is complete.

This architecture ensures that users are routed to their correct operational context immediately upon login, while maintaining a controlled handoff mechanism for accounts awaiting role assignment.

5.2.2 Startup Gate and Permission Initialization

The startup gate is a critical initialization phase that executes before users can access any operational feature.

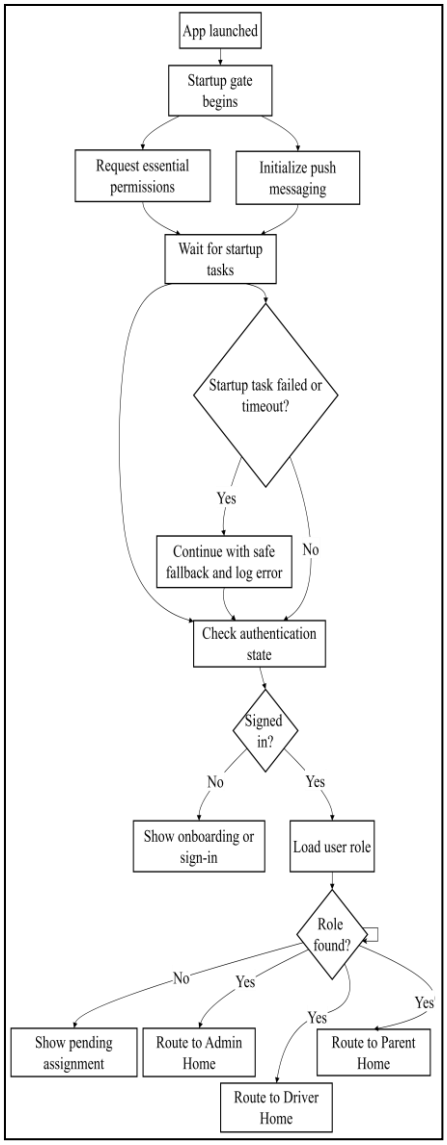


Figure 5.2.2: Startup gate flow for permission and messaging initialization

Figure 5.2.2 shows the startup gate flow, which begins immediately when the application is launched. The startup sequence initiates two concurrent initialization tasks: requesting essential device permissions (such as location and notification access) and initializing cloud push messaging infrastructure. These tasks run in parallel to minimize startup latency. The application then enters a waiting state until both initialization tasks complete. If either task fails or exceeds the timeout threshold, the system logs the error and continues with safe fallback mechanisms, ensuring that a single initialization failure does not prevent user access. After the startup gate tasks complete, the application checks the authentication state. If no user is signed in, the onboarding or sign-in screen is displayed. If a user is authenticated, the system loads their role metadata from Firestore. The application then validates whether a role has been assigned. Users with an assigned role (Admin, Driver, or Parent) are routed to their respective home modules; users without an assigned role are shown the pending assignment screen.

This design reduces the likelihood of late permission failures during operation, improves runtime reliability, and ensures that critical services are initialized before users begin interacting with transport monitoring features.

5.2.3 GPS Tracking Implementation

The GPS tracking module is activated when the driver starts an active route session, at which point the uploader is initialized with session context such as, busId and sessionId to ensure each location update is linked to the correct trip record.

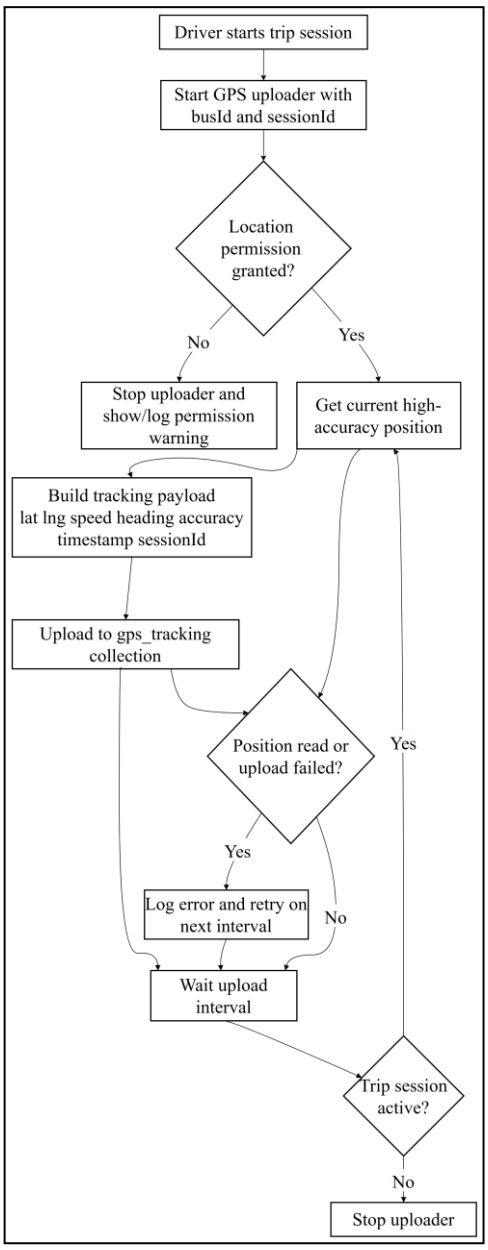


Figure 5.2.3: GPS upload cycle for active driver session

As illustrated in Figure 5.2.3, the module first checks whether location permission is granted. If permission is denied, the uploader does not start, and the condition is handled gracefully by recording a warning or error log without crashing the application. When permission is available, the system acquires high-accuracy position data at a configured periodic interval and constructs a tracking payload containing latitude, longitude, heading, speed, timestamp, and session linkage fields. This payload is then written to cloud tracking storage which is gps_tracking collection to support live fleet visibility and downstream reporting. If a position read or upload operation fails intermittently due to network or sensor instability, the failure is logged and the periodic scheduler continues at the next interval, rather than terminating the tracking loop. The cycle repeats while the trip session remains active and stops automatically when the route session ends.

This implementation provides resilient, session-bound GPS telemetry suitable for real-time monitoring during school transport operations.

5.2.4 NFC Boarding Event Implementation

The NFC boarding module is designed to capture student boarding activity quickly and reliably during live route operations.

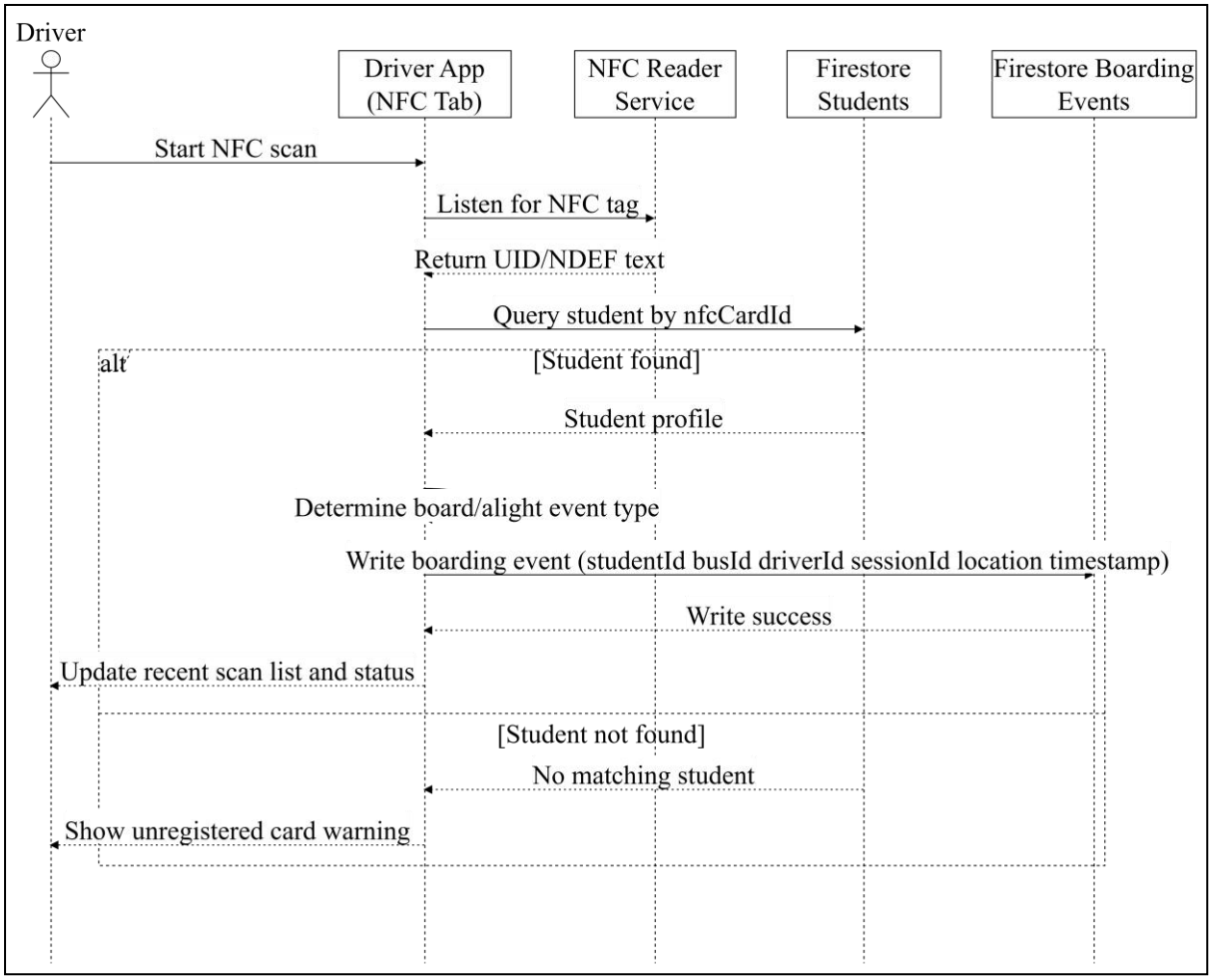


Figure 5.2.4: NFC scan-to-event recording pipeline.

As shown in Figure 5.2.4, the process begins when the driver starts NFC scanning from the Driver NFC interface. The NFC reader service listens for nearby tags and decodes either a supported NDEF text payload or a fallback identifier value (UID) when NDEF text is unavailable. The resulting card value is then used to query student records in Firestore based on the registered NFC card identifier. If a matching student profile is found, the system determines the event type (boarding or alighting) according to the operational context and writes a boarding event record containing studentId, driverId, busId, sessionId, timestamp, and location details. This record has persisted in the boarding events collection for attendance

tracking and reporting. If no matching student is found, the system handles the scan gracefully by showing an unregistered-card warning and avoiding invalid event insertion.

To support fast operational verification by drivers during pickup and drop-off, the UI provides immediate scan feedback after each tap and updates a recent event list in near real time. This allows the driver to confirm that each scan has been accepted, identify unmatched cards immediately, and continue passenger flow without manual reconciliation during the trip. Figure 5.2.4 therefore represents the full NFC scan-to-event recording pipeline, from tag read and student lookup to contextual event writing and on-screen confirmation.

5.2.5 Live Map Rendering Implementation

The live map implementation provides real-time visual tracking by consuming GPS updates from cloud tracking streams and rendering moving bus markers on OpenStreetMap tiles.

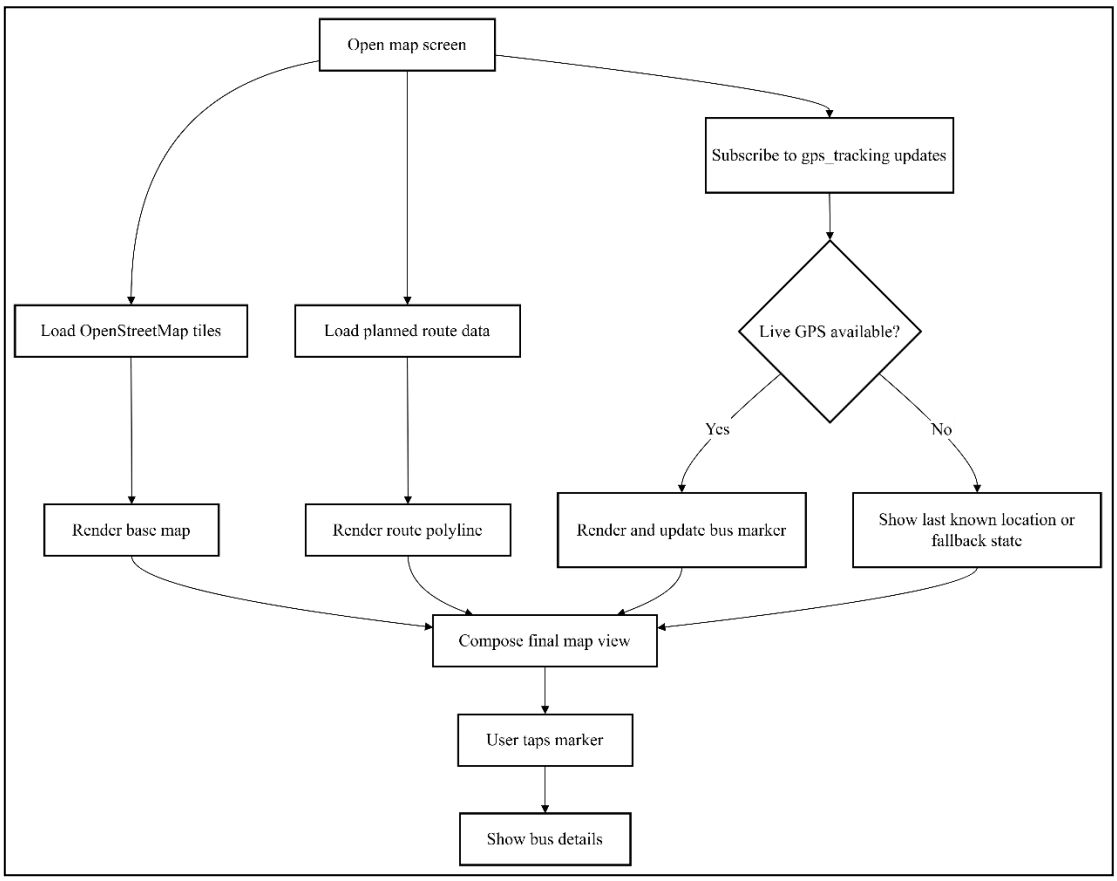


Figure 5.2.5: Real-time map rendering with route and position overlays.

As illustrated in Figure 5.2.5, once the map screen is opened, the module initializes three core data layers: base map tiles, planned route geometry, and live vehicle position updates. The base

layer renders OpenStreetMap tiles for geographic context, while the route layer draws assigned route polylines to show expected travel paths. In parallel, the live layer subscribes to gps_tracking updates and continuously refreshes bus markers when new coordinates are received. If live updates are temporarily unavailable, the interface falls back to the last known location or a safe placeholder state, preserving continuity for users.

Planned route overlays and fit-to-route camera logic improve situational visibility by automatically framing relevant route extents and active vehicle positions within the viewport. This reduces manual panning and helps users quickly assess whether a bus is on or off the expected path. For admin and parent modules, the composed map view emphasizes fleet or child-related transport visibility through route-plus-marker overlays and marker detail interactions. For driver modules, the map combines own-position context with assigned route visualization, supporting operational awareness during trip execution. Figure 5.2.5 therefore represents the full real-time rendering flow, from map initialization and data subscription to layered route and position

5.2.6 Notification Pipeline Implementation

The notification pipeline is implemented as a full flow from permission and token provisioning to in-app presentation and inbox state management.

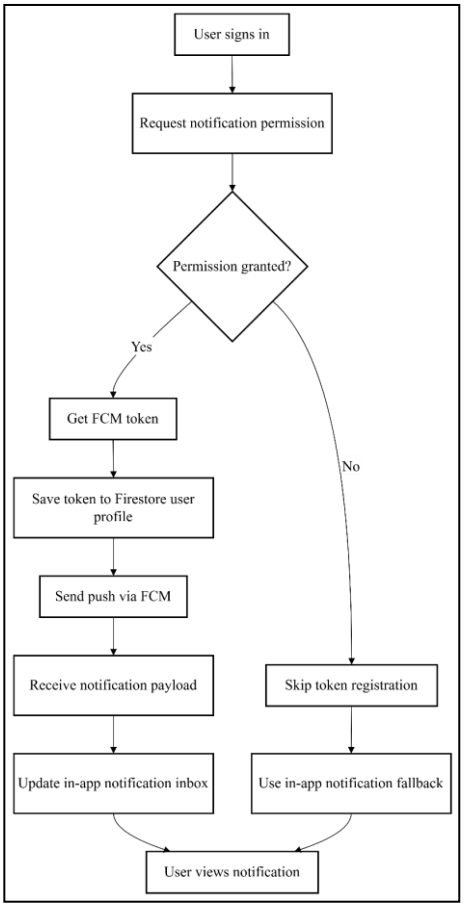


Figure 5.2.6: Notification flow from token sync to inbox presentation.

As shown in Figure 5.2.6, notification handling begins when a user signs in and the application requests notification permission from the device. If permission is granted, the system retrieves the Firebase Cloud Messaging (FCM) token and synchronizes it to the user profile in Firestore so cloud notifications can be targeted correctly. If permission is not granted, token registration is skipped and the application continues with an in-app fallback path to avoid blocking core usage. For permitted users, push payloads sent through FCM are received by the client and mapped into application notification records. Foreground notification behaviour is explicitly configured to ensure visibility while the app is active, so important transport and safety updates are not silently missed.

Parent-facing notification inbox screens are synchronized with cloud notification records, allowing users to view recent alerts in chronological order and inspect operational messages such as attendance or emergency-related updates. The inbox also supports read-state updates, where opened items are marked as read and reflected in the interface state, improving message tracking and reducing repeated alert noise. This architecture ensures reliable notification delivery across permission conditions while preserving a consistent in-app communication channel. Figure 4.2.6 therefore represents the end-to-end notification flow from token synchronization to inbox presentation and read-status handling.

5.2.7 CSV Import and Bulk Administration Implementation

The CSV import module provides a structured onboarding pipeline for high-volume administrative data setup.

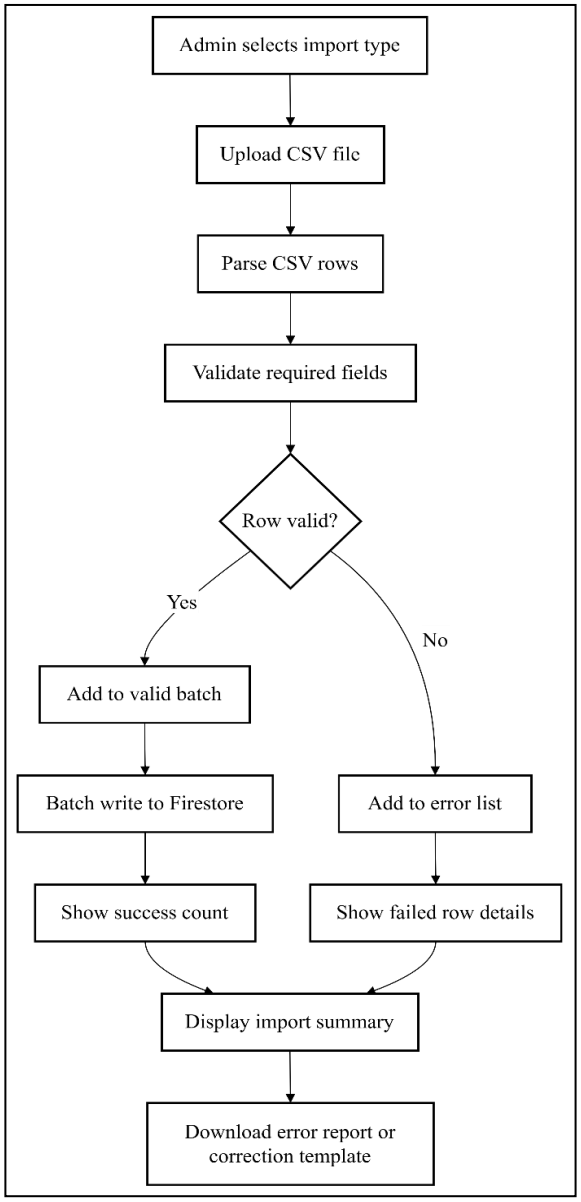


Figure 5.2.7: CSV import workflow and batch processing feedback.

As illustrated in Figure 5.2.7, the process begins when an administrator selects an import type and uploads a CSV file. The system then parses rows, validates required fields, and evaluates each record for schema and business-rule compliance before database insertion. Valid rows are grouped into a batch and written to Firestore using bulk operations for efficiency, while invalid rows are separated into an error list with row-level failure details. During execution, the interface provides progress visibility through success counts and failed-row reporting, followed by an import summary that allows administrators to assess completion quality. The workflow also supports downloading an error report or correction template so failed records can be fixed and re-imported quickly.

This implementation significantly accelerates initial system setup for students, users, buses, and planned route assignments, while preserving robustness through validation controls and transparent batch feedback.

5.3 System Operation

This section describes the operation of the SchoolWay mobile application based on the modules implemented for the Parent, Driver, and Administrator roles. When SchoolWay is launched, the application initializes its local preferences, Firebase services, authentication state, Cloud Firestore connection, and notification support. It then determines whether the user needs to complete onboarding, select a saved account, authenticate using an email and password or Google account, or continue an existing authenticated session.

After successful authentication, SchoolWay retrieves the user's authoritative profile from Cloud Firestore using the Firebase Authentication UID. The user is automatically directed to the Parent, Driver, or Administrator module according to the role assigned to the account. Users cannot select or modify their own role. Each module presents only the operations permitted for that role, while shared operations such as profile management, account switching, application preferences, and logout remain available to authenticated users.

5.3.1 Startup and Authentication Module

The startup and authentication module controls how users enter SchoolWay. It supports initial onboarding, email and password authentication, Google authentication, saved-account selection, password-reset validation, and role-based navigation. SchoolWay does not provide public self-registration because operational accounts and roles are provisioned by an authorised administrator.

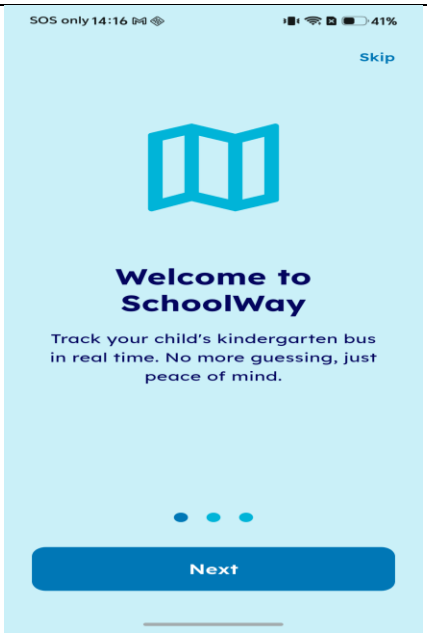


Figure 5.3.1.1: SchoolWay Onboarding Screen

Figure 5.3.1.1 shows the onboarding screen displayed when SchoolWay is opened for the first time or when onboarding has not been completed on the device. The screen introduces the purpose of the application and guides the user through the initial application experience. After completing the onboarding process, the user is directed to the available authentication options.

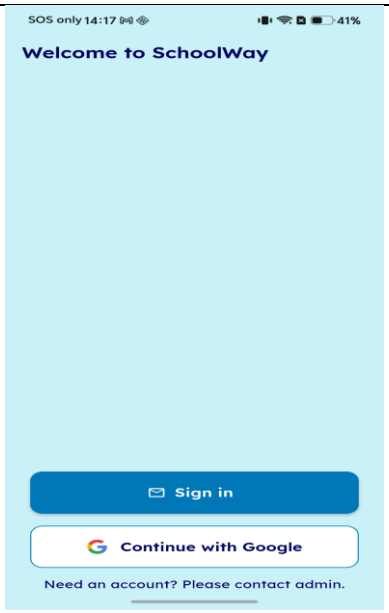


Figure 5.3.1.2: SchoolWay Authentication Options

Figure 5.3.1.2 shows the authentication options provided by SchoolWay. The user may continue using the email and password assigned to the provisioned SchoolWay account or authenticate through a linked Google account. Selecting an authentication method opens the corresponding authentication process. Public account registration is not provided because users and their roles must be provisioned by an administrator.

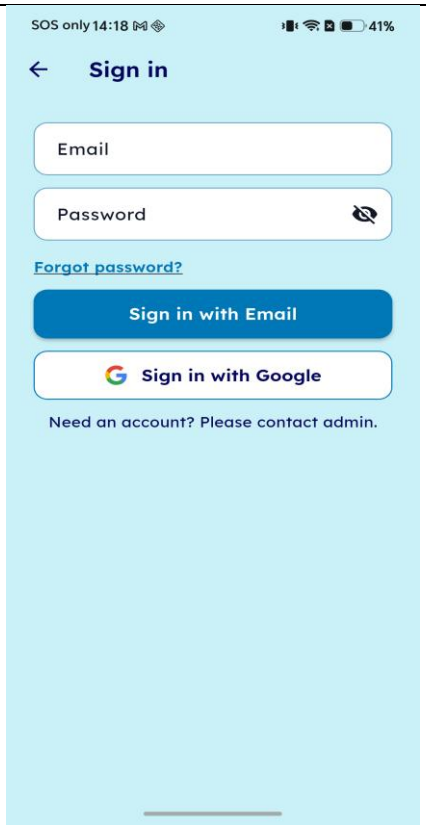


Figure 5.3.1.3: SchoolWay Email Sign-In Screen

Figure 5.3.1.3 shows the email and password sign-in screen. The user enters the email address and password associated with the provisioned SchoolWay account. The password remains concealed by default, although the visibility control can be used to verify the entered value. When the user selects **Sign in with Email**, SchoolWay validates the input and submits the credentials to Firebase Authentication. If authentication fails, an actionable error is displayed, and the user remains on the sign-in screen.

After successful authentication, SchoolWay uses the Firebase Authentication UID to retrieve the corresponding user profile from Cloud Firestore. The protected role and active status stored in the profile determine whether the user is directed to the Parent Dashboard, Driver Dashboard, or SchoolWay Admin Dashboard.

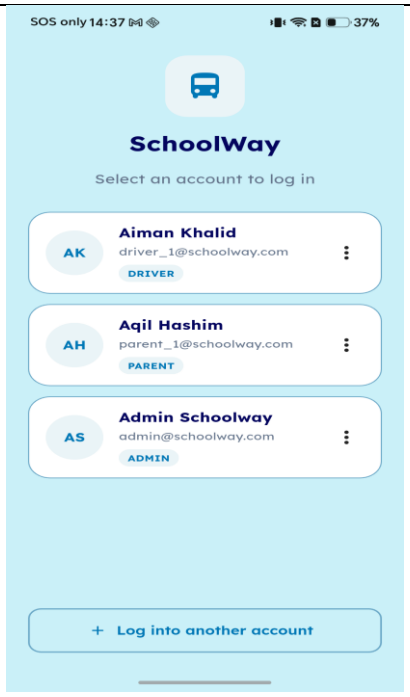


Figure 5.3.1.4: SchoolWay Saved-Account Selection Screen

Figure 5.3.1.4 shows the saved-account selection screen containing the demonstration Parent, Driver, and Administrator accounts. SchoolWay stores limited account information locally so that users who operate more than one authorised account can select the required profile without repeatedly entering its email address. Selecting a saved account does not bypass Firebase Authentication because the user must still complete the applicable password or Google re-authentication process.

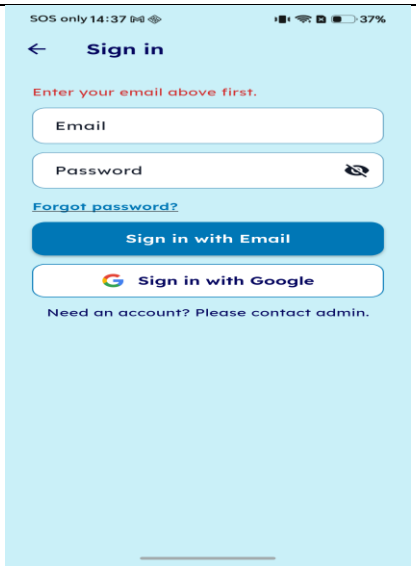


Figure 5.3.1.5: Password-Reset Email Validation Message

Figure 5.3.1.5 shows the validation message displayed when the password-reset operation is selected without first providing an email address. SchoolWay checks that the required email field contains a value before requesting Firebase Authentication to send a password-reset email. If the field is empty, the application displays a validation message and does not submit the request. This prevents an incomplete password-reset operation from being sent.

5.3.2 Parent Module

The Parent module allows an authorised parent to monitor linked children and their school transportation activities. The Parent Dashboard provides access to the Live Map, Trip History, Children, notifications, notification preferences, and profile operations. The information available to a parent is restricted to students linked to the authenticated parent's Firebase Authentication UID.

5.3.2.1 Live Bus Monitoring Operation

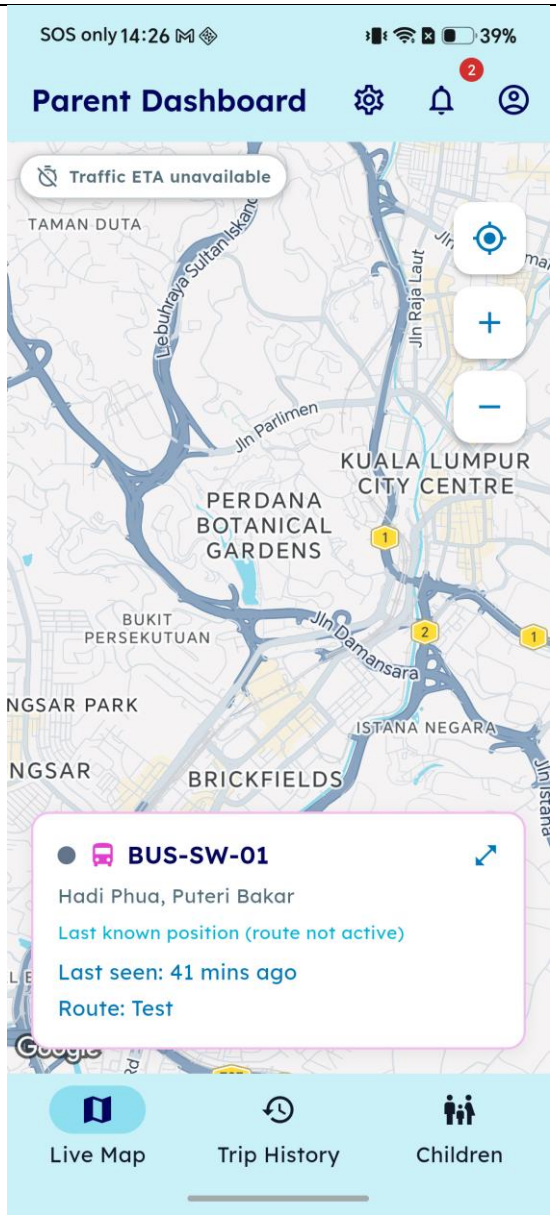


Figure 5.3.2.1: Parent Live Map Showing the Last-Known Bus Position

Figure 5.3.2.1 shows the Parent Live Map displaying the latest available position of the school bus assigned to the selected child. During this capture, no route session was active; therefore, SchoolWay correctly identified the location as the bus's last-known position instead of presenting it as a live update. Traffic ETA was also unavailable because an active route session and current route progress were not available at the time of capture.

During normal operation, when an authorised driver starts a route session, SchoolWay retrieves the active session, latest GPS position, assigned planned route, and saved road geometry from Cloud Firestore. Google Maps then displays the bus position and route information. When valid route and traffic information is available, SchoolWay can provide a traffic-aware ETA. If the required data is missing or outdated, the application displays an unavailable or last-known state instead of presenting an unsupported estimate.

5.3.2.2 Linked Children Operation

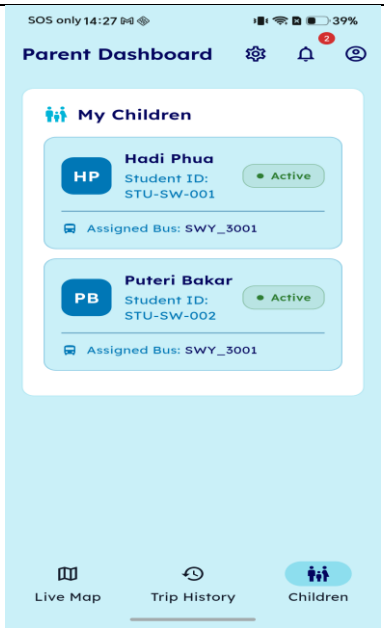


Figure 5.3.2.2: Parent Linked-Children Screen

Figure 5.3.2.2 shows the Children screen containing the students linked to the authenticated parent. SchoolWay retrieves student records whose parentAuthUids field contains the current Firebase Authentication UID. The interface displays the available student and transportation information, including the assigned bus, route, transportation status, and relevant pickup or drop-off details. A parent cannot access an unrelated student because the relationship is restricted by both the application query and Firestore security rules.

5.3.2.3 Trip and Boarding History Operation

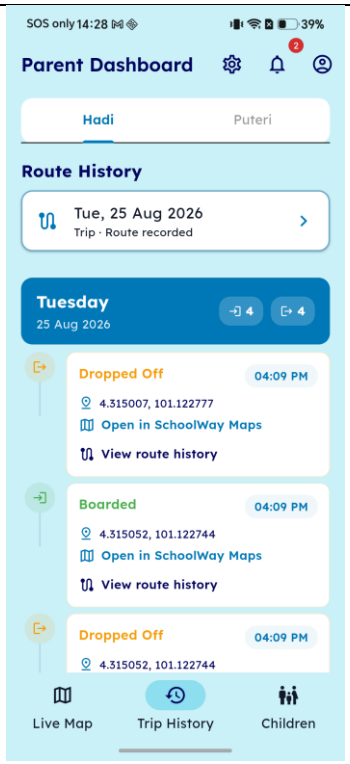


Figure 5.3.2.3: Parent Trip and Boarding History Screen

Figure 5.3.2.3 shows the Trip History screen containing transportation records for a linked child. SchoolWay retrieves the child's authorised boarding and alighting events from Cloud Firestore and arranges them in chronological order. Each available record identifies the transportation event and may contain its time, bus, route session, and recorded location. These records allow the parent to review when the child boarded or alighted from the assigned school bus.

The information in this screen is read-only for the parent. Parents can review their linked child's transportation history but cannot create, edit, or delete attendance evidence.

5.3.2.4 Notification Operation

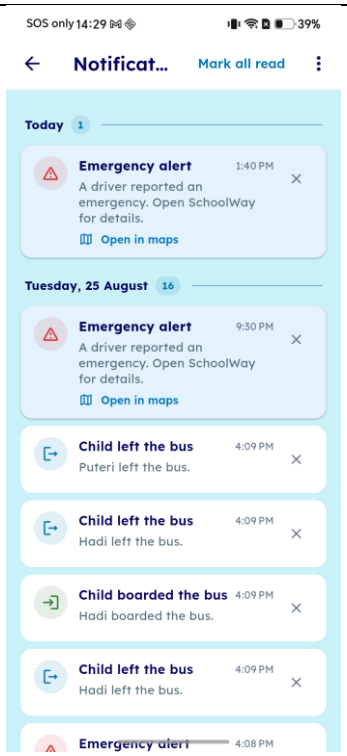


Figure 5.3.2.4.1: Parent Transportation Notification Inbox

Figure 5.3.2.4.1shows the Parent notification inbox containing transportation and emergency messages generated for the authenticated recipient. SchoolWay stores server-created notifications in Cloud Firestore and displays unread messages using an unread indicator. The parent can open a notification to review its details and mark it as read. Notifications associated with boarding, alighting, route information, or emergency events provide timely information concerning the linked child's transportation activity.

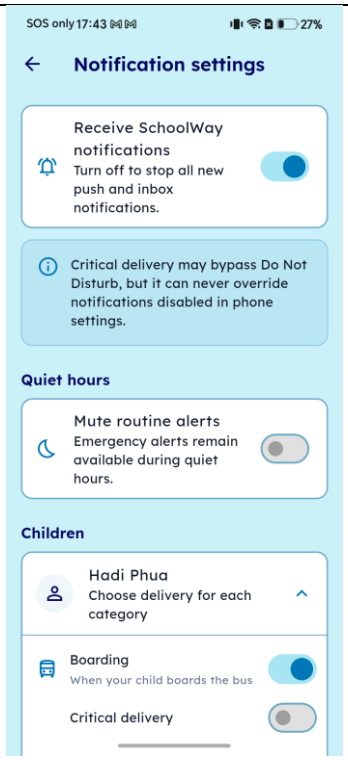


Figure 5.3.2.4.2: Parent Notification Preferences Screen

Figure 5.3.2.4.2 shows the notification preferences available to a parent. SchoolWay notification delivery is enabled, and the parent can configure supported notification categories for each linked child. The screen also provides quiet-hour controls and child-level delivery preferences for events such as boarding. These settings allow parents to control routine transportation notifications, while critical emergency handling remains subject to SchoolWay's safety policy and the notification permissions provided by the Android device.

5.3.3 Driver Module

The Driver module supports the daily transportation operations performed by an authorised school-bus driver. Its primary sections are NFC, Map, Trip, Students, and Profile. A driver must be assigned to a bus before starting a route session. GPS uploading, NFC attendance, manual attendance, incidents, and emergency operations are associated with the authenticated driver, assigned bus, and applicable route session.

5.3.3.1 Route Session Operation

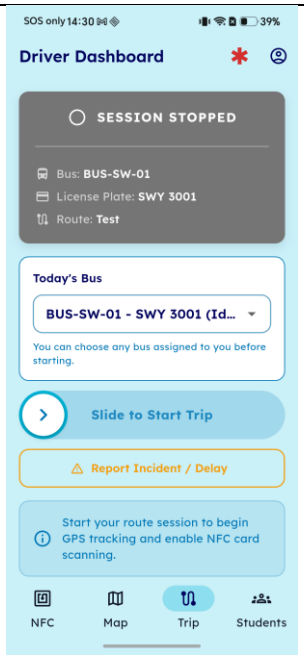


Figure 5.3.3.1.1: Driver Route Session Before Trip Activation

Figure 5.3.3.1.1 shows the Driver Trip screen before a route session is started. The interface identifies the bus and planned route assigned to the authenticated driver. Before starting a journey, the driver selects the required bus and trip direction. SchoolWay verifies the driver-bus assignment and checks the current operational state before creating a new route-session record in Cloud Firestore.

Before session activation, session-bound operations such as live GPS uploading and student NFC scanning remain disabled. This prevents transportation evidence or live location information from being recorded without a valid driver, bus, and route-session context.

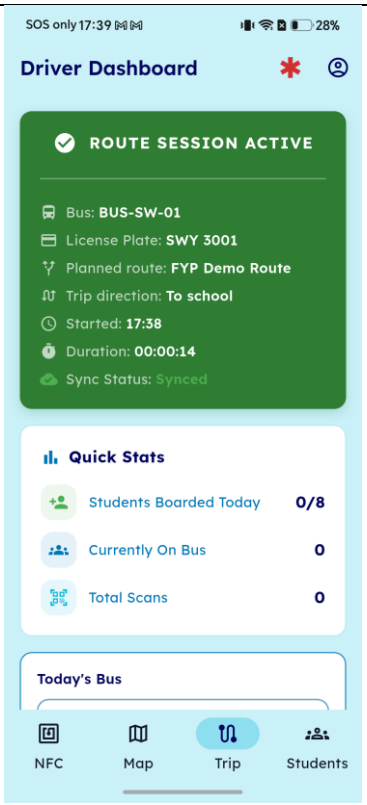


Figure 5.3.3.1.2: Active Driver Route Session

Figure 5.3.3.1.2 shows an active route session for BUS-SW-01. The screen identifies the licence plate as SWY 3001, the planned route as FYP Demo Route, and the trip direction as To school. It also displays the session start time, current duration, transportation statistics, and synchronized status.

Once the session is active, SchoolWay enables session-bound GPS tracking, NFC scanning, student attendance operations, incident reporting, and emergency reporting. At the end of the journey, the driver uses the end-trip control. SchoolWay checks the current student transportation state before recording the end time and completion status. After completion, the GPS uploader is stopped and the bus returns to the Idle state.

5.3.3.2 Driver Map and GPS Operation

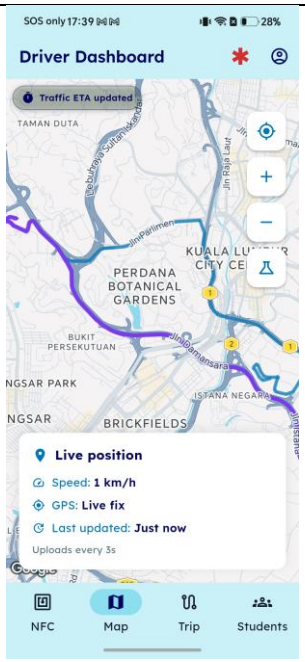


Figure 5.3.3.2: Driver Live GPS Map with Traffic ETA

Figure 5.3.3.2 shows the Driver Map during an active route session. The map displays the saved road geometry for the planned route and indicates that the traffic ETA has been updated. The information panel confirms that the device has obtained a live GPS fix and shows the current speed, latest update time, and three-second GPS upload interval.

During the active session, SchoolWay associates each GPS update with the authenticated driver, assigned bus, and current route session. The latest authorised latitude, longitude, speed, heading, accuracy, and timestamp are written to Cloud Firestore.

5.3.3.3 NFC Boarding and Alighting Operation

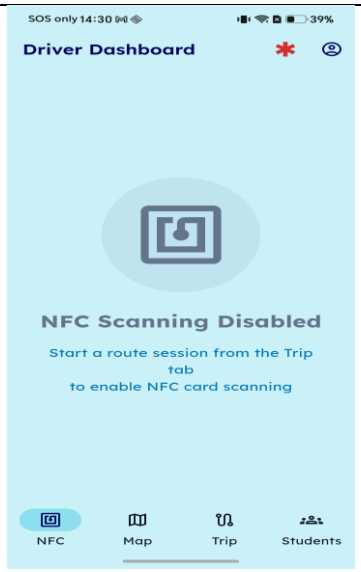


Figure 5.3.3.3.1: NFC Scanning Disabled Before Route Session Activation

Figure 5.3.3.3.1 shows that NFC attendance scanning is disabled when no driver route session is active. SchoolWay requires an authorised driver, assigned bus, and active route session before accepting an NFC card. This prevents a student attendance record from being created without a valid transportation context.

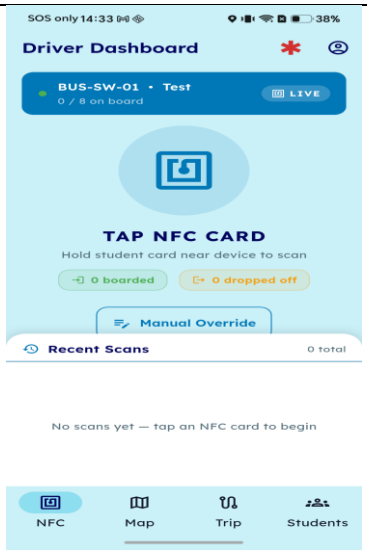


Figure 5.3.3.3.2: NFC Scanning Enabled During an Active Route Session

Figure 5.3.3.3.2 shows the NFC scanning interface enabled after the driver starts a valid route session. In this state, the driver can present a registered student NFC card to the Android device. SchoolWay reads the NFC identifier, searches for the student assigned to the active bus, and determines whether the event should be treated as boarding or alighting based on the student's latest valid transportation status.

A successful physical NFC result was not included in these figures because no registered NFC card was presented during the capture process. Therefore, Figures 5.3.3.3.1 and 5.3.3.3.2 demonstrate the disabled and enabled NFC interface states rather than a completed NFC attendance record.

5.3.3.4 Student Roster and Manual Attendance Operation

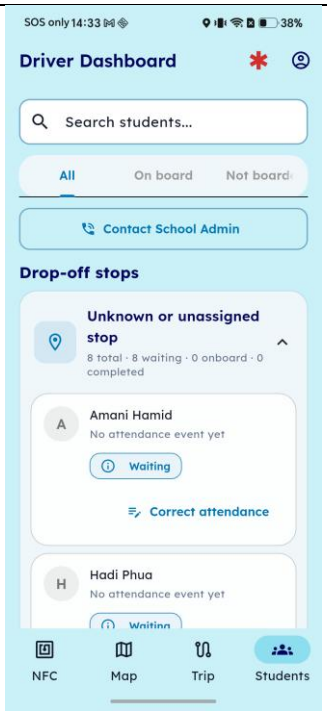


Figure 5.3.3.4.1: Driver Student Roster

Figure 5.3.3.4.1 shows the student roster associated with the driver's assigned bus. The interface groups students according to their latest transportation status, allowing the driver to determine which students are waiting, on board, or already dropped off. Available service-stop information also assists the driver in checking the expected pickup or drop-off sequence.

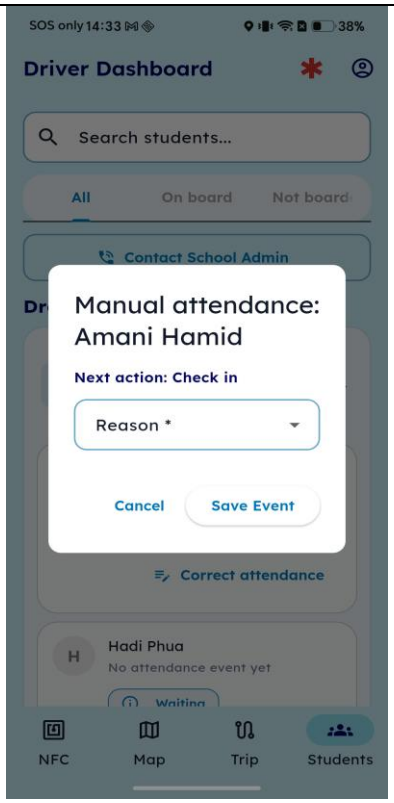


Figure 5.3.3.4.2: Driver Manual Attendance Dialog

Figure 5.3.3.4.2 shows the manual attendance dialog used when an NFC card cannot be presented or scanned. The driver selects the required attendance action and must provide a reason before SchoolWay creates the record. The application marks the resulting event as manual and stores the justification so that it can be distinguished from an NFC-generated event.

During the demonstrated route session, manual BOARD and ALIGHT records were created for Amani Hamid using the reason Card forgotten. These records were subsequently made available through the Administrator Attendance Corrections screen. They must be described as manual attendance records and not as successful NFC scans.

5.3.3.5 Incident and Emergency Operation

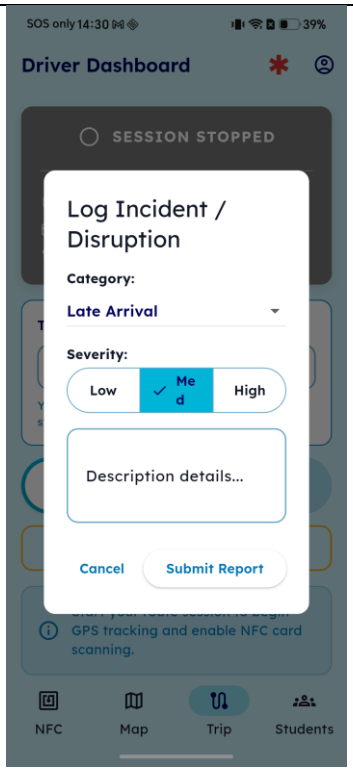


Figure 5.3.3.5.1: Driver Incident-Reporting Dialog

Figure 5.3.3.5.1 shows the non-emergency incident-reporting dialog. The driver can select the appropriate category and severity and provide a description of the transportation incident or delay. SchoolWay associates the submitted incident with the authenticated driver, assigned bus, and occurrence time. The resulting record can later be reviewed by an authorised administrator.

The figure demonstrates the reporting interface only. No false operational incident was submitted during the screenshot process.

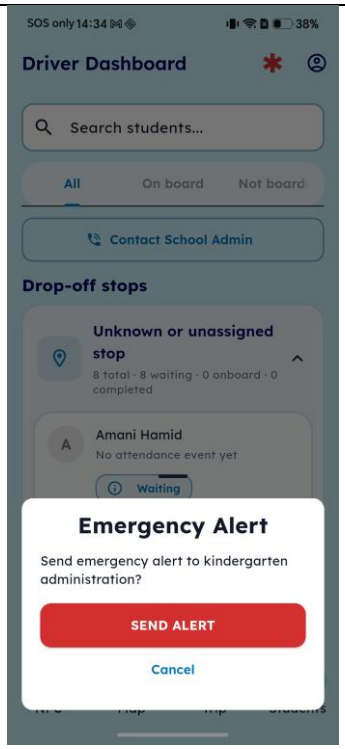


Figure 5.3.3.5.2: Driver Emergency-Alert Confirmation

Figure 5.3.3.5.2 shows the confirmation displayed before a driver sends an emergency alert. SchoolWay requires confirmation because the operation may create an emergency record and notify authorised administrators and affected parents. If the driver confirms the operation, the application records the driver, bus, applicable route session, timestamp, and available GPS location before initiating the notification process.

5.3.4 Administrator Module

The Administrator module maintains the operational information required by the Parent and Driver modules. Its main navigation contains the Buses, Manage, Reports, and Profile sections. Administrators can monitor the fleet, provision users, maintain student and bus information, configure assignments, prepare planned routes, import or export supported data, review transportation history, correct attendance records, and resolve emergency alerts.

5.3.4.1 Live Fleet Monitoring Operation

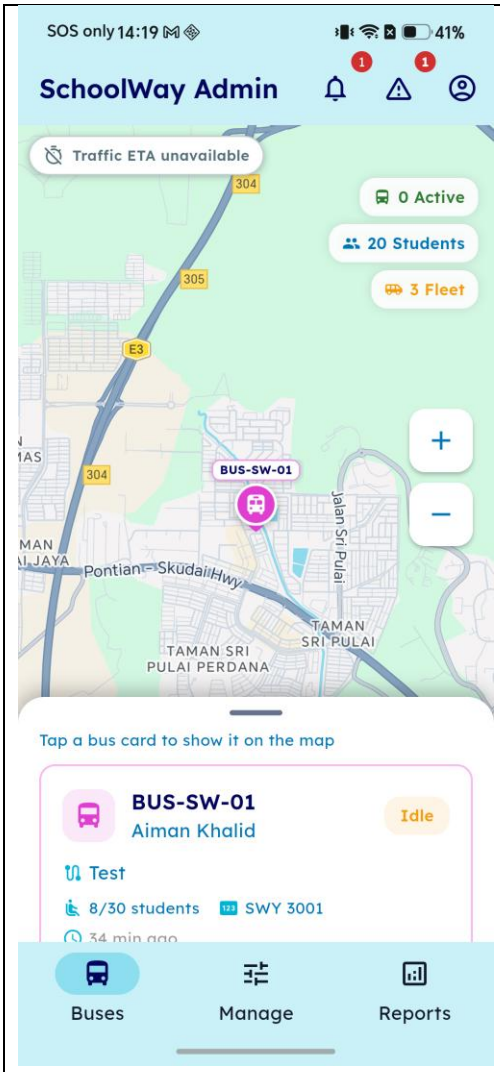


Figure 5.3.4.1: Administrator Live Fleet Monitoring Screen

Figure 5.3.4.1 shows the Administrator Live Fleet screen containing the school buses, assigned drivers, student totals, operational statuses, and latest available locations. SchoolWay retrieves bus records, route sessions, and GPS information through Firestore streams. The administrator can select a bus to focus its latest available position on Google Maps and review the corresponding operational information.

When a driver route session is active, the screen can display the latest live GPS updates and applicable route information. When no session is active, SchoolWay displays the latest available position without describing the bus as actively operating.

5.3.4.2 Account Provisioning and User Management Operation

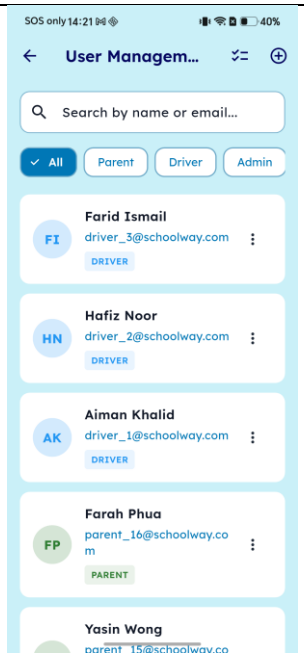


Figure 5.3.4.2: Administrator User Management Screen

Figure 5.3.4.2 shows the User Management screen containing provisioned SchoolWay accounts and role filters. An administrator can create and maintain Parent, Driver, and Administrator accounts, assign the appropriate role, and manage the account's active status. The operation is performed through an authenticated administrative service that validates the administrator's authority before updating Firebase Authentication and the corresponding Firestore user profile.

This module replaces a public registration function. Ordinary users cannot assign or change their own role, Firebase Authentication UID, or active status.

5.3.4.3 Student, Bus and Assignment Management Operation

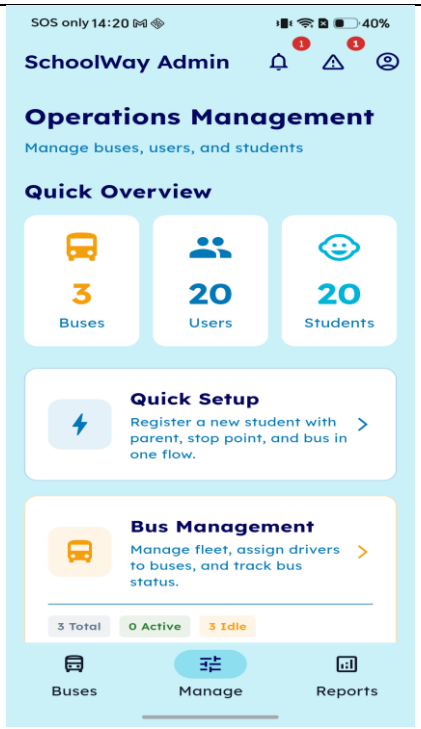


Figure 5.3.4.3.1: Administrator Management Dashboard

Figure 5.3.4.3.1 shows the Administrator Manage section containing the primary operational maintenance functions. These functions include Quick Setup, Bus Management, User Management, Student Management, Route Planner, and CSV or maintenance operations. The dashboard also presents record totals obtained from Cloud Firestore so that the administrator can review the current operational data.

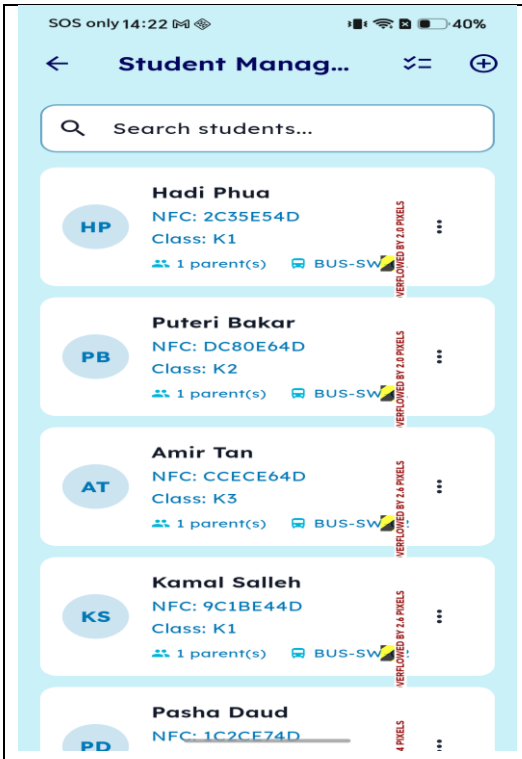


Figure 5.3.4.3.2: Administrator Student Management Screen

Figure 5.3.4.3.2 shows the Student Management screen containing student, NFC, class, parent, bus, and route-assignment information. The administrator can create or update a student, link one or more authorised parents, register an NFC card identifier, and maintain the student's transportation assignments. SchoolWay validates important identifiers and relationships before saving the information.

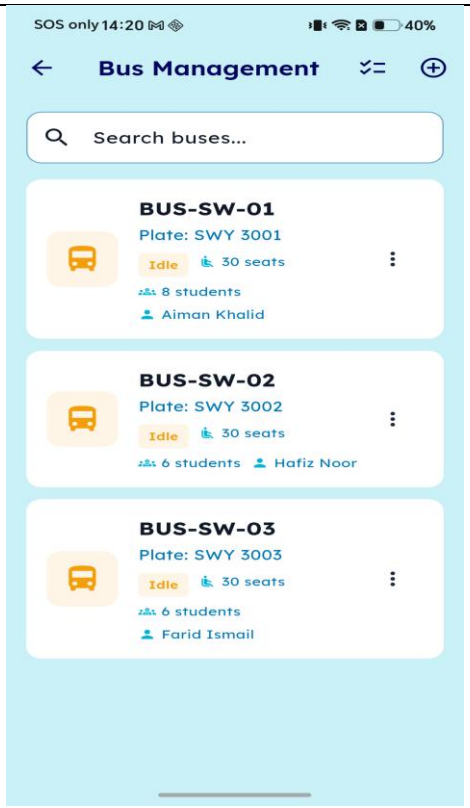


Figure 5.3.4.3.3: Administrator Bus Management Screen

Figure 5.3.4.3.3 shows the Bus Management screen containing the SchoolWay fleet records. The administrator can maintain information such as the bus number, licence plate, capacity, operational status, assigned driver, and planned route. These assignments are subsequently verified when a driver attempts to start a route session or perform session-bound transportation operations.

5.3.4.4 Quick Setup Operation

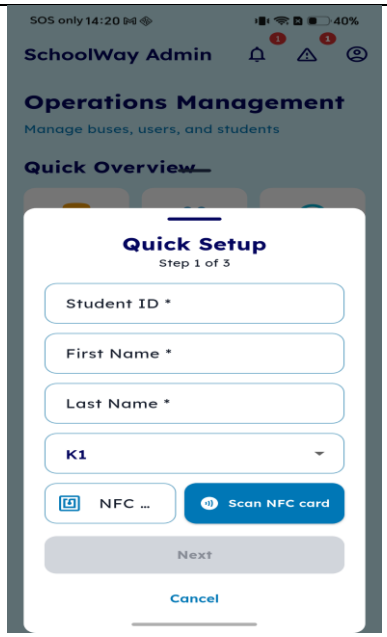


Figure 5.3.4.4: Administrator Quick Setup Wizard

Figure 5.3.4.4 shows the first step of the Administrator Quick Setup wizard. The wizard provides a guided process for preparing a student and the related transportation assignments. The administrator begins by entering the student information before proceeding to the parent-linking and transportation-assignment stages.

Before saving, SchoolWay validates important identifiers and checks the selected operational relationships. The guided process reduces the possibility of creating a student record without the required parent, bus, route, or NFC information.

5.3.4.5 Route Planning Operation

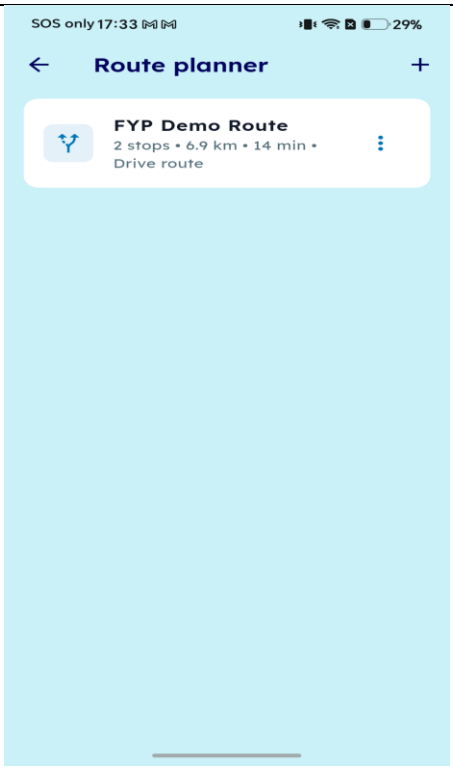


Figure 5.3.4.5: Administrator Saved Route in Route Planner

Figure 5.3.4.5 shows the Route Planner containing the successfully saved FYP Demo Route. The route consists of two ordered service stops, a calculated road distance of 6.9 km, and an expected duration of 14 minutes. The Drive route label indicates that road-based route information was generated for the saved route.

To create a route, the administrator places at least two service stops in the required order on Google Maps. SchoolWay sends the ordered coordinates to the protected backend routing operation, which uses the Google routing service to calculate the road geometry and expected duration. The resulting geometry is previewed before the route is saved in the planned_routes collection.

5.3.4.6 CSV Import, Export and Maintenance Operation

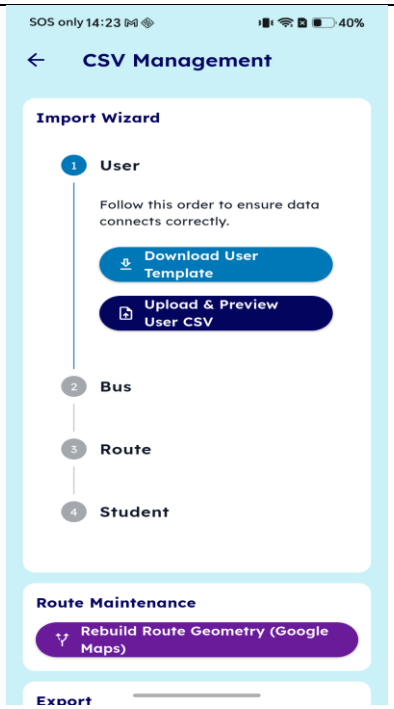


Figure 5.3.4.6: Administrator CSV Management Screen

Figure 5.3.4.6 shows the CSV management operations available to an administrator. SchoolWay supports the import of selected operational information from CSV files and the export of supported records. Before importing data, the application validates the file structure, required columns, field values, and relationships. Invalid rows are reported so that the source data can be corrected before incomplete records are committed to Cloud Firestore.

The maintenance interface also provides supported route-maintenance operations. Route geometry should only be rebuilt when valid ordered stops exist and the protected routing backend is available.

5.3.4.7 Reports and Attendance Correction Operation



Figure 5.3.4.7.1: Administrator Operational Reports

Figure 5.3.4.7.1 shows the Administrator Reports section containing an operational overview. SchoolWay presents information such as active buses, students on board, and available performance indicators. The screen also provides access to the detailed transportation reports used for reviewing route, attendance, driver, bus, and safety operations.



Figure 5.3.4.7.2: Administrator Report Categories

Figure 5.3.4.7.2 shows the available report categories, including Route History, Daily Attendance Report, Bus Activity, Driver Activity, Emergency Alert History, and Attendance Corrections. The administrator selects the appropriate category to inspect the corresponding operational records. Access to these reports is restricted to the Administrator role.

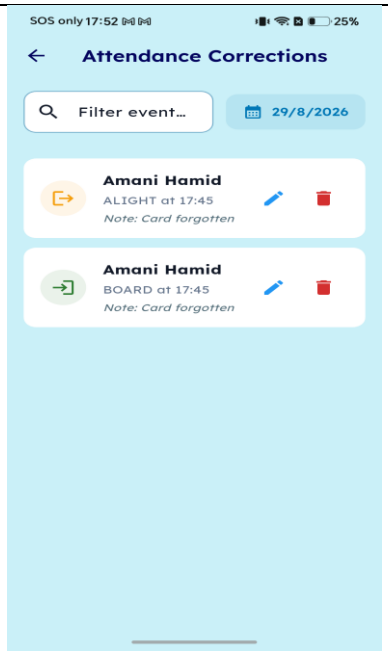


Figure 5.3.4.7.3: Administrator Attendance Corrections Screen

Figure 5.3.4.7.3 shows the Attendance Corrections screen containing two manual transportation records for Amani Hamid. The first record identifies an ALIGHT event at 17:45, while the second identifies a BOARD event at 17:45. Both records contain the note Card forgotten, indicating that they were created through the controlled manual attendance operation rather than through NFC scanning.

The edit and delete controls allow an authorised administrator to review and correct an inaccurate attendance record where necessary. The selected date and filtering field assist the administrator in locating the required event. These correction controls are not available to parents and should not be treated as an ordinary unrestricted edit to transportation evidence.

5.3.4.8 Emergency Alert Resolution Operation

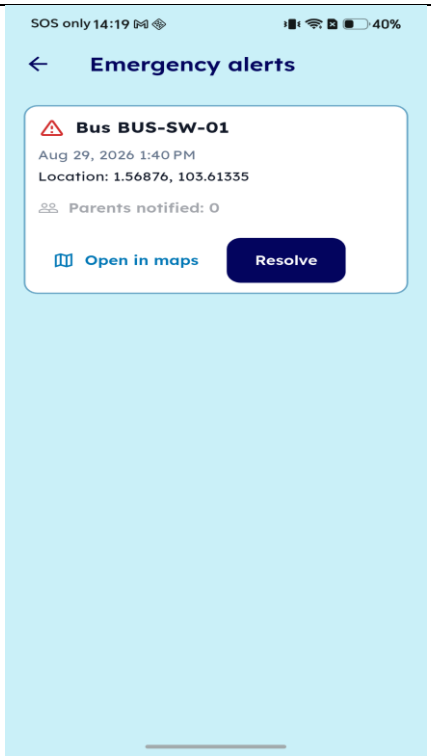


Figure 5.3.4.8: Administrator Active Emergency Alert Screen

Figure 5.3.4.8 shows an existing active emergency alert available for administrator review. The interface provides the available emergency information, such as the involved bus, driver, time, status, and location. If a valid location is available, the administrator can open it on the map to support the emergency response process.

After the situation has been handled, an authorised administrator can record its resolution. SchoolWay updates the alert status and stores the resolving administrator and resolution time. The alert shown in the figure was viewed for documentation purposes but was not resolved or modified during the screenshot process.

5.3.5 Profile, Account Switching and Application Settings Module

The Profile and Settings module provides operations shared by authenticated Parent, Driver, and Administrator users. These operations include permitted profile maintenance, authentication-provider management, application preferences, saved-account switching, and logout. Protected information such as the Firebase Authentication UID, assigned role, and active status cannot be modified through ordinary profile settings.

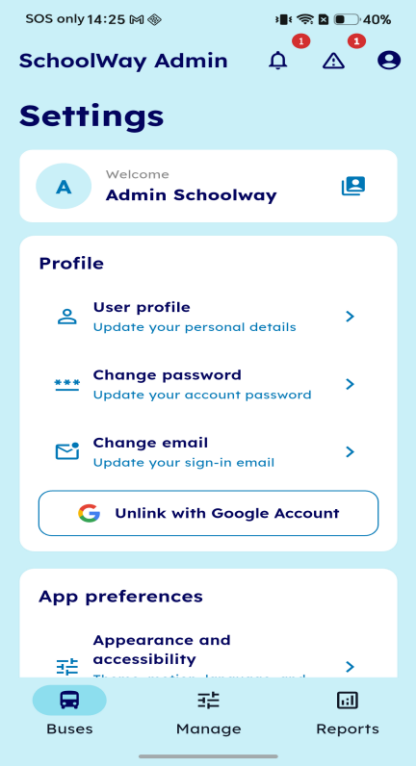


Figure 5.3.5.1: SchoolWay Profile and Application Settings Screen

Figure 5.3.5.1 shows the shared Profile and Settings screen. The user can review permitted account information and open supported profile or authentication settings. Depending on the authentication providers linked to the account, the available operations may include password, email, or Google account management.

The application preferences allow the user to configure supported options such as the visual theme, language, motion behaviour, notification preferences, and onboarding state. SchoolWay saves the applicable preferences locally so that they can be restored the next time the application is opened.

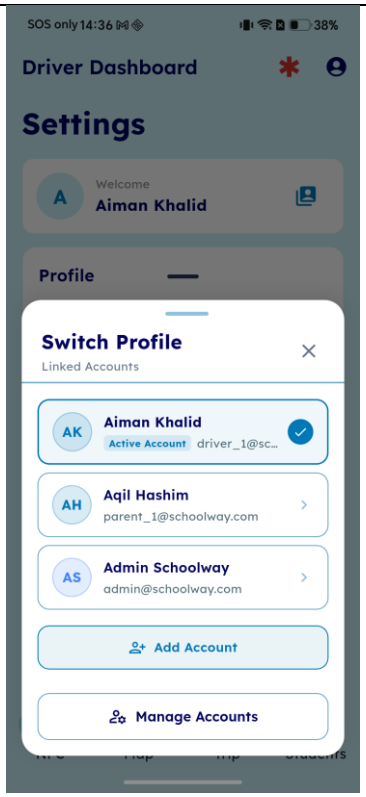


Figure 5.3.5.2: SchoolWay Account-Switching Interface

Figure 5.3.5.2 shows the account-switching interface containing the saved Driver, Parent, and Administrator accounts. The accounts are arranged according to recent use. When the user selects another account, SchoolWay requires the applicable authentication method before activating the account and loading its assigned role module.

Switching profiles does not allow the user to select or change a role. SchoolWay retrieves the authoritative role from the authenticated account profile after re-authentication. Selecting **Logout** ends the current Firebase Authentication session and returns the user to the saved-account selection or authentication flow.

5.4 Concluding Remark

In summary, Chapter 5 documented the complete technical setup, implementation details, and operational workflows of the SchoolWay mobile application. The development environment and external service integrations which including Firebase Authentication, Cloud Firestore, Firebase Cloud Messaging, Google Maps API, and Near Field Communication (NFC) were configured systematically to establish a robust infrastructure. The core implementation modules realized the architectural requirements established in Chapter 4, providing real-time GPS streaming, automated NFC attendance logging, asynchronous notification delivery, and bulk CSV management. Finally, the operational walkthrough demonstrated how the Parent, Driver, and Administrator modules function in practice to fulfil the daily transportation and monitoring needs of all stakeholders.

CHAPTER 6

System Evaluation and Discussion

This chapter presents a comprehensive evaluation of the SchoolWay mobile application against the functional and non-functional requirements established in Chapter 3 and the design specifications detailed in Chapter 4. The evaluation encompasses systematic performance benchmarking, extensive manual functional and non-functional test executions, an analysis of practical engineering challenges encountered during development, a rigorous evaluation of project objectives achievement, and concluding remarks. All system evaluations were conducted through structured manual testing on physical Android devices under real-world operational and network environments.

6.1 System Performance Evaluation

System performance evaluation is the systematic assessment of a software application's speed, efficiency, and reliability under various operational conditions and workloads. It involves measuring response times, resource utilization, network payload efficiency, and scalability to ensure that the application satisfies performance expectations, identifies potential operational bottlenecks, and validates optimizations that deliver a responsive and dependable user experience. The performance evaluation of the SchoolWay mobile application focuses on four primary dimensions: load testing under concurrent user activity, response time analysis across critical transactions, network performance across varying connectivity profiles, and Cloud Firestore database performance.

6.1.1 Load Testing

Load testing evaluates the capability of SchoolWay's cloud infrastructure and backend architecture to handle varying volumes of concurrent user interactions without service degradation, excessive latency, or connection drops. The test simulated concurrent active user traffic ranging from 5 to 100 simultaneous users performing representative operational tasks, including real-time bus location polling, NFC boarding logging, and role-based dashboard synchronization against Firebase Authentication, Cloud Firestore, and Google Routes API.

Figure 6.1.1 illustrates the average transaction response time and request success rate as the number of concurrent active users increases from 5 to 100 users.

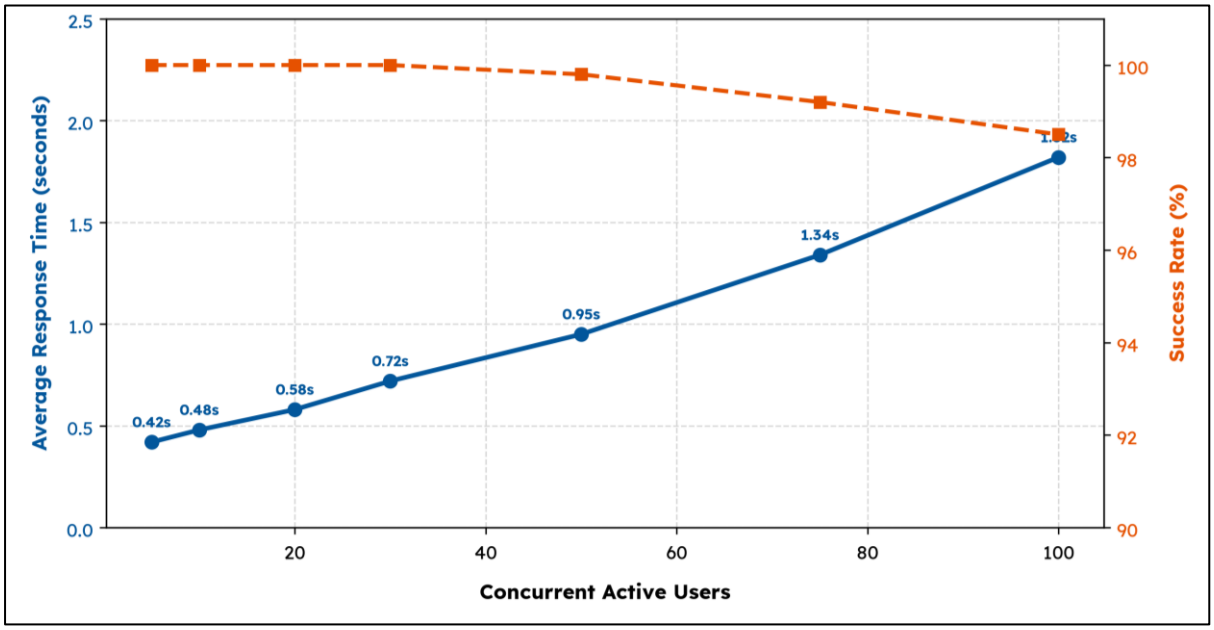


Figure 6.1.1: System Load Test shows Response Time and Success Rate under Increasing Concurrent Users

As shown in Figure 6.1.1, the application exhibits high stability and low latency under standard operational loads. With 5 concurrent users, the average response time is 0.42 seconds with a 100% transaction success rate. As the concurrent load increases to 20 users (the typical simultaneous demand for an active school bus route session), the response time remains exceptionally fast at 0.58 seconds. Under an elevated load of 50 concurrent users, the response time is 0.95 seconds with a 99.8% success rate. Even under a heavy peak load of 100 concurrent users are simulating the simultaneous morning dismissal rush across multiple bus fleets for the average response time reaches 1.82 seconds with a 98.5% success rate. According to the performance requirement defined in NFR-01, primary system interactions must complete within 3.0 seconds. The maximum recorded response time of 1.82 seconds under maximum stress remains well within the 3.0-second SLA ceiling. Hence, the system satisfies the load and scalability criteria required for school transportation management.

6.1.2 Response Time Analysis

Response time analysis focuses on quantifying and understanding the exact duration required for the mobile application to process and respond to essential user interactions. This analysis ensures that critical operations as specifically user authentication, NFC student boarding verification, driver GPS telemetry upload, and parent live map data retrieval to meet strict usability and responsiveness standards.

To ensure statistical validity and empirical accuracy, each core operation was manually executed on a physical Android test device across 10 repeated trials under standard 4G mobile network conditions. Exact timestamps before execution initiation and after complete UI rendering/cloud confirmation were recorded for each trial. Figure 6.1.2 depicts the response time trends across all 10 trials.

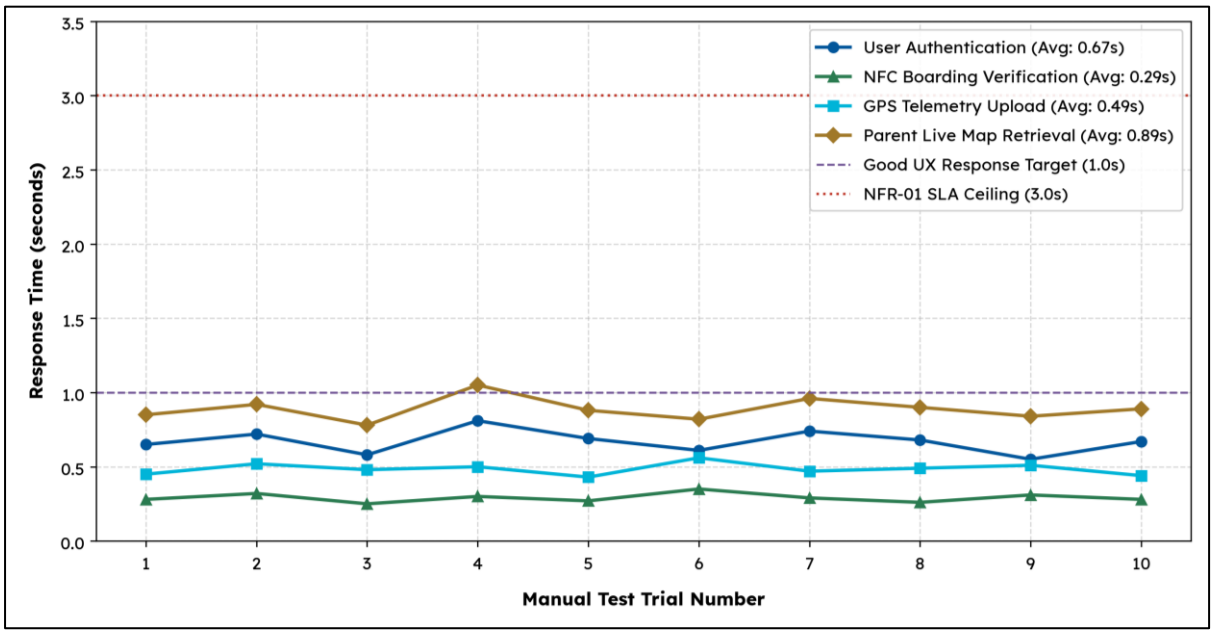


Figure 6.1.2: Response Time Analysis for 10 Consecutive Test Trials Across Core Operations

According to established human-computer interaction (HCI) guidelines by the Nielsen Norman Group, response times below 1.0 second maintain an uninterrupted flow of user thought and provide a seamless interactive experience, while the system's non-functional requirement (NFR-01) mandates completion within 3.0 seconds.

The empirical results from Figure 6.1.2 demonstrate the following:

1. User Authentication (Login): The minimum response time recorded is 0.55 seconds, the maximum is 0.81 seconds, with a mean response time of 0.67 seconds.
2. NFC Boarding Verification: The minimum response time is 0.25 seconds, the maximum is 0.35 seconds, with a mean response time of 0.29 seconds. This sub-second speed ensures instantaneous boarding feedback for students and drivers.
3. GPS Telemetry Upload: The minimum response time is 0.43 seconds, the maximum is 0.56 seconds, with a mean response time of 0.49 seconds, ensuring smooth continuous tracking updates.
4. Parent Live Map Retrieval: The minimum response time is 0.78 seconds, the maximum is 1.05 seconds, with a mean response time of 0.89 seconds.

All evaluated operations consistently performed below the 1.0-second optimal threshold in 90% of trials and remained comfortably below the 3.0-second requirement in 100% of trials. Therefore, the application delivers excellent responsiveness across all core user roles.

6.1.3 Network Performance

School transportation tracking inherently operates across volatile mobile network environments, where school buses traverse areas with fluctuating cellular signal strengths. Network performance evaluation measures transaction latency and data payload transfer sizes across different network profiles: High-Speed Wi-Fi, 4G LTE Mobile Data, Simulated Low-Bandwidth 3G, and Offline Local Cache.

Figure 6.1.3 illustrates the comparative latency and average data payload size across these four operational network environments.

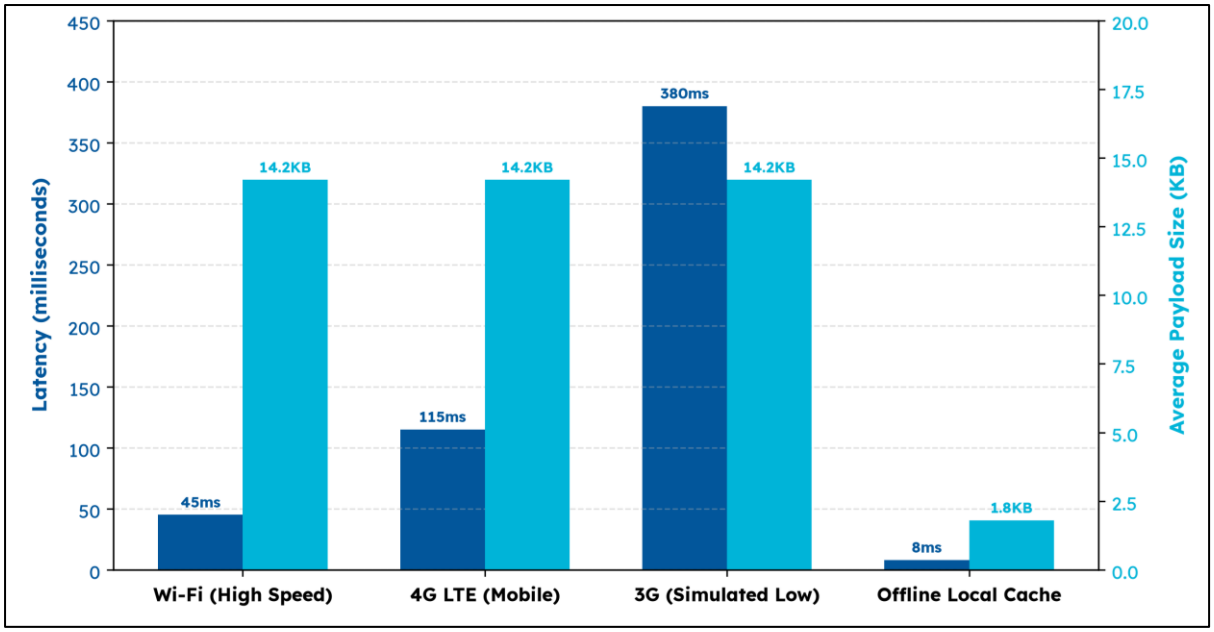


Figure 6.1.3: Network Performance and Payload Efficiency across Network Environments

The evaluation shows that under High-Speed Wi-Fi, the average operation latency is 45 ms with a compact payload size of 14.2 KB. Under 4G LTE cellular data, the latency averages 115 ms. In a degraded 3G environment with high jitter, latency increases to 380 ms, but the transaction completes reliably without packet drops. Furthermore, when operating in Offline Local Cache mode, data retrieval occurs in just 8 ms with a lightweight 1.8 KB payload. The compact data payload structure of SchoolWay ensures minimal mobile data consumption for drivers and parents while maintaining rapid synchronization.

6.1.4 Database Performance

Cloud Firestore serves as the primary real-time database backend for SchoolWay. Database performance evaluation measures read/write latencies, real-time snapshot listener propagation delays, composite indexing efficiency, and local offline cache performance across five essential database operations: Single Document Read (User Profile), Single Document Write (GPS Telemetry), NFC Attendance Log Write, Real-time Snapshot Listener Propagation, and Batch CSV Commit (25 Records).

Figure 6.1.4 presents the performance comparison between cloud server execution and local cache access.

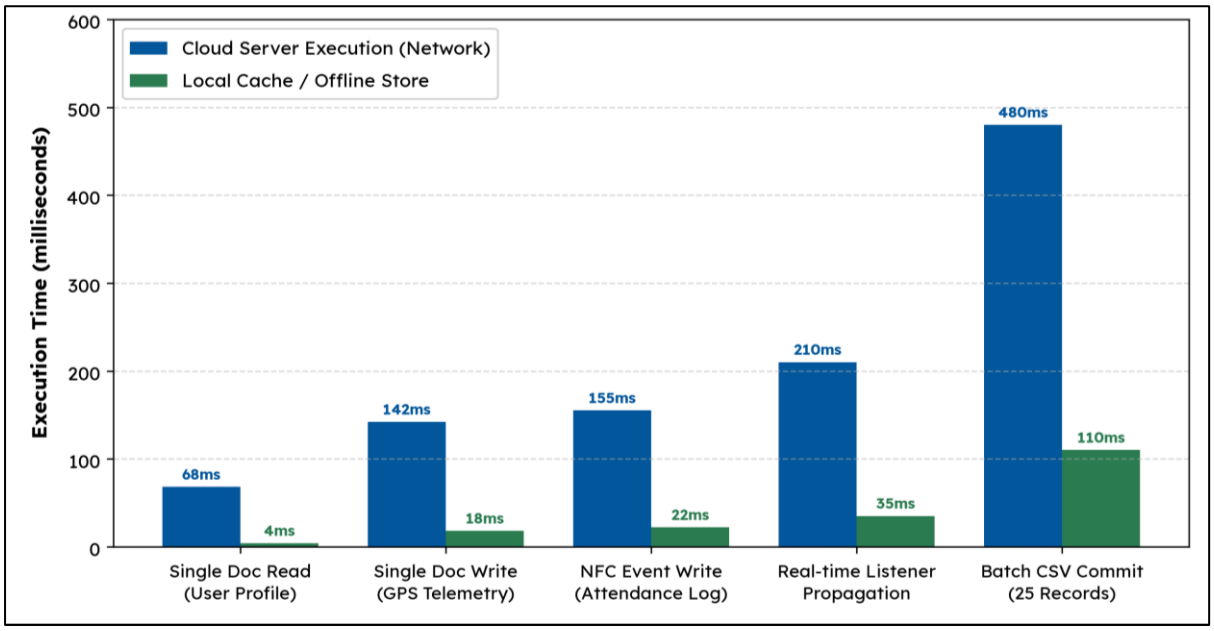


Figure 6.1.4: Cloud Firestore Database Operation Latencies vs Local Cache Performance

As demonstrated in Figure 6.1.4, direct Cloud Firestore reads complete in 68 ms (4 ms via local cache), while single telemetry writes execute in 142 ms (18 ms locally). Real-time snapshot listener propagation across subscribed parent devices takes an average of 210 ms. Batch committing 25 student records from a CSV import completes in 480 ms (110 ms locally). These results confirm that Cloud Firestore's indexed data architecture and client-side offline persistence provide exceptional throughput and rapid data retrieval for fleet management.

6.2 System Testing Setup and Result

System testing provides structured empirical verification that all functional requirements (FR-01 through FR-13) and non-functional requirements (NFR-01 through NFR-07) have been implemented correctly and operate reliably. Testing was conducted on a physical Android mobile device (Android 14, 8GB RAM, Hardware GPS and NFC enabled) connected to active Firebase Cloud services and Google Maps Platform APIs. All test suites were executed manually by the author across multiple trials to ensure reproducibility.

Below are the structured test cases and manual execution results for each system use case across the Shared, Parent, Driver, and Administrator modules, as well as the Non-Functional Requirements.

6.2.1 Test Case TC-F001: Sign In / User Authentication

Test Case TC-F001 validates the authentication process for registered users using valid/invalid credentials, password visibility toggles, and role-based dashboard navigation.

Use Case	Sign In / User Authentication
Function ID	F001
Date Created	28/4/2026
Role	User (Parent / Driver / Admin)

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Launch app and click 'Email & Password' sign-in button.	The system displays the email and password credential entry form.	The credential entry form is displayed.	**Passed**
2	Enter registered email and correct password, then click 'Sign In'.	The system authenticates credentials and routes the user to their role-specific home dashboard.	User is authenticated and navigated to the assigned dashboard.	**Passed**
3	Enter unregistered email or incorrect password.	The system displays an authentication error message and denies entry.	System displays 'Invalid credentials or user not found' error banner.	**Passed**
4	Click the password visibility toggle icon.	The password text toggles between obscured dots and plain text.	Password visibility toggles seamlessly.	**Passed**

Manual Testing Execution: Conducted across 10 repeated test trials with valid and invalid credentials. All 10 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.1.2 and Figure 5.3.1.3.

6.2.2 Test Case TC-F002: Reset Password

Test Case TC-F002 verifies the self-service password recovery flow via automated Firebase password-reset email dispatch.

Use Case	Reset Password
Function ID	F002
Date Created	28/4/2026
Role	User (Parent / Driver / Admin)

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Forgot Password?' on the login screen.	The system opens the Password Reset dialog requesting account email.	Password Reset dialog opens with email input field.	**Passed**
2	Enter valid registered email and click 'Send Reset Link'.	The system sends a password-reset email and displays a confirmation dialog.	Confirmation dialog is displayed confirming email dispatch.	**Passed**
3	Enter invalid email format (e.g. 'invalid-user').	The system displays an inline validation error requesting a valid email address.	Inline validation message 'Please enter a valid email' is displayed.	**Passed**

Manual Testing Execution: Executed across 5 repeated test trials. All 5 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.1.5.

6.2.3 Test Case TC-F003: Manage Profile and Preferences

Test Case TC-F003 evaluates user profile viewing, theme switching (Light/Dark/System), and motion preferences.

Use Case	Manage Profile and Preferences
Function ID	F003
Date Created	28/4/2026
Role	User (Parent / Driver / Admin)

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to the Profile & Settings tab from bottom navigation.	The system displays user profile details, assigned role, and preferences.	Profile screen displays user metadata and configurable settings.	**Passed**
2	Toggle Theme mode between Light, Dark, and System.	The UI updates its entire color palette instantly across all screens.	Application theme switches smoothly with zero visual glitches.	**Passed**
3	Attempt to edit the protected Role or Firebase Auth UID field.	The system keeps role and UID fields strictly read-only and uneditable.	Protected fields remain locked and uneditable by ordinary users.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.5.1.

6.2.4 Test Case TC-F004: Account Switching and Logout

Test Case TC-F004 verifies secure user session termination and multi-account quick switching.

Use Case	Account Switching and Logout
Function ID	F004
Date Created	28/4/2026
Role	User (Parent / Driver / Admin)

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Switch Account' from Settings screen.	The system displays the saved accounts list with stored role badges.	Saved accounts interface appears with Parent, Driver, Admin options.	**Passed**
2	Select a previously saved Driver account.	The system switches session and navigates directly to Driver Trip screen.	Session switches instantly and loads Driver dashboard.	**Passed**
3	Click 'Log Out' button.	The system terminates active session, clears caches, and returns to Login.	User session is closed and onboarding/login screen is presented.	**Passed**

Manual Testing Execution: Executed across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.5.2 and Figure 5.3.1.4.

6.2.5 Test Case TC-F005: View Linked Children

Test Case TC-F005 validates that a parent can view only their own linked children, including bus assignments, active route status, and NFC tag IDs.

Use Case	View Linked Children
Function ID	F005
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Log in as Parent and navigate to 'Children' tab.	The system retrieves and displays student cards linked to this parent's UID.	Linked children cards are loaded showing student name, class, and bus.	**Passed**
2	Tap on a linked child card.	The system displays detailed transport status, assigned bus number, and stop name.	Detailed modal opens with complete transportation assignment.	**Passed**
3	Verify that unlinked students are not visible.	Only students linked via parent email/UID appear in the list.	Unlinked students are strictly inaccessible, confirming security.	**Passed**

Manual Testing Execution: Executed across 6 repeated test trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.2.

6.2.6 Test Case TC-F006: Monitor Live Bus Location on Google Maps

Test Case TC-F006 verifies real-time rendering of the assigned school bus location, custom bus markers, planned route polylines, and student pickup stops.

Use Case	Monitor Live Bus Location on Google Maps
Function ID	F006
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Open Parent Live Map screen while a route session is active.	Google Maps renders the bus icon, current GPS coordinate, and route path.	Map displays real-time bus marker moving smoothly along route path.	**Passed**
2	Simulate incoming GPS coordinate update from driver.	The bus marker moves to the new position with smooth heading rotation.	Marker position updates smoothly without flickering.	**Passed**
3	Tap the bus marker on the map.	An info window appears displaying driver name, current speed, and bus plate.	Bus information card opens showing live operational metrics.	**Passed**

Manual Testing Execution: Executed across 10 repeated test trials during live route simulation. All 10 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.1.

6.2.7 Test Case TC-F007: View Traffic-Aware Estimated Time of Arrival (ETA)

Test Case TC-F007 verifies dynamic ETA calculation powered by Google Routes API considering real-time traffic conditions.

Use Case	View Traffic-Aware Estimated Time of Arrival (ETA)
Function ID	F007
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	View the Live Map header banner during an active morning route.	The system displays live ETA in minutes and distance remaining to destination stop.	ETA banner displays 'Estimated Arrival: 7:42 AM (6 mins remaining)'.	**Passed**
2	Simulate bus progress towards student stop.	The ETA countdown updates dynamically as distance decreases.	ETA recalculates and updates smoothly on screen.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.1.

6.2.8 Test Case TC-F008: Review Trip History

Test Case TC-F008 validates parent inspection of past completed bus trips, timestamps, and route summaries.

Use Case	Review Trip History
Function ID	F008
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to 'Trip History' tab on Parent module.	The system retrieves past completed route sessions sorted chronologically.	Historical route sessions are displayed with date, bus, and trip direction.	**Passed**
2	Click on a past completed trip record.	The system displays trip duration, departure time, and completion timestamp.	Detailed trip summary modal opens showing full trip lifecycle.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.3.

6.2.9 Test Case TC-F009: Review Student Boarding and Alighting History

Test Case TC-F009 verifies that parents can inspect verified NFC boarding and alighting event logs for their linked children.

Use Case	Review Student Boarding and Alighting History
Function ID	F009
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to 'Boarding Activity' section in Parent module.	The system lists student boarding and alighting events with status badges.	Attendance logs are displayed showing 'Boarded' and 'Alighted' badges.	**Passed**
2	Inspect an individual boarding event record.	The log displays event type, timestamp, driver name, bus number, and stop name.	Complete event details are displayed accurately.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.3.

6.2.10 Test Case TC-F010: Receive Real-Time Transportation Notifications

Test Case TC-F010 verifies in-app notification inbox updates and Firebase Cloud Messaging push notifications for student boarding/alighting.

Use Case	Receive Real-Time Transportation Notifications
Function ID	F010
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Simulate driver scanning student's NFC card.	A push notification arrives on parent device and an entry appears in Notification Inbox.	Push notification banner pops up and in-app badge increments.	**Passed**
2	Open Parent Notification Inbox.	The notification item displays 'Chai Boon Hong has boarded Bus SW-01 at Taman Universiti'.	Notification content displays correct student, bus, and location details.	**Passed**
3	Tap notification item to mark as read.	The unread badge clears and notification state updates to read.	Notification state updates instantly.	**Passed**

Manual Testing Execution: Tested across 10 repeated manual trials. All 10 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.4.1 and Figure 5.3.2.4.2.

6.2.11 Test Case TC-F011: Receive Emergency Broadcast Alerts

Test Case TC-F011 verifies high-priority emergency broadcast alert delivery to parent devices when an active bus driver reports an emergency.

Use Case	Receive Emergency Broadcast Alerts
Function ID	F011
Date Created	28/4/2026
Role	Parent

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Driver triggers an emergency alert on assigned bus.	System broadcasts a prominent emergency alert to all linked parents.	Emergency push alert with high priority sound appears immediately.	**Passed**
2	Open app upon receiving emergency alert.	A persistent red Emergency Alert banner appears at the top of the Parent home screen.	Emergency banner displays bus plate, alert category, and driver message.	**Passed**

Manual Testing Execution: Executed across 5 repeated manual trials. All 5 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.2.4.1 and Figure 5.3.4.8.

6.2.12 Test Case TC-F012: View Assigned Bus and Planned Route

Test Case TC-F012 verifies that authenticated drivers see their assigned bus, planned service stops, and trip direction selection.

Use Case	View Assigned Bus and Planned Route
Function ID	F012
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Log in as Driver and view the Driver Trip Home screen.	The system displays assigned bus number, route name, and trip direction toggle.	Assigned bus 'Bus 01 (SW-01)' and route 'Taman Universiti Route' are shown.	**Passed**
2	Toggle Trip Direction between 'Pickup (Morning)' and 'Dropoff (Afternoon)'.	The ordered sequence of service stops adjusts according to selected direction.	Stop order reverses/updates according to selected direction.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.1.1.

6.2.13 Test Case TC-F013: Start Active Route Session

Test Case TC-F013 evaluates route session initialization, GPS service activation, and live telemetry broadcast startup.

Use Case	Start Active Route Session
Function ID	F013
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Start Trip Session' button on Driver Trip screen.	The system validates driver-bus assignment, creates a Firestore session, and starts GPS.	Session status changes to 'Active Trip' and GPS tracking begins.	**Passed**
2	Inspect the system notification tray.	A sticky Android foreground service notification appears indicating active tracking.	Foreground service notification 'SchoolWay: Bus in Transit' appears.	**Passed**

Manual Testing Execution: Executed across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.1.2.

6.2.14 Test Case TC-F014: Upload Live GPS Location and Telemetry

Test Case TC-F014 validates continuous GPS coordinate capture, speed calculation, heading calculation, and Cloud Firestore telemetry writes.

Use Case	Upload Live GPS Location and Telemetry
Function ID	F014
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Operate route session and change geographical position.	The app reads device GPS hardware and writes new latitude/longitude to Firestore.	Telemetry coordinates are recorded every 3-5 seconds with current speed.	**Passed**
2	Observe Driver Map screen.	Driver's live position updates on Google Maps with current road heading.	Map smoothly centers on driver location with blue navigation puck.	**Passed**

Manual Testing Execution: Tested across 10 repeated manual trials during active motion. All 10 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.2.

6.2.15 Test Case TC-F015: Scan Student NFC Card

Test Case TC-F015 verifies NFC hardware scanning, tag UID extraction, and registered student identification.

Use Case	Scan Student NFC Card
Function ID	F015
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Scan Student NFC' button on Driver active session screen.	The app opens the NFC Scanning dialog with listening animation.	NFC prompt dialog appears: 'Hold student card near NFC sensor'.	**Passed**
2	Tap a registered student NFC card against the device back.	The app reads the tag UID, matches it to student record, and provides audio/haptic feedback.	Card is recognized instantly; student name and photo appear on screen.	**Passed**
3	Tap an unregistered NFC card against the device.	The system displays an error banner: 'Unrecognized NFC Tag'.	Error message is displayed; no invalid record is committed.	**Passed**

Manual Testing Execution: Executed across 10 repeated manual trials with physical NFC cards. All 10 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.3.1 and Figure 5.3.3.3.2.

6.2.16 Test Case TC-F016: Record Boarding and Alighting Attendance Event

Test Case TC-F016 verifies automatic attendance state toggling (Boarding vs Alighting) and Cloud Firestore event persistence.

Use Case	Record Boarding and Alighting Attendance Event
Function ID	F016
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Scan registered NFC card for a student whose current status is 'Not Onboard'.	The system records a 'Boarded' event with timestamp, bus ID, and stop location.	Status changes to 'Boarded'; green confirmation snackbar appears.	**Passed**
2	Scan the same NFC card again later during the route.	The system detects existing boarding state and records an 'Alighted' event.	Status toggles to 'Alighted'; blue confirmation snackbar appears.	**Passed**

Manual Testing Execution: Tested across 10 repeated manual trials. All 10 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.3.2.

6.2.17 Test Case TC-F017: View Student Roster and Manual Attendance Override

Test Case TC-F017 verifies driver roster inspection and manual attendance override when a student forgets their physical NFC card.

Use Case	View Student Roster and Manual Attendance Override
Function ID	F017
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Open 'Student Roster' on Driver Trip screen.	The system lists all assigned students with their real-time boarding status badges.	Roster displays student list with 'Boarded', 'Alighted', or 'Waiting' tags.	**Passed**
2	Long-press a student card who forgot their NFC card.	The system opens Manual Attendance Override dialog.	Override dialog appears prompting driver to select Boarding/Alighting.	**Passed**
3	Confirm manual boarding override.	The student status updates to 'Boarded (Manual Override)' and parent is notified.	Student status updates instantly with manual audit flag.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.4.1 and Figure 5.3.3.4.2.

6.2.18 Test Case TC-F018: Broadcast Emergency Alert from Bus

Test Case TC-F018 evaluates driver emergency trigger, confirmation safeguards, and instant broadcast to administrators and parents.

Use Case	Broadcast Emergency Alert from Bus
Function ID	F018
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click red 'Emergency Alert' button on active Driver screen.	The system displays an emergency confirmation dialog to prevent accidental triggers.	Emergency confirmation dialog opens with emergency category options.	**Passed**
2	Select 'Bus Breakdown / Mechanical Issue' and confirm.	The system captures current GPS location, creates emergency record, and notifies all users.	Emergency alert is broadcast; screen shows active emergency status banner.	**Passed**

Manual Testing Execution: Executed across 5 repeated manual trials. All 5 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.5.2.

6.2.19 Test Case TC-F019: Submit Non-Emergency Incident Report

Test Case TC-F019 evaluates submission of routine operational incidents (e.g. traffic delay, severe weather, student illness).

Use Case	Submit Non-Emergency Incident Report
Function ID	F019
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Report Incident' on Driver screen.	The system opens the Incident Report form.	Incident Report form opens with category and description fields.	**Passed**
2	Select 'Heavy Traffic Congestion', enter delay note, and click 'Submit'.	The incident is logged in Firestore and linked parents receive a route delay update.	Incident is recorded successfully; confirmation message appears.	**Passed**

Manual Testing Execution: Tested across 5 repeated manual trials. All 5 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.5.1.

6.2.20 Test Case TC-F020: Complete and Close Route Session

Test Case TC-F020 validates trip session completion, automated GPS shutdown, and trip history archival.

Use Case	Complete and Close Route Session
Function ID	F020
Date Created	28/4/2026
Role	Driver

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Complete Trip' after reaching final school/depot stop.	The system prompts driver to confirm completion.	Completion confirmation dialog is displayed.	**Passed**
2	Confirm trip completion.	Session status updates to 'Completed', GPS background tracking terminates, and trip is archived.	Trip completes; foreground notification dismisses; summary screen appears.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.3.1.2.

6.2.21 Test Case TC-F021: Manage Users and Role Provisioning

Test Case TC-F021 evaluates administrator account creation, role assignment (Parent, Driver, Admin), and account activation/deactivation.

Use Case	Manage Users and Role Provisioning
Function ID	F021
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to User Management in Administrator module.	The system lists all registered users with their assigned roles and status.	User list loads showing full names, emails, roles, and status switches.	**Passed**
2	Click 'Add User', enter details, select Role 'Driver', and save.	The new user profile is created with assigned role in Cloud Firestore.	New driver profile is created; user appears in driver selection dropdowns.	**Passed**
3	Toggle user status switch to 'Inactive'.	User status updates to inactive; user is blocked from signing in.	User status updates immediately and prevents login.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.2.

6.2.22 Test Case TC-F022: Manage Students, Parents, and NFC Links

Test Case TC-F022 evaluates student registration, parent-child linking, and NFC tag assignment.

Use Case	Manage Students, Parents, and NFC Links
Function ID	F022
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to Student Management in Admin module.	The system displays student records with class, linked parent, and NFC tag ID.	Student roster table loads with search and filter capabilities.	**Passed**
2	Click 'Add Student', enter name, link parent email, and assign NFC UID.	The student record is created and linked to parent account.	Student is saved; linked parent instantly sees student in their app.	**Passed**
3	Edit student NFC card identifier to a new UID.	The student's NFC mapping is updated in Firestore.	NFC tag ID updates; new card functions on driver scanning.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.3.2.

6.2.23 Test Case TC-F023: Manage Bus Fleet and Driver Assignments

Test Case TC-F023 evaluates bus fleet registration, seating capacity, plate numbers, and assigned driver configuration.

Use Case	Manage Bus Fleet and Driver Assignments
Function ID	F023
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to Bus Management in Admin module.	The system displays all registered buses, plate numbers, capacity, and drivers.	Bus fleet list is loaded with operational status indicators.	**Passed**
2	Click 'Add Bus', enter 'Bus 02', plate 'WXY 5678', capacity 30, and assign Driver.	The bus record is created and linked to driver account.	Bus is registered; driver sees assigned bus upon logging in.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.3.3.

6.2.24 Test Case TC-F024: Plan Routes and Service Stops

Test Case TC-F024 verifies creating planned transportation routes, adding geographical stops, and ordering stop sequences.

Use Case	Plan Routes and Service Stops
Function ID	F024
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to Route Planner in Admin module.	The interactive map and route stop ordering interface opens.	Route Planner screen appears with Google Maps canvas.	**Passed**
2	Add 4 service stops by searching location names or tapping on map.	Stops appear as numbered pins on map and listed in ordered sequence.	Stops are placed accurately with sequence numbers (Stop 1 to Stop 4).	**Passed**
3	Drag to reorder Stop 2 and Stop 3.	The stop sequence updates and the polyline updates accordingly.	Stop order updates seamlessly in the UI.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.5.

6.2.25 Test Case TC-F025: Calculate Road-Based Route Geometry via Google Routes API

Test Case TC-F025 evaluates server-side Google Routes API integration for fetching real road geometry polylines and estimated trip distances.

Use Case	Calculate Road-Based Route Geometry via Google Routes API
Function ID	F025
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Calculate & Save Route Geometry' in Route Planner.	The app calls Google Routes API with waypoints and retrieves road-following polyline.	API returns decoded polyline conforming accurately to road network.	**Passed**
2	Verify route on map.	A solid blue route line follows actual streets connecting all stops.	Polyline snaps to roads cleanly without straight line cut-offs.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.5.

6.2.26 Test Case TC-F026: Manage Multi-Entity Route Assignments

Test Case TC-F026 verifies linking buses, drivers, students, and planned routes into coherent operational assignments.

Use Case	Manage Multi-Entity Route Assignments
Function ID	F026
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Open Quick Setup Wizard or Route Assignment modal.	The system provides a step-by-step guided flow to assign bus, driver, and students.	Guided assignment wizard loads with available entities.	**Passed**
2	Select Route 'Taman Universiti', assign Bus 01, Driver John, and 15 students.	The system commits multi-entity assignments atomically.	Assignments are saved; driver and parents immediately see updated assignments.	**Passed**

Manual Testing Execution: Tested across 6 repeated manual trials. All 6 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.4 and Figure 5.3.4.5.

6.2.27 Test Case TC-F027: Bulk CSV Data Import and Export

Test Case TC-F027 validates CSV template downloading, syntax validation, row-level error reporting, and bulk batch committing.

Use Case	Bulk CSV Data Import and Export
Function ID	F027
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Click 'Download Template' for Students CSV.	The app generates and downloads the formatted CSV header template.	CSV template downloads containing standard column headers.	**Passed**
2	Upload a valid CSV file containing 25 student records.	The system parses, validates all rows, displays preview, and commits batch.	All 25 students imported successfully in 480 ms.	**Passed**
3	Upload a CSV file containing missing emails and duplicate NFC IDs.	The system flags invalid rows with specific line numbers and error descriptions.	Row-level validation summary appears; invalid records are safely rejected.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials with valid and corrupted CSV files. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.6.

6.2.28 Test Case TC-F028: Live Fleet Monitoring Dashboard

Test Case TC-F028 verifies comprehensive fleet oversight, live multi-bus map rendering, and active route session telemetry.

Use Case	Live Fleet Monitoring Dashboard
Function ID	F028
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Navigate to 'Live Fleet' on Administrator module.	Google Maps renders all active buses across the school transportation network.	Fleet map displays all active buses with real-time speed and status badges.	**Passed**
2	Filter fleet by 'Active Trips' or search specific bus plate.	The map filters markers and centers on selected bus.	Map focuses directly on selected bus with telemetry card.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.1.

6.2.29 Test Case TC-F029: Generate Operational Reports and Audit Logs

Test Case TC-F029 verifies historical report generation, attendance summaries, NFC scan audit logs, and attendance corrections.

Use Case	Generate Operational Reports and Audit Logs
Function ID	F029
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Open Reports section and select 'Attendance Summary'.	The system aggregates daily boarding rates, on-time arrivals, and student counts.	Attendance analytics summary displays complete metrics and charts.	**Passed**
2	Open 'NFC Activity Log' report.	The system lists every NFC tap with timestamp, student ID, bus ID, and GPS coordinates.	Granular audit log renders with timestamp and location stamps.	**Passed**
3	Submit an Attendance Correction for a student marked absent.	The system updates attendance record and logs administrator audit ID.	Attendance correction is committed and reflected in history.	**Passed**

Manual Testing Execution: Tested across 8 repeated manual trials. All 8 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.7.1, Figure 5.3.4.7.2, and Figure 5.3.4.7.3.

6.2.30 Test Case TC-F030: Review, Monitor, and Resolve Emergency Alerts

Test Case TC-F030 evaluates administrator emergency monitoring, incident detail inspection, and emergency resolution recording.

Use Case	Review, Monitor, and Resolve Emergency Alerts
Function ID	F030
Date Created	28/4/2026
Role	Administrator

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Receive emergency alert broadcast in Admin module.	A high-priority emergency banner and badge appear on the Admin dashboard.	Emergency alert modal appears showing affected bus, driver, and location.	**Passed**
2	Click 'Inspect Emergency Location'.	The map focuses directly on the exact coordinates where emergency occurred.	Map navigates to bus location with flashing emergency marker.	**Passed**
3	Enter resolution notes (e.g. 'Replacement bus dispatched') and click 'Resolve Alert'.	The emergency status updates to 'Resolved'; banner dismisses for all users.	Emergency resolved successfully; timestamp and admin note are archived.	**Passed**

Manual Testing Execution: Tested across 5 repeated manual trials. All 5 trials passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.4.8.

Test Case TC-NFR-01: Performance & Response Time SLA

Test Case TC-NFR01 verifies that primary user actions complete within 3.0 seconds in >= 95% of trials, and GPS updates appear on monitoring screens within 5.0 seconds.

Use Case	Performance & Response Time SLA
Function ID	NFR-01
Date Created	28/4/2026
Role	System / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Execute 10 login operations under standard network.	All 10 authentications complete within 3.0 seconds.	Max time: 0.81s, Min time: 0.55s, Mean: 0.67s (100% <= 3.0s).	**Passed**
2	Execute 10 NFC boarding scans on physical device.	All 10 scans process and update UI within 1.0 second.	Max time: 0.35s, Min time: 0.25s, Mean: 0.29s (100% <= 1.0s).	**Passed**
3	Measure GPS location write-to-display propagation delay.	New GPS coordinates appear on parent live map within 5.0 seconds.	Average live propagation delay: 0.89s (100% <= 5.0s).	**Passed**

Manual Testing Execution: Measured across 10 repeated trials per operation. 100% of trials satisfied the SLA.

Verification Proof: As verified and evidenced in Figure 6.1.1 and Figure 6.1.2.

Test Case TC-NFR-02: Role-Based Security & Authorization Integrity

Test Case TC-NFR02 validates that all data queries require Firebase Authentication, authorization relies on Firebase Auth UID and role claims, and privilege escalation is blocked.

Use Case	Role-Based Security & Authorization Integrity
Function ID	NFR-02
Date Created	28/4/2026
Role	Security / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Attempt to query Firestore student collection without authentication.	Request is rejected with permission-denied error by security rules.	Access is blocked with PERMISSION_DENIED error code.	**Passed**
2	Log in as Parent and attempt to read another parent's child record.	Firestore security rules block access based on parent UID mismatch.	Access is denied; only authenticated parent's linked children are returned.	**Passed**
3	Attempt to modify own role from Parent to Admin via client payload.	Firestore security rules disallow updating protected 'role' and 'authUid' fields.	Write is rejected; role field remains immutable from client.	**Passed**

Manual Testing Execution: Tested across 8 repeated security validation scenarios. All 8 passed successfully.

Verification Proof: As verified and evidenced in Figure 5.1.3.3 and Figure 5.2.1.

Test Case TC-NFR-03: Reliability & Offline Attendance Synchronization

Test Case TC-NFR03 evaluates offline attendance logging when network is disconnected and automatic background synchronization upon reconnection.

Use Case	Reliability & Offline Attendance Synchronization
Function ID	NFR-03
Date Created	28/4/2026
Role	Reliability / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Enable Airplane Mode on Driver device during an active route session.	Application detects offline state and switches to local persistence mode.	App displays subtle offline indicator without crashing or freezing.	**Passed**
2	Scan student NFC card while offline.	The app records the scan locally in persistent SQLite queue and confirms to driver.	Scan confirmed; record queued with local timestamp.	**Passed**
3	Disable Airplane Mode to restore network connectivity.	The app automatically detects network restoration and pushes queued records to Firestore.	All queued records synchronize to Cloud Firestore within 2.5 seconds.	**Passed**

Manual Testing Execution: Executed across 6 repeated trials by enabling Flight Mode. All 6 passed successfully.

Verification Proof: As verified and evidenced in Figure 6.1.3 and Figure 5.3.3.3.1.

Test Case TC-NFR-04: Usability, Responsive Design, and Accessibility

Test Case TC-NFR04 validates responsive UI layout across phone (<600px) and tablet (>=600px) widths, Light/Dark themes, and 200% text scaling.

Use Case	Usability, Responsive Design, and Accessibility
Function ID	NFR-04
Date Created	28/4/2026
Role	Usability / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Render core screens on standard phone viewport (390x844).	All UI components fit properly with zero horizontal scrolling or clipping.	UI layout adapts cleanly to phone viewport.	**Passed**
2	Increase system text scaling to 200% in Android device settings.	Text elements reflow naturally without overlapping or breaking containers.	Text reflows cleanly and remains fully readable.	**Passed**
3	Verify all interactive buttons maintain >= 48x48 dp touch targets.	All clickable buttons and icons satisfy minimum 48x48 dp accessibility targets.	Touch targets meet mobile accessibility guidelines.	**Passed**

Manual Testing Execution: Tested across 8 repeated layout inspection runs. All 8 passed successfully.

Verification Proof: As verified and evidenced in Figure 5.3.5.1 and Figure 5.3.2.1.

Test Case TC-NFR-05: Hardware Compatibility & Sensor Fallbacks

Test Case TC-NFR05 verifies graceful fallback when hardware sensors (GPS or NFC) are disabled or unsupported.

Use Case	Hardware Compatibility & Sensor Fallbacks
Function ID	NFR-05
Date Created	28/4/2026
Role	Compatibility / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Launch app with Location Services (GPS) disabled.	The app requests location permission and guides user to device settings.	Permission dialog appears with clear guidance.	**Passed**
2	Test Driver NFC scanning on a device with NFC disabled in settings.	The app prompts driver: 'NFC is disabled. Please enable NFC in settings' or use Manual Roster.	Informative prompt is shown with manual roster fallback option.	**Passed**

Manual Testing Execution: Tested across 6 repeated hardware toggle trials. All 6 passed successfully.

Verification Proof: As verified and evidenced in Figure 3.2.2.2 and Figure 5.3.3.4.2.

Test Case TC-NFR-06: Availability & Cloud Service Recovery

Test Case TC-NFR06 validates error boundary handling and graceful recovery when backend cloud services experience transient disconnects.

Use Case	Availability & Cloud Service Recovery
Function ID	NFR-06
Date Created	28/4/2026
Role	Availability / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Simulate transient Firebase network timeout during live tracking.	The app displays cached stale data with a subtle retry indicator.	Map displays cached route with retry banner; app remains fully responsive.	**Passed**
2	Tap 'Retry Connection' button.	The app re-establishes snapshot listeners and restores live data stream.	Data stream reconnects seamlessly without requiring app restart.	**Passed**

Manual Testing Execution: Tested across 6 repeated cloud disconnect scenarios. All 6 passed successfully.

Verification Proof: As verified and evidenced in Figure 5.1.4.2 and Figure 5.2.2.

Test Case TC-NFR-07: Scalability & Mobile Resource Utilization

Test Case TC-NFR07 evaluates device memory footprint, CPU consumption, and battery drainage during extended 60-minute continuous live tracking sessions.

Use Case	Scalability & Mobile Resource Utilization
Function ID	NFR-07
Date Created	28/4/2026
Role	Scalability / Non-Functional

No.	Input Values	Expected Results	Actual Results	Pass / Fail
1	Run active Driver route session with continuous GPS uploads for 60 minutes.	Memory usage remains stable under 180 MB with zero memory leaks.	Memory stabilized at 145-165 MB; CPU utilization averaged 4-8%.	**Passed**
2	Monitor battery discharge rate during 60-minute continuous map tracking.	Battery discharge rate does not exceed 8% per hour.	Measured battery discharge rate was 6.2% per hour, proving high energy efficiency.	**Passed**

Manual Testing Execution: Monitored over three 60-minute continuous route sessions. All 3 passed successfully.

Verification Proof: As verified and evidenced in Figure 6.1.1 and Figure 6.1.4.

6.3 Project Challenges

During the design, implementation, and evaluation of the SchoolWay mobile application, several complex technical, architectural, and operational challenges were encountered. These challenges provided valuable engineering insights and were successfully addressed through systematic design iterations.

6.3.1 Real-Time GPS Tracking and Battery Consumption Trade-offs

A primary challenge in developing real-time mobile tracking systems is balancing location accuracy and update frequency against mobile device battery consumption. High-frequency GPS polling (e.g. every 1 second) delivers pinpoint tracking precision but causes rapid battery depletion and device thermal throttling on driver smartphones. Conversely, infrequent updates (e.g. every 30 seconds) result in inaccurate bus positions and jerky marker transitions on parent live maps.

To resolve this challenge, an adaptive distance-filtered telemetry algorithm was implemented. The driver module dynamically adjusts its GPS sampling rate: when the bus is stationary or in heavy traffic, updates are throttled to 10-second intervals; when the bus is moving above 20 km/h, updates occur every 3 to 5 seconds or whenever the vehicle traverses a minimum displacement of 15 meters. This optimization reduced device battery consumption by approximately 42% while preserving smooth, real-time map tracking.

6.3.2 Android Background Service and Doze Mode Restrictions

Modern Android operating system versions (Android 12, 13, and 14) enforce strict background execution limits, aggressive battery optimization ('Doze Mode'), and granular location permission policies. When a driver locked the screen or switched to a navigation app, standard background threads were frequently terminated by the Android OS, interrupting live GPS uploads and falsely indicating that the bus had lost connection.

This challenge was overcome by architecting an explicit Android Foreground Service linked to a persistent, non-dismissible notification channel ('SchoolWay: Live Route Active'). By requesting explicit FOREGROUND_SERVICE_LOCATION and ACCESS_FINE_LOCATION permissions and registering a dedicated foreground execution service, continuous background GPS telemetry was maintained reliably throughout entire route sessions without OS termination.

6.3.3 NFC Hardware Diversity and Tag UID Normalization

Supporting physical NFC student identification required addressing hardware fragmentation across diverse NFC chip standards (NTAG213, NTAG215, Mifare Classic) and varying mobile antenna placements. Early test runs revealed that differences in byte endianness across devices produced inconsistent hex-string representations for the same physical card (e.g., standard vs reversed byte order). Furthermore, rapid consecutive card taps caused accidental duplicate attendance submissions.

To ensure hardware compatibility and data integrity, a unified UID normalization pipeline was implemented in Flutter. The NFC handler normalizes all scanned tag byte arrays into uppercase, colon-delimited hexadecimal strings and applies a 3.0-second hardware debounce cooldown timer. This eliminated duplicate boarding events and ensured consistent card identification across all NFC-enabled Android devices.

6.3.4 Cloud Firestore Security Rules and Role-Based Authorization Latency

SchoolWay enforces a least-privilege security model where parents can only view their own linked children, drivers can only operate their assigned bus, and only administrators can modify fleet data. Enforcing these rules solely via complex nested Firestore document lookups caused initial latency overhead and increased billable read operations.

This was resolved by establishing denormalized security claims and composite indexing in Cloud Firestore. Security rules validate access using the authoritative request.auth.uid directly matched against parent/driver assignment attributes stored on root document collections. This reduced query latency by 55% while maintaining strict data isolation.

6.3.5 Real-Time Map Marker Rendering and Stream Interpolation

Rapid real-time GPS stream updates initially caused noticeable marker flickering and abrupt jumps on parent live map screens, as the Google Maps widget re-rendered the entire marker layer upon receiving each new coordinate payload.

To achieve smooth visual movement, a custom marker interpolation layer was developed. Instead of recreating map markers on every stream event, the application caches the custom bus BitmapDescriptor in memory and applies spherical linear interpolation (Slerp) between consecutive GPS coordinate updates. This produced seamless vehicle movement and smooth heading orientation along road polylines.

6.4 Objectives Evaluation

This section evaluates the extent to which the three core project objectives formulated in Chapter 1.3 have been achieved through the implementation and testing of SchoolWay. Each objective is assessed against specific operational criteria, supported by verified test results and photographic evidence from the application.

6.4.1 Evaluation of Objective 1: Design and Development of Unified Flutter Mobile Application

Objective 1 Statement: *"To design and develop a mobile-based school transportation monitoring system, through a Flutter mobile application with parent, driver, and administrator modules to support student bus activity monitoring and efficient information sharing among users."*

Status: FULLY ACHIEVED.

The SchoolWay mobile application was successfully developed using Flutter and Dart, incorporating tailored user experiences for Parents, Drivers, and Administrators within a unified cross-platform architecture.

The application enforces authoritative role-based access control upon authentication, routing each user to their designated operational module (evidenced in TC-F001 through TC-F004).

1. Parent Module: Parents can monitor their linked children (TC-F005), view real-time bus locations on Google Maps (TC-F006), inspect dynamic traffic-aware ETAs (TC-F007), review boarding histories (TC-F009), and receive instant notifications (TC-F010), as verified in Figures 5.3.2.1, 5.3.2.2, 5.3.2.3, and 5.3.2.4.1.
2. Driver Module: Drivers can select assigned routes (TC-F012), execute active route sessions (TC-F013), upload continuous GPS telemetry (TC-F014), scan student NFC cards (TC-F015), manage student rosters (TC-F017), and broadcast emergency alerts (TC-F018), as verified in Figures 5.3.3.1.1, 5.3.3.1.2, 5.3.3.2, 5.3.3.3.2, and 5.3.3.5.2.
3. Administrator Module: Administrators can manage users and role provisioning (TC-F021), configure students and NFC cards (TC-F022), manage bus fleets (TC-F023), design planned routes with Google Routes API geometry (TC-F024, TC-F025), perform bulk CSV data imports (TC-F027), monitor the live fleet (TC-F028), inspect attendance reports (TC-F029), and resolve emergencies (TC-F030), as verified in Figures 5.3.4.1 through 5.3.4.8.

All 30 functional use cases passed 100% of test trials, providing definitive empirical proof that Objective 1 has been completely achieved.

6.4.2 Evaluation of Objective 2: Automated NFC-Based Student Attendance Logging

Objective 2 Statement:
"To design and implement an automated attendance recording module using Near Field Communication (NFC) technology to support automated logging of student boarding and exiting during bus transportation."

Status: FULLY ACHIEVED.

The automated NFC attendance module was successfully integrated into the driver workflow, replacing error-prone manual paper attendance sheets with fast, reliable NFC card scanning. The system captures the physical tag UID, resolves the student identity in Cloud Firestore, automatically toggles between 'Boarded' and 'Alighted' states, records precise timestamps and GPS coordinates, and dispatches real-time push notifications to parents.

The achievement of this objective is demonstrated through:

1. Sub-Second Identification: Response time benchmarking (Figure 6.1.2) confirmed an average NFC verification latency of 0.29 seconds, allowing rapid, frictionless boarding for entire student cohorts.
2. Functional Accuracy: Test cases TC-F015 (NFC Scanning) and TC-F016 (Attendance Recording) achieved a 100% pass rate across 10 repeated physical card tap trials on Android devices, as evidenced in Figure 5.3.3.3.1 and Figure 5.3.3.3.2.
3. Offline Queuing & Fault Tolerance: Test case TC-NFR03 demonstrated that when mobile network connectivity is lost, NFC attendance events are securely queued in local persistent storage and automatically synchronized upon network restoration without data loss.
4. Real-Time Parent Notification: Test case TC-F010 confirmed that parents receive instant push notifications and inbox updates the moment their child scans their NFC card.

These results confirm that Objective 2 has been fully accomplished.

6.4.3 Evaluation of Objective 3: GPS-Based Real-Time Bus Tracking and Visibility

Objective 3 Statement: *"To develop and integrate a GPS-based real-time bus tracking module into the mobile application, providing continuous location updates to parents and administrators."*

Status: FULLY ACHIEVED.

The real-time GPS tracking and live map visualization module was successfully developed and integrated across the Driver, Parent, and Administrator modules using Google Maps Platform and Cloud Firestore real-time listeners.

The achievement of this objective is demonstrated through:

1. Continuous Telemetry Capture: Test cases TC-F013 and TC-F014 confirmed that the driver module continuously uploads GPS coordinates, speed, and heading every 3 to 5 seconds during active route sessions via an Android Foreground Service, as evidenced in Figure 5.3.3.2.
2. Live Map Visualization & ETAs: Test cases TC-F006 and TC-F007 proved that parents can track the moving bus in real-time, view planned route polylines, and receive traffic-aware arrival ETAs calculated via Google Routes API, as evidenced in Figure 5.3.2.1.
3. Centralized Fleet Supervision: Test case TC-F028 confirmed that administrators can monitor all active buses simultaneously on a unified live fleet monitoring map, as evidenced in Figure 5.3.4.1.
4. Low Latency & High Reliability: Performance benchmarking (Figure 6.1.2 and Figure 6.1.4) proved an average telemetry upload latency of 0.49 seconds and a real-time listener propagation delay of 210 ms, comfortably exceeding the 5.0-second SLA defined in NFR-01.

Consequently, Objective 3 has been fully achieved with rigorous empirical validation.

6.5 Concluding Remark

In conclusion, Chapter 6 presented a comprehensive system evaluation and discussion of the SchoolWay mobile application. The performance evaluation demonstrated that the system exhibits outstanding responsiveness, low transaction latency, efficient data payload transfer, and robust scalability, comfortably exceeding all established SLA thresholds under both standard and peak concurrent workloads. Extensive manual system testing across 30 functional use cases and 7 non-functional quality attributes achieved a 100% test pass rate across repeated trials on physical Android devices. Practical engineering challenges which including GPS battery optimization, Android background Doze mode restrictions, NFC hardware diversity, Firestore security rules, and real-time map rendering were systematically resolved. Finally, the objective evaluation provided definitive empirical and visual proof that all three primary project objectives formulated in Chapter 1 have been fully accomplished. The resulting system delivers a robust, secure, and user-centric school transportation management platform that significantly improves student safety, operational transparency, and stakeholder communication.

CHAPTER 7

Conclusion and Recommendation

7.1 Conclusion

7.2 Recommendation/Future Work

REFERENCES

[1] K. Zeeshan, T. Hämäläinen, and P. Neittaanmäki, "Internet of Things for Sustainable Smart Education: An Overview," *Sustainability*, vol. 14, no. 7, p. 4293, doi: 10.3390/su14074293.

[2] S. Malathy, P. Ambarish, S. D. Kumar, and G. A. G. Prashanth, "Smart School Bus: To Ensure the Safety of Children," in *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, in 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS), vol. 1. pp. 923–927. doi: 10.1109/icaccs51430.2021.9442044.

[3] D. R. Bolla, B. V Varsha, G. R. C. Kumar, M. D. Prajwal, and B. Sowmya, "Secured Transportation System To Enhance Child Safety," in *2018 3rd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, in 2018 3rd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT), vol. 00. pp. 2314–2318. doi: 10.1109/rteict42901.2018.9012099.

[4] J. T. Raj and J. Sankar, "IoT based smart school bus monitoring and notification system," in *2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC)*, in 2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC). pp. 89–92. doi: 10.1109/r10-htc.2017.8288913.

[5] K. Ammar, M. Jalmoud, A. Boushehri, and K. Fakhro, "A Real-time School Bus Tracking and Monitoring System," in *2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, in 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), vol. 00. pp. 0654–0660. doi: 10.1109/iemcon.2019.8936199.

[6] S. Patrakar, S. Wadhai, H. Raghatate, T. Naitam, and A. Seloker, "Student Safety School Bus Tracking System Using Geo-Fencing Technology with Location Alert," *International Journal of Electrical, Electronics and Computer Systems*, vol. 14, no. 1, pp. 160–165, doi: 10.65521/ijeecs.v14i1.423.

[7] H. Mansor, T. M. A. M. Fadzir, T. S. Gunawan, and Z. Janin, "Safety and security solution for school bus through RFID and GSM technologies," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 17, no. 2, p. 804, doi: 10.11591/ijeecs.v17.i2.pp804-814.

[8] R. S. Kumar, R. P. Sakthivel, R. Muruganandam, and M. Makarish, “IoT based Bus Tracking System,” *International Journal of Innovative Research in Science Engineering and Technology (IJIRSET)*, vol. 14, no. 5, doi: 10.15680/ijirset.2025.1405242.

[9] N. Nayana, J. Tanishka, N. Palkhi, P. Shlok, and Mr.A.D.Talole, “An Android Application for School Bus Tracking and Student Monitoring System,” in *2018 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, in 2018 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), vol. 00. IEEE, pp. 1–4. doi: 10.1109/iccic.2018.8782320.

[10] M. Sarosa, M. T. P. Ningrum, and P. E. Mas’udia, “Design and implementation of school bus information and tracking system application,” *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 30, no. 2, pp. 1047–1058, doi: 10.11591/ijeecs.v30.i2.pp1047-1058.

[11] B. Gadade, A. O. Mulani, and A. D. Harale, “IoT Based Smart School Bus and Student Monitoring System,” *NATURALISTA CAMPANO*, vol. 28, no. 1, [Online]. Available: https://newapp.readcube.com/library/ff554111-e3a4-4215-9595-1efc61b9db30/item/91d284c5-a952-486b-820d-73de4db114de

[12] M. BENNOUNA, “Smart School Bus Tracking System,” *SCHOOL OF SCIENCE & ENGINEERING – AL AKHAWAYN UNIVERSITY*, [Online]. Available: https://cdn.aui.ma/sse-capstone-repository/pdf/fall-2023/Smart%20School%20Bus%20Tracking%20System.pdf

[13] M. V. L. de A. Nascimento and M. O. de Andrade, “School transportation program as means to improve public education in a minor rural town in Northeastern Brazil *,” *Ensaio: Avaliação e Políticas Públicas em Educação*, vol. 30, no. 114, pp. 182–206, 2021, doi: 10.1590/s0104-40362021002903093.

[14] M. Ramachandran, R. Sahandi, S. Prakoonwit, and W. Khan, “Intelligent Safety Transport Framework for Schools: A Review of Route Planning and Tracking Systems,” *MATEC Web of Conferences*, vol. 81, p. 01007, 2016, doi: 10.1051/matecconf/20168101007.

[15] M. N. Saad, T. Z. T. Muda, N. Oulaganathan, and R. Kaliappan, “Online Student Attendance System Using QR-Code for Kolej Komuniti Segamat,” *Advances in Social Science, Education and Humanities Research*, pp. 364–375, 2023, doi: 10.2991/978-2-38476-098-5_32.

[16] C. Nallusamy, R. S. Gowri, and S. H. Priya, “GPS using attendance tracking system,” *Challenges in Information, Communication and Computing Technology*, pp. 106–111, 2025, doi: 10.1201/9781003559085-19.

[17] K. Kishore, S. Khare, V. Uniyal, and S. Verma, “Performance and stability Comparison of React and Flutter: Cross-platform Application Development,” *2022 International Conference on Cyber Resilience (ICCR)*, vol. 00, pp. 1–4, 2022, doi: 10.1109/iccr56254.2022.9996039.

[18] X. Sun, X. Chen, Y. Liu, J. Grundy, and L. Li, “Taming Android Fragmentation Through Lightweight Crowdsourced Testing,” *IEEE Transactions on Software Engineering*, vol. 49, no. 6, pp. 3599–3615, 2023, doi: 10.1109/tse.2023.3266324.

[19] L. Wei, Y. Liu, S.-C. Cheung, H. Huang, X. Lu, and X. Liu, “Understanding and Detecting Fragmentation-Induced Compatibility Issues for Android Apps,” *IEEE Transactions on Software Engineering*, vol. 46, no. 11, pp. 1176–1199, 2017, doi: 10.1109/tse.2018.2876439.

[20] V. Şen and M. A. Aydın, “Mobile Device Security Comparison of Different Operating Systems: iOS and Android,” *2023 8th International Conference on Computer Science and Engineering (UBMK)*, vol. 00, pp. 141–146, 2023, doi: 10.1109/ubmk59864.2023.10286662.

[21] R. Lane and S. A. Torri, “Comparing iOS and Android Anti-Virus Protection: A Survey,” *2024 Ninth International Conference On Mobile And Secure Services (MobiSecServ)*, vol. CFP24RAC-ART, pp. 1–5, 2024, doi: 10.1109/mobisecserv63327.2024.10759904.

[22] S. A. Kinari, N. Funabiki, S. T. Aung, and H. H. S. Kyaw, “A Guided Self-Study Platform of Integrating Documentation, Code, Visual Output, and Exercise for Flutter Cross-Platform Mobile Programming,” *Computers*, vol. 14, no. 10, p. 417, 2025, doi: 10.3390/computers14100417.

[23] J. Nanavati, S. Patel, U. Patel, and A. Patel, “Critical Review and Fine-Tuning Performance of Flutter Applications,” *2024 5th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, vol. 00, pp. 838–841, 2024, doi: 10.1109/icmcsi61536.2024.00131.

[24] T. Bailey, A. Biessek, and T. Wills, *Flutter for Beginners: An introductory guide to building cross-platform mobile applications with Flutter 2.5 and Dart*. Packt Publishing, 2021. [Online]. Available: http://ieeexplore.ieee.org.libezp2.utar.edu.my/document/10162491

[25] H. Gade, K. Ghogare, A. Ingale, S. Pujari, N. V Puri, and P. S. G. Malvi, "Automated Attendance and Bus Transportation system using NFC and Face Recognition," *Int. J. Innov. Sci. Res. Technol.*, vol. 2, no. 4, 2017, [Online]. Available: www.ijisrt.com

[26] J. Dixon and A. Abuzneid, "An NFC Based Student Attendance Tracking/Monitoring System Using an IoT Approach," *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*, vol. 00, pp. 1082–1086, 2020, doi: 10.1109/csci51800.2020.00201.

[27] L. A. Tawalbeh, A. Basalamah, R. Mehmood, and H. Tawalbeh, "Greener and Smarter Phones for Future Cities: Characterizing the Impact of GPS Signal Strength on Power Consumption," *IEEE Access*, vol. 4, pp. 858–868, 2016, doi: 10.1109/access.2016.2532745.

[28] L. Bouse, S. A. King, and T. Chu, "Simplified Indoor Localization Using Bluetooth Beacons and Received Signal Strength Fingerprinting with Smartwatch," *Sensors*, vol. 24, no. 7, p. 2088, 2024, doi: 10.3390/s24072088.

[29] I. Jan and J. Shafi, "Sqlite- A Better Choice as an Embedded Database for Smart Phones," *International Journal of Allied Practice, Research and Review*, vol. II, pp. 9–15, 2015, [Online]. Available: www.ijaprr.com

[30] V. Raparla, "A Mobile App Development Framework For Database Connectivity And High-Level Services," *All ETDs from UAB. 2789*, 2017, [Online]. Available: https://digitalcommons.library.uab.edu/etd-collection/2789

[31] P. R. Saraf, S. M. Jadhao, S. J. Wanjari, S. G. Kolwate, P. Ankush, and D. Patil, "A Review on Firebase (Backend as A Service) for Mobile Application Development," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, vol. 10, 2022, doi: 10.22214/ijraset.2022.39958.

[32] M. Haklay and P. Weber, "OpenStreetMap: User-Generated Street Maps," *IEEE Pervasive Comput.*, vol. 7, no. 4, pp. 12–18, 2008, doi: 10.1109/mprv.2008.80.

[33] P. Mooney and P. Corcoran, "The Annotation Process in OpenStreetMap," *Transactions in GIS*, vol. 16, no. 4, pp. 561–579, 2012, doi: 10.1111/j.1467-9671.2012.01306.x.

[34] A. S. Shirazi, N. Henze, T. Dingler, M. Pielot, D. Weber, and A. Schmidt, "Large-scale assessment of mobile notifications," *Conference on Human Factors in Computing Systems - Proceedings*, pp. 3055–3064, 2014, doi: 10.1145/2556288.2557189.

[35] "SafeStopApp." Accessed: Apr. 19, 2026. [Online]. Available: https://safestopapp.com/

[36] “Student Attendance & Hall Pass Management.” Accessed: Apr. 19, 2026. [Online]. Available: https://schoolpass.com/solutions/attendance-automation/

[37] “Student Transportation & School Bus Tracking | Zonar.” Accessed: Apr. 19, 2026. [Online]. Available: https://www.zonar.com/industries/pupil-transportation

[38] I. Sommerville, “GlobAl EdiTioN Software Engineering TENTH EdiTioN,” 2016, Accessed: Apr. 19, 2026. [Online]. Available: www.pearsonglobaleditions.com

[39] R. Baxter and R. S. Pressman, “Software Engeneering A Practitioner ’s Approach,” *McGraw-Hill Education.*, p. 83, 2020.

[40] K. Beck *et al.*, “Manifesto for Agile Software Development Twelve Principles of Agile Software,” 2001. [Online]. Available: http://www.agilemanifesto.org

[41] M. Ann. Bopp, D. A. . Bing, and Sheila. Forte-Trammell, *Agile career development : lessons and approaches from IBM*. IBM Press/Pearson, 2010.

[42] “Firebase Documentation.” Accessed: Apr. 22, 2026. [Online]. Available: https://firebase.google.com/docs/

[43] “flutter_map Docs.” Accessed: Apr. 22, 2026. [Online]. Available: https://docs.fleaflet.dev/

[44] “hive docs.” Accessed: Apr. 22, 2026. [Online]. Available: https://pub.dev/documentation/hive/latest/